

EMPLOYEE CONSOLE

คู่มือเรียนรู้ฉบับมือใหม่

NestJS · Prisma · OpenAPI · Docker · Jenkins · การทดสอบ · เตรียมตอบคำถาม

ที่มา	docs/learn/ (12 ไฟล์)
Commit	3793181
สร้างเมื่อ	4 ตุลาคม 2026

ฉบับ PDF แปลงจาก Markdown — ถ้าเนื้อหาไม่ตรงกับไฟล์ใน repo ให้ยึดไฟล์ใน repo

คู่มือเรียนรู้ Employee Console (ฉบับมือใหม่)

คู่มือชุดนี้เขียนให้คนที่ **ยังไม่เคยเขียน NestJS และไม่รู้จักร OpenAPI หรือ Jenkins** อ่านแล้วเข้าใจว่าโปรเจกต์นี้ทำงานอย่างไร ทุกคนจะอธิบายด้วยภาพก่อน แล้วค่อยชี้ไปที่ไฟล์จริงในโปรเจกต์ เปิดโค้ดตามไปได้ทันที

เอกสารเดิมใน `docs/` (architecture, runbook, decisions ฯลฯ) ยังเป็นฉบับอ้างอิงที่ละเอียดและแม่นยำที่สุด คู่มือชุดนี้เป็นฉบับอ่านง่ายที่พาไปถึงเอกสารเหล่านั้น ถ้าสองที่พูดไม่ตรงกัน ให้ยึดโค้ดกับเอกสารเดิมไว้ก่อน

ลำดับการอ่าน

#	บท	อ่านจบแล้วจะเข้าใจ
1	ภาพรวมทั้งระบบ (docs/learn/01-big-picture.md)	ชิ้นส่วนแต่ละตัวคืออะไร คอยกันอย่างไร พร้อมตามรอยการ "แก้เงินเดือน Bob" ตั้งแต่กดปุ่มไปจนถึงฐานข้อมูล
2	NestJS สำหรับมือใหม่ (docs/learn/02-nestjs.md)	Module, Controller, Service, Dependency Injection, Pipe, Guard, Interceptor และ Filter ผ่านโค้ด <code>employees</code> ของจริง
3	ฐานข้อมูลและ Prisma (docs/learn/03-database-prisma.md)	ตาราง, migration, seed และเหตุผลที่ใช้ SQL ตรง ๆ
4	OpenAPI (docs/learn/04-openapi.md)	"สัญญา" ระหว่าง API กับเว็บ และสิ่งที่เกิดขึ้นเมื่อรัน <code>pnpm openapi:generate</code>
5	Docker (docs/learn/05-docker.md)	image, container, volume, compose และวิธีประกอบ staging
6	Jenkins และ CI/CD (docs/learn/06-jenkins.md)	Jenkinsfile ที่ละ stage, controller กับ agent, การ deploy และ rollback
7	แนวคิดสำคัญของโปรเจกต์ (docs/learn/07-key-concepts.md)	<code>If-Match /version</code> , <code>Idempotency-Key</code> , เงินที่เป็น string, วันที่ที่ไม่มีเวลา และรูปแบบ error
8	การทดสอบ (docs/learn/08-testing.md)	unit, integration, Postman และ E2E ต่างกันอย่างไร
9	แผนที่เอกสารใน <code>docs/</code> (docs/learn/09-docs-guide.md)	เอกสารแต่ละไฟล์บอกอะไร ควรอ่านตอนไหน (สรุปแบบอ่านง่าย)
10	เตรียมตอบคำถาม (docs/learn/10-qa-prep.md)	คำถามที่น่าจะโดนถามตอนนำเสนอ พร้อมคำตอบที่ชี้ไปที่โค้ด
—	อภิธานศัพท์ (docs/learn/glossary.md)	คำศัพท์ทั้งหมดในที่เดียว เปิดดูได้เมื่อสงสัย

ถ้ามีเวลาน้อย ให้อ่านบท 1 → 2 → 7 ก่อน สามบทนี้ครอบคลุมสิ่งที่คนถามบ่อยที่สุด

สัญลักษณ์ในคู่มือ

- **ลองเอง** — คำสั่งที่รันได้จริงบนเครื่อง (ต้อง `pnpm dev:up` และ `pnpm dev` ไว้ก่อน) แต่ละบล็อกมีคำสั่งเดียว
- **กับดัก** — จุดที่มือใหม่ (และ AI) พลาดบ่อยในโปรเจกต์นี้
- **ชื่อไฟล์โค้ด** แสดงเป็น path จาก root ของ repo เช่น `apps/api/src/employees/employees.controller.ts`

บท 1 – ภาพรวมทั้งระบบ

โปรเจกต์นี้คืออะไร

Employee Console เป็นเว็บทะเบียนพนักงาน ข้อมูลตั้งต้นมาจากไฟล์ Excel ของโจทก์ (test-exam-data.xlsx (apps/api/prisma/seed-data/test-exam-data.xlsx)) มีพนักงาน 5 คน รหัส 101–105 แต่ละคนมี 7 ฟิลด์ ได้แก่ ID, Name, Department, Salary, Join Date, Status และ Last Updated Date

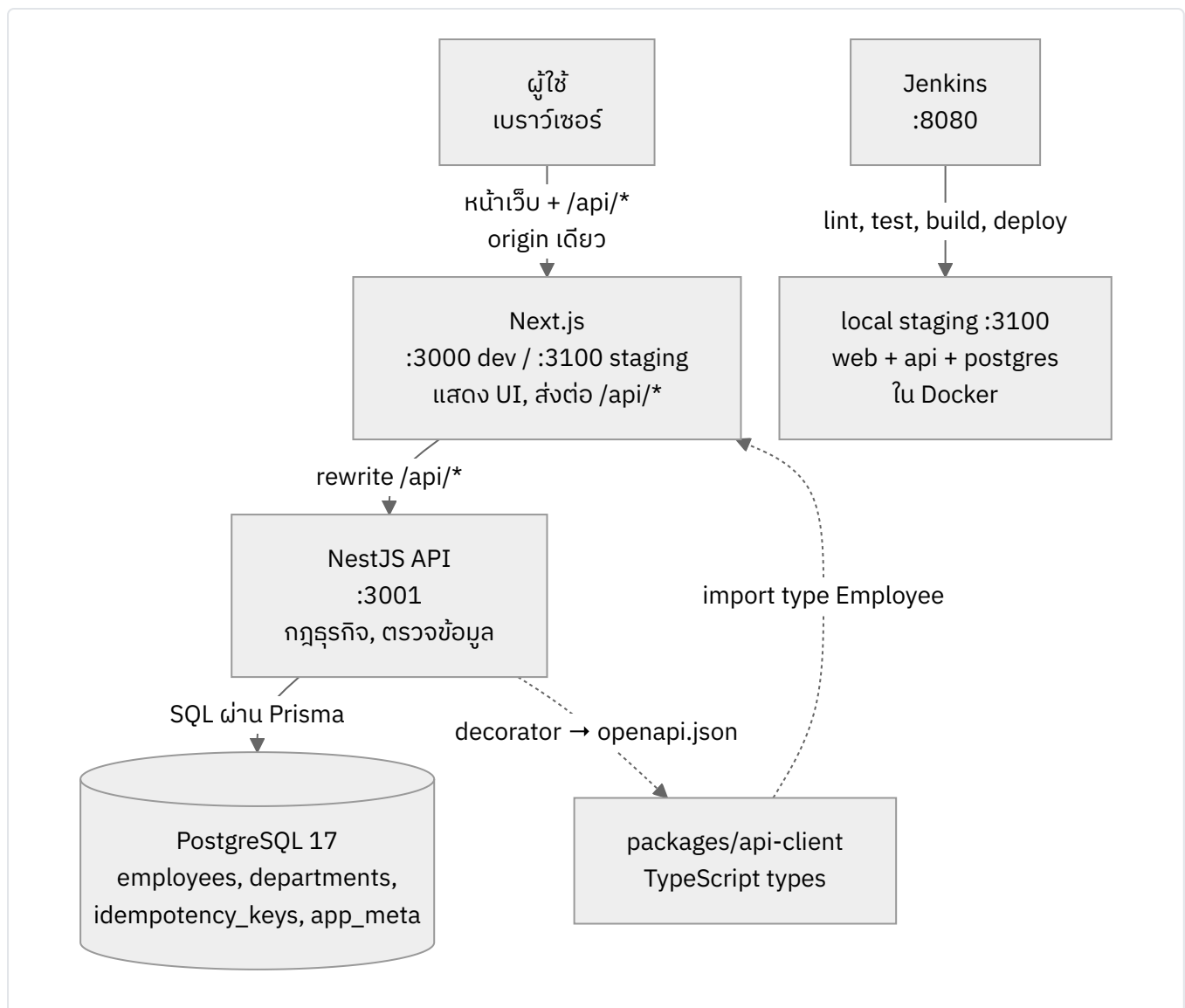
สิ่งที่ผู้ใช้งานทำได้คือ ดูรายการ, ค้นหาชื่อ, กรองตามแผนกหรือสถานะ, เรียงลำดับ, แบ่งหน้า และ เพิ่ม/แก้/ลบ พนักงาน (CRUD) ไม่มีระบบ login ใครเข้าเว็บได้ก็ใช้งานได้ทุกอย่าง ซึ่งเป็นการตัดสินใจโดยตั้งใจ (D-46)

เปรียบเทียบกับร้านอาหาร

ถ้ายังนึกภาพไม่ออกว่าแต่ละชิ้นทำอะไร ลองนึกถึงร้านอาหาร

ในร้านอาหาร	ในโปรเจกต์	อยู่ที่
ลูกค้า	เบราว์เซอร์ของผู้ใช้	—
หน้าร้านกับพนักงานเสิร์ฟ: จัดโต๊ะ รับออเดอร์ แล้วเดินไปส่งครัว	Next.js — แสดงหน้าเว็บ และส่งต่อคำขอ <code>/api/*</code> ไปให้ API	apps/web
ครัว: รู้สูตร ตรวจสอบวัตถุดิบ ตัดสินว่าทำได้หรือไม่	NestJS — กฎธุรกิจทั้งหมด ตรวจสอบข้อมูล ตัดสินว่าบันทึกได้หรือไม่	apps/api
ห้องเก็บของกับสมุดบัญชี	PostgreSQL — เก็บข้อมูลจริง	Docker container
เมนูที่เขียนชัดว่าสั่งอะไรได้และจะได้อะไรกลับ	OpenAPI — เอกสารสัญญาของ API ที่เว็บใช้อ้างอิง	packages/api-client
ผู้ตรวจสอบคุณภาพกับรถส่งของ: ตรวจสอบทุกจานก่อนออกจากครัวแล้วส่งไปสาขา	Jenkins — ตรวจสอบโค้ดทุกครั้ง แล้ว deploy ไป staging	Jenkinsfile
กล่องข้าวสำเร็จรูปที่อุ่นแล้วกินได้เหมือนกันทุกที่	Docker image — แอปที่แพ็คเกจพร้อมรัน	infra/docker

ประเด็นสำคัญคือ **พนักงานเสิร์ฟไม่ทำอาหารเอง** Next.js ไม่ต่อฐานข้อมูลและไม่มีกฎธุรกิจ ทุกอย่างที่เป็น "การตัดสินใจ" อยู่ใน NestJS ที่เดียว



- เส้นทึบ = คำขอที่วิ่งตอนใช้งานจริง
- เส้นประ = ความสัมพันธ์ตอนพัฒนา (types ถูก generate จาก API แล้วเว็บ import ไปใช้)

ทำไมเบราว์เซอร์ไม่คุยกับ API ตรงๆ

เบราว์เซอร์รู้จักแค่ `http://localhost:3000` (dev) หรือ `http://localhost:3100` (staging) เมื่อหน้าเว็บเรียก `fetch('/api/v1/employees')` Next.js จะส่งต่อคำขอนั้นไปที่ NestJS ให้เอง ตามกฎใน `next.config.ts` (`apps/web/next.config.ts`)

```

async rewrites() {
  return [{ source: '/api/:path*', destination: `${apiInternalUrl}/api/:path*` }];
}

```

ผลที่ได้คือ

1. ไม่ต้องเปิด CORS เพราะเบราว์เซอร์คุยกับ origin เดียว
2. บน staging พอร์ตของ API ไม่ต้องเปิดออกนอก Docker เลย (`compose.staging.yaml` publish แค่พอร์ตของ web)

พื้นฐานที่ควรรู้ก่อนอ่านบทต่อไป

HTTP request และ response

ทุกครั้งที่หน้าเว็บขอข้อมูล จะส่ง **request** ไปหนึ่งก่อน แล้วได้ **response** กลับมาหนึ่งก่อน

```
PATCH /api/v1/employees/104      ← method + path
If-Match: "1"                    ← header (ข้อมูลประกอบ)
Content-Type: application/json    ← บรรทัดว่างคั่น
{ "salary": "75000.00" }         ← body (เนื้อหา JSON)
```

```
HTTP/1.1 200 OK                  ← status code
ETag: "2"                        ← header ที่ server ส่งกลับ
X-Request-Id: 3f1c...

{ "data": { "id": 104, ... "version": 2 }, "meta": { "requestId": "3f1c...", "changed": true } }
```

Method ที่โปรเจกต์ใช้

API นี้ออกแบบแบบ REST คือมองข้อมูลเป็น "resource" ที่มี URL ของตัวเอง แล้วใช้ method บอกว่าจะทำอะไรกับมัน

Method	ความหมาย	ตัวอย่างในโปรเจกต์
GET	อ่าน	GET /api/v1/employees?q=john = ค้นหา, GET /api/v1/employees/104 = ดูคนเดียว
POST	สร้างใหม่	POST /api/v1/employees + header Idempotency-Key
PATCH	แก้บางฟิลด์	PATCH /api/v1/employees/104 + header If-Match
DELETE	ลบ	DELETE /api/v1/employees/104 + header If-Match

Status code ที่จะเจอบ่อย

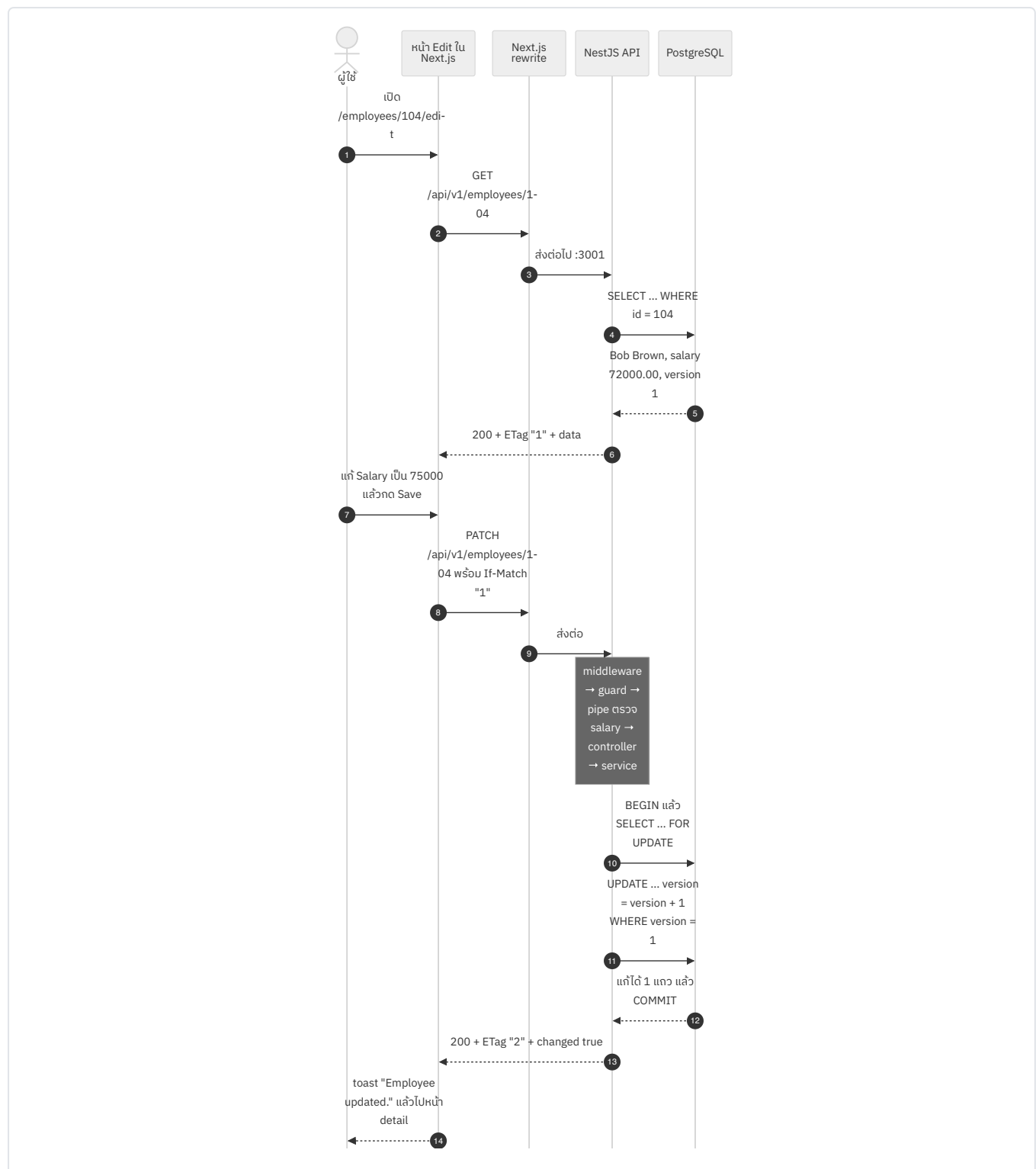
Code	ความหมาย	เกิดเมื่อไรในโปรเจกต์
200	สำเร็จ	อ่านหรือแก้สำเร็จ
201	สร้างสำเร็จ	POST สำเร็จ
204	สำเร็จแต่ไม่มีเนื้อหา	ลบสำเร็จ
400	ข้อมูลที่ส่งมาผิด	ชื่อว่าง, salary มีทศนิยม 3 ตำแหน่ง
404	ไม่พบ	ไม่มีพนักงาน ID นี้
409	ชนกัน	มีคนแก้ record นี้ไปก่อน (version ไม่ตรง)
415	ส่ง body ผิดชนิด	ไม่ได้ส่งเป็น application/json
428	ขาดเงื่อนไขที่บังคับ	แก้หรือลบโดยไม่ส่ง If-Match
429	ขอถี่เกินไป	เกิน rate limit ต่อ IP
503	ระบบยังไม่พร้อม	ฐานข้อมูลล่ม

คำอื่นที่จะเจอ

- **JSON** — รูปแบบข้อมูลที่ส่งไปมา หน้าตาเหมือน object ของ JavaScript
- **TypeScript** — JavaScript ที่มีชนิดข้อมูล (type) ทั้งเว็บและ API เขียนด้วยภาษานี้
- **Decorator** — ป้ายที่แปะไว้บน class หรือ method เช่น `@Get()` เพื่อบอก framework ว่าโค้ดนี้คืออะไร (อธิบายละเอียดในบท 2)
- **Monorepo** — repo เดียวที่มีหลายโปรเจกต์ย่อย (`apps/api` , `apps/web` , `packages/api-client` , `tests/e2e`) จัดการด้วย pnpm workspace (pnpm-workspace.yaml)

ตามรอย: แก้เงินเดือน Bob Brown จาก 72,000 เป็น 75,000

ตัวอย่างนี้ร้อยทุกชั้นเข้าด้วยกัน อ่านคู่กับโค้ดจะเห็นภาพชัดที่สุด



ขั้น	เกิดอะไรขึ้น	ไฟล์
1-2	หน้า Edit โหลดข้อมูลด้วย hook <code>useEmployee(104, { editing: true })</code>	<code>edit-employee-view.tsx</code> (<code>apps/web/src/app/(console)/employees/%5Bid%5D/edit/edit-employee-view.tsx</code>), <code>queries.ts</code> (<code>apps/web/src/lib/queries.ts</code>)
3	Next.js ส่งต่อ <code>/api/*</code> ไปที่ <code>API_INTERNAL_URL</code>	<code>next.config.ts</code> (<code>apps/web/next.config.ts</code>)
4-6	Controller เรียก service ให้อ่านแอดนั้น แล้วใส่ header <code>ETag: "1"</code> (เลข version)	<code>employees.controller.ts</code> (<code>apps/api/src/employees/employees.controller.ts</code>)
7	ฟังก์ชัน <code>diff()</code> เทียบค่าในฟอร์มกับค่าเดิม แล้วส่งเฉพาะฟิลด์ที่เปลี่ยน คือ <code>{ salary }</code>	<code>edit-employee-view.tsx</code>
8	<code>useUpdateEmployee</code> เรียก <code>api()</code> พร้อม header <code>If-Match: "1"</code>	<code>queries.ts</code> (<code>apps/web/src/lib/queries.ts</code>), <code>api.ts</code> (<code>apps/web/src/lib/api.ts</code>)
10	NestJS ตรวจสอบ rate limit, ตรวจสอบว่า salary เป็นเลขทศนิยมแบบ string ที่ถูกต้อง แล้วเรียก <code>update()</code>	<code>rate-limit.ts</code> (<code>apps/api/src/rate-limit/rate-limit.ts</code>), <code>employee-rules.ts</code> (<code>apps/api/src/employees/employee-rules.ts</code>)
11-13	Service ล็อกแถว ตรวจสอบ version แล้ว UPDATE พร้อมเพิ่ม version และตั้ง Last Updated Date เป็น "วันนี้" ตามเวลากรุงเทพ	<code>employees.service.ts</code> (<code>apps/api/src/employees/employees.service.ts</code>)
14	<code>EnvelopeInterceptor</code> ห่อผลลัพธ์เป็น <code>{ data, meta }</code>	<code>envelope.interceptor.ts</code> (<code>apps/api/src/common/envelope.interceptor.ts</code>)
15	เว็บอัปเดต cache ของ TanStack Query แสดง toast แล้วเปลี่ยนหน้า	<code>edit-employee-view.tsx</code>

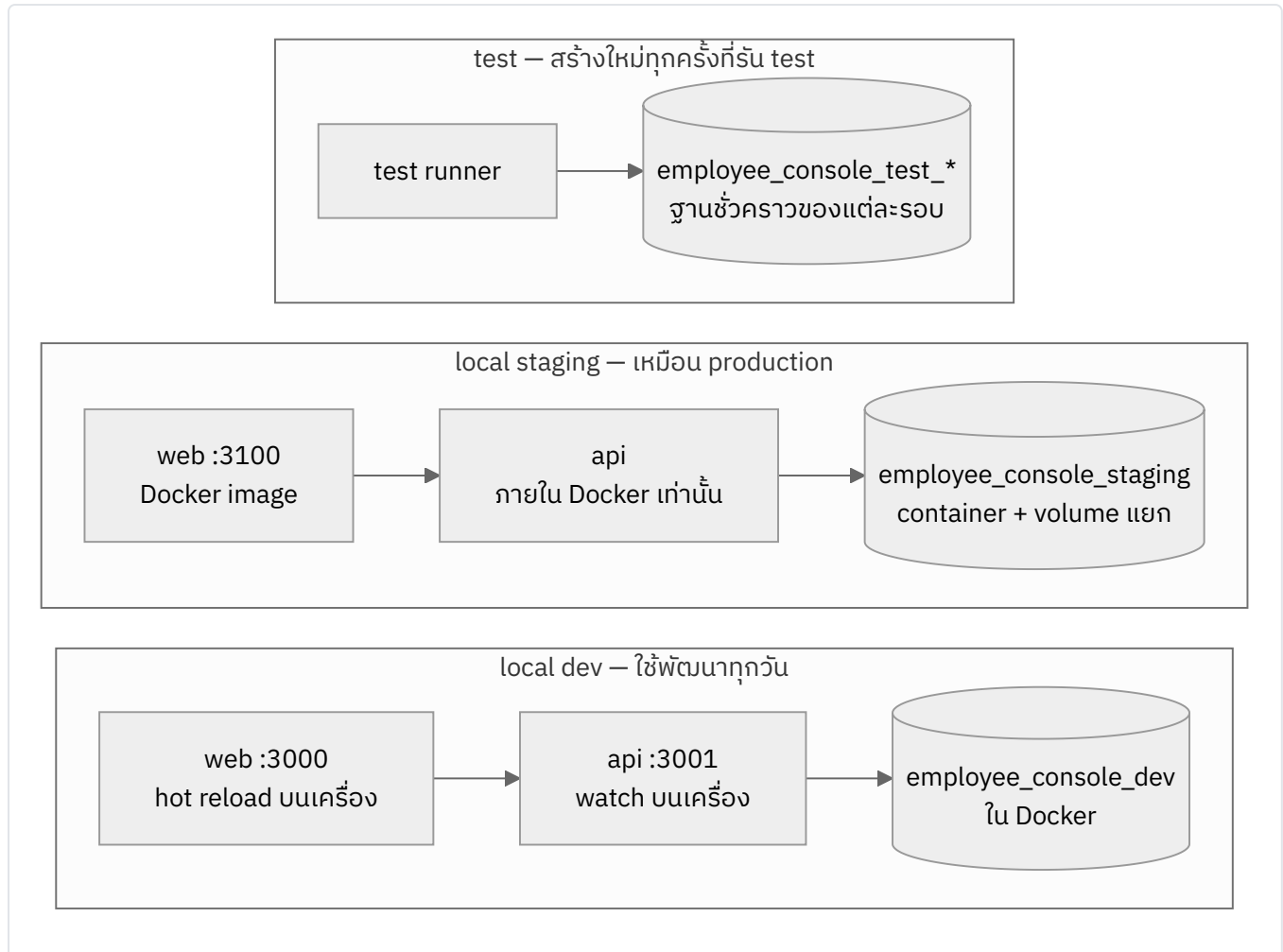
ถ้าระหว่างนั้นมีอีกแท็บเก่า Bob ไปก่อน version ในฐานข้อมูลจะกลายเป็น 2 แล้ว ขั้นที่ 11 จะพบว่า version ไม่ตรง และตอบ **409 VERSION_CONFLICT** แทน หน้าเว็บจะขึ้นข้อความ "This employee was changed by another user. Reload the latest version." (รายละเอียดในบท 7)

โครงสร้าง folder แบบ "อยากทำอะไร ไปที่ไหน"

อยากแก้	ไปที่
หน้าตา ปุ่ม ตาราง ฟอรัม	apps/web/src/components
route ของหน้าเว็บ (<code>/employees</code> , <code>/employees/new</code> ฯลฯ)	apps/web/src/app
สถานะของหน้า list ใน URL (ค้นหา, กรอง, หน้า)	list-params.ts (apps/web/src/lib/list-params.ts)
endpoint ของ API	apps/api/src/<feature>/*.controller.ts
กฎของแต่ละฟิลด์ (ชื่อยาวได้เท่าไร, salary รับรูปแบบไหน)	employee-rules.ts (apps/api/src/employees/employee-rules.ts)
กฎของการค้นหา เรียง และแบ่งหน้า	employee-query.ts (apps/api/src/employees/employee-query.ts)
SQL ที่อ่านหรือเขียนพนักงาน	employees.service.ts (apps/api/src/employees/employees.service.ts)
ตารางในฐานข้อมูล	schema.prisma (apps/api/prisma/schema.prisma) + migration ใหม่
environment variable	app-config.ts (apps/api/src/config/app-config.ts) + .env.example + configuration.md (docs/configuration.md)
ขั้นตอน CI/CD	Jenkinsfile
วิธี deploy staging	scripts/staging.mjs
test ผ่านเบราว์เซอร์	tests/e2e/specs

Environment ทั้ง 3 แบบ

ระบบเดียวกันรันได้หลาย "สภาพแวดล้อม" แยกกันด้วยตัวแปร `APP_ENV` และแต่ละแบบมีฐานข้อมูลของตัวเอง ข้อมูลจึงไม่ปนกัน



คำสั่งที่ใช้ทุกวัน

ติดตั้งครั้งแรก

```
pnpm install
```

```
pnpm run setup
```

เปิดฐานข้อมูล (และ migrate + seed ในครั้งแรก)

```
pnpm dev:up
```

รันเว็บกับ API แบบ hot reload แล้วเปิด <http://localhost:3000>

```
pnpm dev
```

ปิด container ทั้งหมด (ข้อมูลยังอยู่)

```
pnpm down
```

กับดัก: ต้องพิมพ์ `pnpm run setup` และ `pnpm run doctor` (มีคำว่า `run`) เพราะ pnpm มีคำสั่งในตัวชื่อเดียวกัน (D-34)

ต่อไป: un 2 – NestJS สำหรับมือใหม่ (docs/learn/02-nestjs.md)

บท 2 – NestJS สำหรับมือใหม่

บทนี้สอน NestJS ผ่านโค้ดจริงของโปรเจกต์ เริ่มจาก controller ที่เล็กที่สุดก่อน แล้วค่อยไล่ไปจนเห็นการทำงานของคำขอหนึ่งคำขอตั้งแต่เข้าจนออก

NestJS คืออะไร

NestJS เป็น framework สำหรับเขียน backend ด้วย TypeScript ข้างในยังใช้ **Express** (ตัวรับ HTTP ที่นิยมที่สุดของ Node.js) แต่ Nest เพิ่ม "โครง" ให้ว่าโค้ดแต่ละประเภทต้องอยู่ที่ไหน

ถ้าเขียน Express เปล่า ๆ ทุกอย่างมักไปกองรวมในฟังก์ชันเดียว ทั้งรับ request, ตรวจสอบข้อมูล, คุยฐานข้อมูล และจัดการ error พอโปรเจกต์โตก็หาอะไรไม่เจอ Nest จึงแยกงานเป็นชั้น ๆ และให้แต่ละชั้นมีหน้าที่เดียว

คำศัพท์ที่ต้องรู้

คำ	เปรียบเทียบ	ในโปรเจกต์นี้
Module	แผนกในบริษัท รวมคนที่ทำงานเรื่องเดียวกัน	EmployeesModule , DepartmentsModule , HealthModule
Controller	พนักงานต้อนรับ รับเรื่อง แล้วส่งต่อให้คนที่ทำจริง	EmployeesController รับ GET/POST/PATCH/DELETE
Service (Provider ชนิดหนึ่ง)	ผู้เชี่ยวชาญที่ลงมือทำงานจริง	EmployeesService เขียน SQL และกฎทางธุรกิจ
Dependency Injection (DI)	ฝ่ายบุคคลจัดคนมาให้ ไม่ต้องไปจ้างเอง	Nest สร้าง PrismaService แล้วส่งให้ EmployeesService อัตโนมัติ
Middleware	รปภ. หน้าตึก ทำกับทุกคนที่เดินเข้ามา	ใส่ request ID, security headers, ตรวจสอบ Content-Type
Guard	คนเฝ้าประตูห้อง ตัดสินว่าให้เข้าหรือไม่	RateLimitGuard ปฏิเสธเมื่อขอถี่เกินไป
Pipe	ด่านตรวจสอบเอกสาร ตรวจสอบและแปลงข้อมูลก่อนถึงมือคนทำงาน	bodyValidationPipe ตรวจสอบ body ตาม DTO
Interceptor	คนห่อของขวัญ ห่อผลลัพธ์ก่อนส่งออก	EnvelopeInterceptor ห่อเป็น { data, meta }
Exception Filter	ฝ่ายรับเรื่องร้องเรียน แปลงทุกปัญหาเป็นคำตอบมาตรฐาน	HttpExceptionFilter แปลง error เป็น { error: { ... } }

Provider คือของทุกอย่างที่ Nest สร้างแล้วแจกจ่ายผ่าน DI ได้ Service เป็น provider ที่พบบ่อยที่สุด แต่ค่า config, Clock และ guard ก็เป็น provider เหมือนกัน

Decorator คืออะไร

Decorator คือ **ป้ายที่ขึ้นต้นด้วย @** แปะไว้บน class, method, parameter หรือ property เพื่อบอก Nest ว่าโค้ดนั้นมีหน้าที่อะไร ตัว decorator เองไม่ได้ทำงาน แต่ Nest จะอ่านป้ายเหล่านี้ตอนเริ่มโปรแกรม แล้วต่อสายทุกอย่างให้

ดูตัวอย่างที่เล็กที่สุดในโปรเจกต์ departments.controller.ts

(apps/api/src/departments/departments.controller.ts)

```
@ApiTags('departments') // ① ป้ายสำหรับ OpenAPI: จัดกลุ่มว่า "departments"
@Controller('api/v1/departments') // ② class นี้เป็น controller ของ path
/api/v1/departments
export class DepartmentsController {
  constructor(private readonly prisma: PrismaService) {} // ③ ขอ PrismaService ผ่าน DI

  @Get() // ④ method นี้ตอบ GET /api/v1/departments
  @ApiResponse({ status: 200, type: DepartmentDto[] }) // ⑤ ป้าย OpenAPI: ตอบเป็น { data:
  DepartmentDto[], meta }
  @ApiErrors(429) // ⑥ ป้าย OpenAPI: อาจตอบ 429
  async list() {
    const rows = await this.prisma.department.findMany({ orderBy: { sortOrder: 'asc' } });
    return respond(rows.map((d) => ({ id: d.id, name: d.name, sortOrder: d.sortOrder })));
  }
}
```

ทั้งไฟล์มีแค่นี้ แต่ได้ endpoint ที่ใช้งานได้จริงหนึ่งตัว

- ② + ④ รวมกันเป็น route `GET /api/v1/departments`
- ③ คือ DI: เราไม่ได้เขียน `new PrismaService()` เอง Nest ส่งมาให้
- `respond(...)` สร้าง `ApiResponse` แล้ว interceptor จะห่อเป็น `{ data, meta }` ให้ภายหลัง
- ① ⑤ ⑥ ไม่มีผลกับการทำงาน มีไว้ให้ `pnpm openapi:generate` อ่านไปสร้างเอกสาร (un 4)

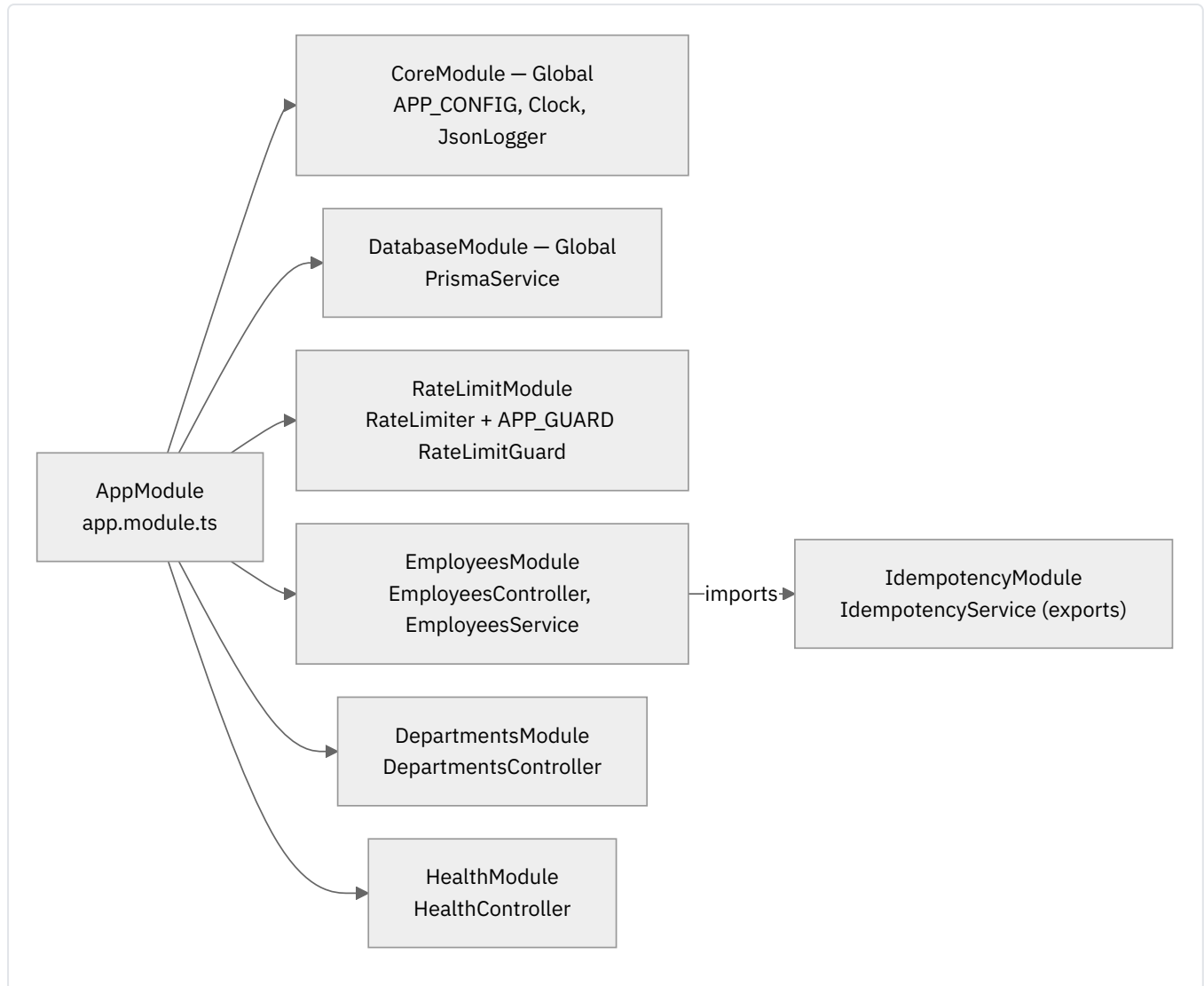
Decorator ที่จะเจอใน controller ของ employees

Decorator	อ่านอะไรจาก request	ตัวอย่าง
<code>@Param('id')</code>	ค่าใน path	<code>/api/v1/employees/104</code> → <code>"104"</code>
<code>@Query()</code>	query string ทั้งหมด	<code>?q=john&page=2</code> → <code>{ q: 'john', page: '2' }</code>
<code>@Body()</code>	JSON body	<code>{ "salary": "75000.00" }</code>
<code>@Headers('if-match')</code>	header หนึ่งตัว	<code>"1"</code>
<code>@Res({ passthrough: true })</code>	object response ของ Express	ใช้ตั้ง header เช่น <code>ETag</code> แล้วยังให้ Nest ส่งค่าที่ return ตามปกติ

กับดัก: ค่าจาก `@Param` และ `@Query` เป็น **string เสมอ** โปรเจกต์นี้ตั้งใจปิดการแปลงชนิดอัตโนมัติ (`transform: false`) เพื่อไม่ให้ `"false"` กลายเป็น boolean โดยไม่ตั้งใจ จึงต้องแปลงเองผ่าน rule เช่น `pageRule`

Module — กล้องที่รวมของที่เกี่ยวข้องกัน

ทุก controller และ service ต้องอยู่ใน module ใด module หนึ่ง และ module ทั้งหมดถูกรวมไว้ที่ `AppModule` ซึ่งเป็นรากของต้นไม้



ดูโค้ดของ `employees.module.ts` (`apps/api/src/employees/employees.module.ts`)

```
@Module({
  imports: [IdempotencyModule],      // ขอใช้ของที่ IdempotencyModule export ไว้
  controllers: [EmployeesController], // controller ของ module นี้
  providers: [EmployeesService],     // service ที่ Nest จะสร้างและแจกจ่าย
})
export class EmployeesModule {}
```

- **imports** — ขอหยิบของจาก module อื่น ได้เฉพาะของที่ module นั้นใส่ไว้ใน **exports**
- **@Global()** — `CoreModule` และ `DatabaseModule` ประกาศเป็น global ทุก module จึงใช้ `PrismaService`, `Clock` และ config ได้โดยไม่ต้อง import (เพราะแทบทุกที่ที่ต้องใช้)
- **AppModule.register(config, clock, logger)** ใน `app.module.ts` (`apps/api/src/app.module.ts`) เป็น "dynamic module" คือ module ที่รับค่าตอนสร้าง ทำแบบนี้เพื่อให้ตอน test ส่ง `FakeClock` เข้าไปแทนนาฬิกาจริงได้

กฎของโปรเจกต์ (จาก apps/api/AGENTS.md): feature ใหม่ = folder ของตัวเองที่มี `<name>.module.ts` + controller + service แล้วเพิ่มเข้า `AppModule` และไฟล์ module ต้องแยกจากไฟล์ controller เสมอ

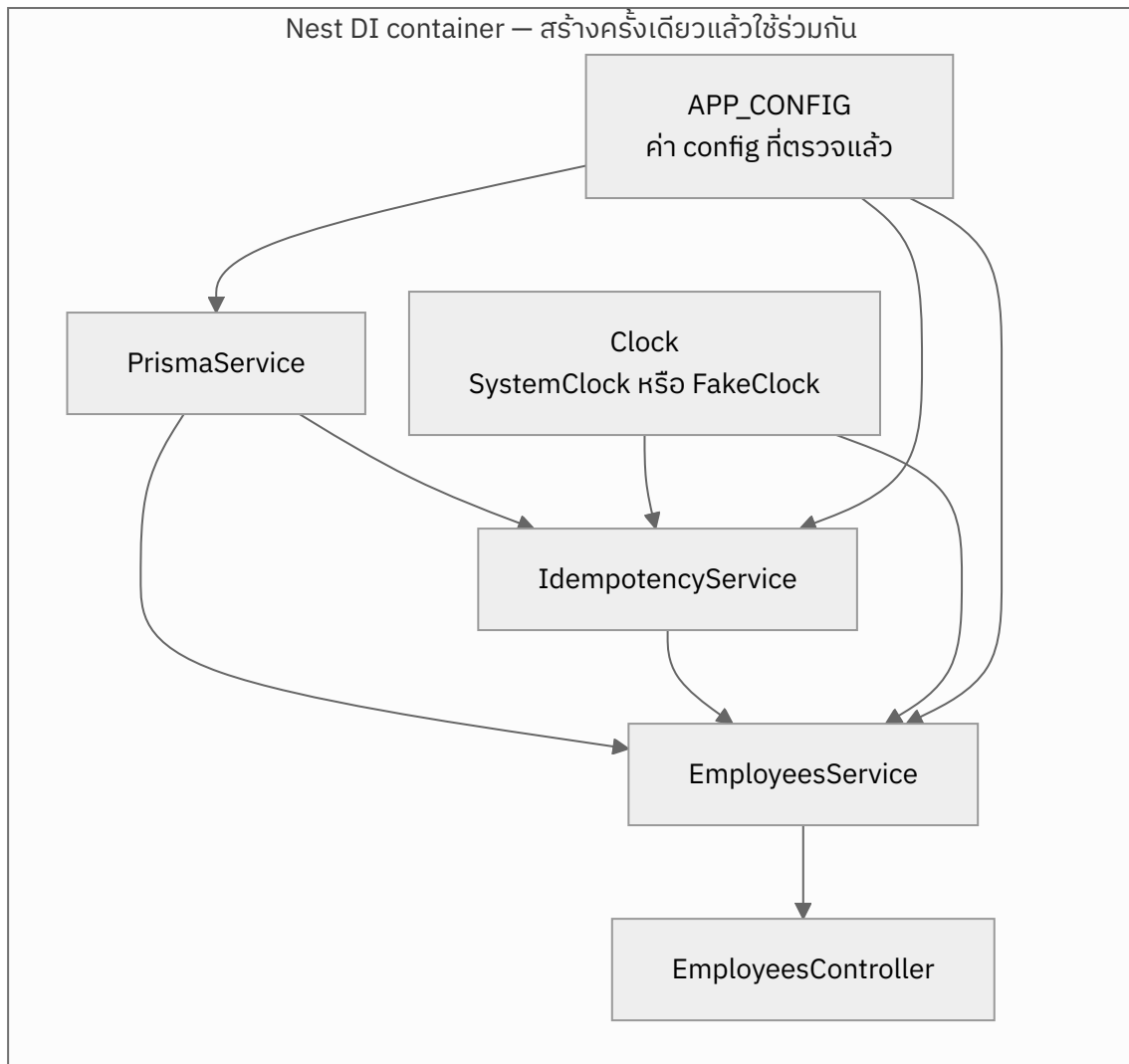
Dependency Injection – ไม่ต้อง new เอง

ดู constructor ของ `EmployeesService` ใน `employees.service.ts`

(apps/api/src/employees/employees.service.ts)

```
@Injectable() // บอก Nest ว่า class นี้ให้ DI สร้างได้
export class EmployeesService {
  constructor(
    private readonly prisma: PrismaService, // ขอดด้วย "ชนิท" ของ class
    private readonly idempotency: IdempotencyService,
    private readonly clock: Clock,
    @Inject(APP_CONFIG) private readonly config: AppConfig, // ขอดด้วย "token" เพราะ AppConfig
    // เป็นแค่ interface
  ) {}
}
```

Service ไม่รู้เลยว่าของพวกนี้ถูกสร้างอย่างไร แค่ประกาศว่า "ต้องใช้" ตอนแอปเริ่ม Nest จะไล่ดูว่าใครต้องใช้อะไร แล้วสร้างให้ตามลำดับ



ลูกศร A → B หมายถึง "B ต้องใช้ A"

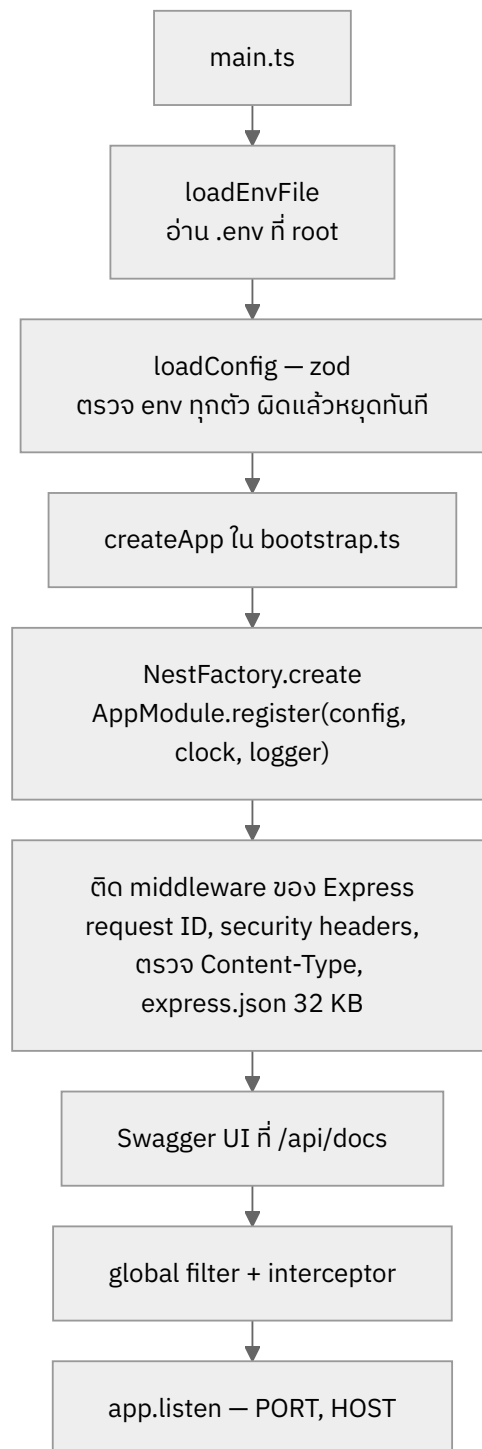
ทำไมต้องลำบากขนาดนี้

ประโยชน์ที่เห็นชัดที่สุดในโปรเจกต์นี้คือ **การทดสอบ** Last Updated Date ต้องเป็น "วันนี้" ถ้าใช้ณาฬิการจริง test จะได้ผลต่างกันทุกวัน แต่เพราะ service ขอ `Clock` ผ่าน DI ตอน test จึงส่ง `FakeClock` ที่ตั้งเวลาไว้ที่ 2026-10-01 10:00 (เวลากรุงเทพ) เข้าไปแทนได้ (harness.ts (apps/api/test/support/harness.ts))

```
const clock = new FakeClock(new Date(now));
const app = await createApp(config, { clock, logger: ... });
```

`Clock` ใน clock.ts (apps/api/src/common/clock.ts) เป็น abstract class ใช้เป็นทั้ง "ชนิด" และ "token" ของ DI ส่วน `AppConfig` เป็น interface ซึ่งหายไปหลัง compile จึงต้องใช้ token `APP_CONFIG` กับ `@Inject(...)` แทน

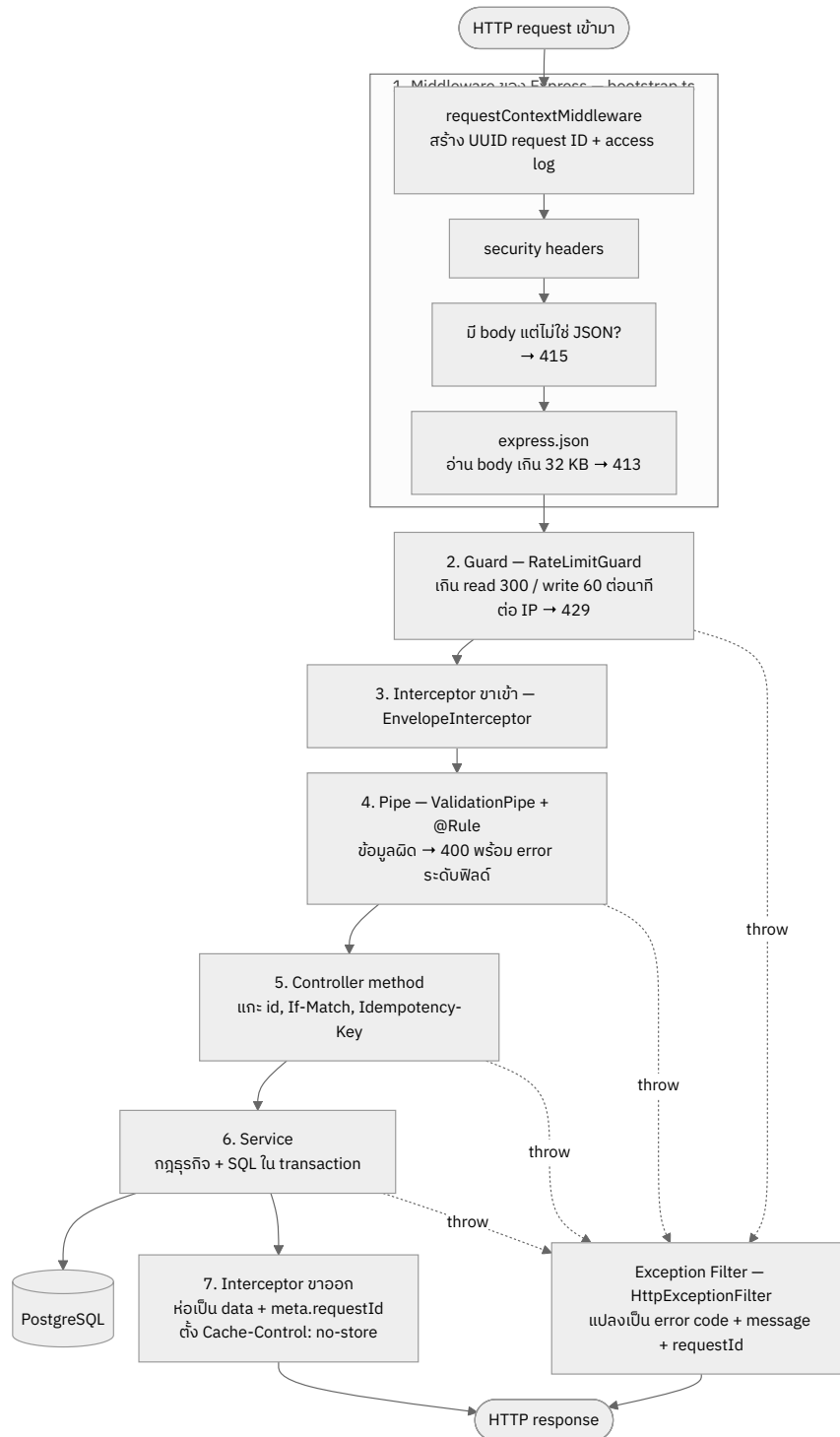
แอปเริ่มทำงานอย่างไร



- `main.ts` (`apps/api/src/main.ts`) เป็นจุดเริ่ม
- `bootstrap.ts` (`apps/api/src/bootstrap.ts`) มีฟังก์ชัน `createApp()` ที่ประกอบแอปทั้งหมด ใช้ร่วมกันสามที่: ตอนรันจริง, ตอน test (`startApp` ใน harness) และตอนสร้าง OpenAPI ทำให้ทั้งสามที่ได้แอปหน้าตาเดียวกัน
- ถ้า env ผิด เช่น `DATABASE_URL` หาย process จะหยุดพร้อมรายการปัญหา (ไม่พิมพ์ค่า secret ออกมา)

ทางเดินของ request หนึ่งคำขอ

นี่คือภาพที่สำคัญที่สุดของบอทนี้ ทุก request ผ่านด้านตามลำดับนี้เสมอ



ด้าน	ไฟล์	ประกาศไว้ที่ไหน
Middleware	request-id.middleware.ts (apps/api/src/common/request-id.middleware.ts), bootstrap.ts (apps/api/src/bootstrap.ts)	app.use(...) ใน createApp
Guard	rate-limit.ts (apps/api/src/rate-limit/rate-limit.ts)	{ provide: APP_GUARD, useClass: RateLimitGuard } ใน rate-limit.module.ts (apps/api/src/rate-limit/rate-limit.module.ts) = ใช้กับทุก route
Interceptor	envelope.interceptor.ts (apps/api/src/common/envelope.interceptor.ts)	app.useGlobalInterceptors(...)
Pipe	validation.pipe.ts (apps/api/src/validation/validation.pipe.ts), field-rule.decorator.ts (apps/api/src/validation/field-rule.decorator.ts)	@UsePipes(bodyValidationPipe) บนแต่ละ method
Exception Filter	http-exception.filter.ts (apps/api/src/common/http-exception.filter.ts)	app.useGlobalFilters(...)

route ที่ไม่ต้องการ rate limit (เช่น health) ปิดได้ด้วย `@RateLimit('none')` — guard อ่านป้ายนี้ผ่าน `Reflector`

Validation: DTO + `@Rule()`

DTO (Data Transfer Object) คือ class ที่บอกว่า body หรือ query ต้องมีหน้าตาแบบไหน ดู `employee.dto.ts`
(apps/api/src/employees/employee.dto.ts)

```
export class UpdateEmployeeDto {
  @ApiPropertyOptional({ type: String, example: '63000.00' }) // สำหรับ OpenAPI
  @Rule(salaryRule, { optional: true }) // สำหรับตรวจสอบจริง
  salary?: string;
  // ... name, departmentId, joinDate, isActive
}
```

โปรเจกต์นี้ไม่ใช่ decorator สำเร็จรูปอย่าง `@IsString()` แต่เขียนกฎเป็น **ฟังก์ชันธรรมดา** ใน `employee-rules.ts`
(apps/api/src/employees/employee-rules.ts) แล้วผูกกับฟิลด์ด้วย `@Rule(...)` (D-19)

```
export function salaryRule(value: unknown): RuleResult<string> {
  if (typeof value !== 'string') {
    return fail('SALARY_TYPE_INVALID', 'Salary must be sent as a decimal string, for example "65000.00".');
  }
  // ... ตรวจสอบรูปแบบ แล้วคืนค่าที่ normalize เป็นทศนิยม 2 ตำแหน่ง เช่น "65000" → "65000.00"
}
```

กฎชุดเดียวกันถูกใช้ซ้ำ 4 ที่ ได้แก่ ตรวจสอบ DTO, normalize ใน service, ตอน seed และใน unit test error code จึงตรงกันทุกชั้น

ValidationPipe ตั้งค่าให้เข้มไว้

- `whitelist` + `forbidNonWhitelisted` — ส่งฟิลด์ที่ไม่รู้จักมาจะโดน `UNKNOWN_FIELD` ส่วนฟิลด์ที่ระบบกำหนดเอง (`id`, `version`, `lastUpdatedDate`) จะโดน `READ_ONLY_FIELD`

- `transform: false` — ไม่แปลงชนิดให้อัตโนมัติ

เดินผ่านโค้ด: `PATCH /api/v1/employees/:id`

Controller — แกะข้อมูลจาก request แล้วส่งต่อ

จาก `employees.controller.ts` (`apps/api/src/employees/employees.controller.ts`)

```
@Patch('/:id') // PATCH /api/v1/employees/:id
@UsePipes(bodyValidationPipe) // ตรวจสอบ body ด้วย UpdateEmployeeDto
@ApiHeader({ name: 'If-Match', required: true, ... }) // บ้าย OpenAPI
@ApiErrors(400, 404, 409, 413, 415, 428, 429) // บ้าย OpenAPI: error ที่เป็นไปได้
async update(
  @Param('id') id: string,
  @Body() body: UpdateEmployeeDto,
  @Headers('if-match') ifMatch: string | undefined,
  @Res({ passthrough: true }) res: Response,
) {
  const employeeId = parseId(id); // "104" → 104 ถ้าไม่ใช่ตัวเลขที่ถูกต้อง → 404
  if (Object.keys(body ?? {}).length === 0) { // body ว่าง → 400 EMPTY_PATCH
    throw Errors.validation([{ field: 'body', code: 'EMPTY_PATCH', ... }]);
  }
  const version = parseIfMatch(ifMatch); // ไม่มี If-Match → 428, รูปแบบผิด → 400
  const { employee, changed } = await this.employees.update(employeeId, body, version);
  res.setHeader('ETag', `"$${employee.version}"`); // version ใหม่ให้ client ใช้ครั้งถัดไป
  return respond(employee, { changed }); // interceptor จะห่อเป็น { data, meta }
}
```

สังเกตว่า controller **ไม่มีกฎธุรกิจ** ทำแค่แปลง HTTP ให้เป็นค่าที่ใช้ได้ แล้วโยนให้ service

Service — กฎธุรกิจกับฐานข้อมูล

จาก `employees.service.ts` (`apps/api/src/employees/employees.service.ts`) (ตัดให้สั้นลง)

```
async update(id, patch, expectedVersion) {
  const normalized = normalizePartial(patch); // ใช้ rule ชุดเดิม normalize ค่า
  return this.prisma.$transaction(async (tx) => { // ทุกอย่างข้างในสำเร็จหรือล้มพร้อมกัน
    const current = await this.findOne(tx, id, true); // SELECT ... FOR UPDATE = ล็อกแถวนี้
    if (!current) throw Errors.employeeNotFound(); // 404
    if (current.version !== expectedVersion) throw Errors.versionConflict(current.version); //
409

    const next = { ...ค่าเดิม, ...ค่าใหม่ };
    if (ค่าไม่เปลี่ยนเลย) return { employee: current, changed: false }; // no-op: ไม่เขียน, version
   เก่าเดิม

    await tx.$executeRaw`
      UPDATE employees SET ..., last_updated_date = ${this.today()}::date, version = version +
1
      WHERE id = ${id} AND version = ${expectedVersion}`;
    return { employee: await this.findOne(tx, id), changed: true };
  });
}
```

สามจุดที่ควรจำ

1. **Transaction + row lock** — กันไม่ให้สองคำขอแก้แถวเดียวกันพร้อมกันจนข้อมูลเพี้ยน
2. **ตรวจสอบ version สองชั้น** — ทั้ง `if` และ `WHERE version = ...` ใน SQL
3. **No-op** — กด Save โดยไม่ได้ทำอะไร จะไม่เขียนฐานข้อมูล และ Last Updated Date ไม่เปลี่ยน (PRD §9.4)

Error — โยน `Errors.*` แล้ว filter จัดการต่อ

ทุก error ของแอปสร้างจาก `api-exception.ts` (`apps/api/src/common/api-exception.ts`) ซึ่งมี code คงที่

```
versionConflict: (currentVersion?: number) =>
  new ApiException(409, 'VERSION_CONFLICT', 'This employee was changed by another user. Reload
the latest version.', ...),
```

`HttpExceptionFilter` จะแปลงเป็น JSON แบบนี้เสมอ และไม่มี stack trace หรือข้อความ SQL หลุดออกไป

```
{
  "error": {
    "code": "VERSION_CONFLICT",
    "message": "This employee was changed by another user. Reload the latest version.",
    "requestId": "3f1c2a9e-...",
    "currentVersion": 2
  }
}
```

กับดัก: ห้าม `throw new HttpException(...)` ตรงๆ ให้ใช้ `Errors.*` เสมอ เพราะ UI กับ Postman test อ่าน `code` เพื่อตัดสินใจว่าจะทำอะไรต่อ

ลองเอง

รัน `pnpm dev` ไว้ก่อน แล้วเปิด terminal อีกหน้าต่าง คำสั่งด้านล่างยังตรงไปที่ API พอร์ต 3001 (ถ้าเปลี่ยน `PORT` ใน `.env` ให้เปลี่ยนตาม)

1. ค้นหาชื่อ — ต้องใส่ URL ในเครื่องหมายคำพูด เพราะ `?` เป็น wildcard

```
curl -s 'http://localhost:3001/api/v1/employees?q=john'
```

2. ดูพนักงานคนเดียวพร้อม header — สังเกต `ETag` และ `X-Request-Id`

```
curl -i http://localhost:3001/api/v1/employees/104
```

3. แก้โดยไม่ส่ง `If-Match` — ได้ 428 `PRECONDITION_REQUIRED` และข้อมูลไม่เปลี่ยน

```
curl -i -X PATCH http://localhost:3001/api/v1/employees/104 -H 'Content-Type: application/json' -d '{"salary": "75000.00"}'
```

4. แก้โดยส่ง version ผิด — ได้ 409 `VERSION_CONFLICT` พร้อม `currentVersion` และข้อมูลไม่เปลี่ยน

```
curl -i -X PATCH http://localhost:3001/api/v1/employees/104 -H 'Content-Type: application/json' -H 'If-Match: "99"' -d '{"salary": "75000.00"}'
```

5. ส่ง salary เป็นตัวเลขแบบ string — ได้ 400 `VALIDATION_ERROR` พร้อม `SALARY_TYPE_INVALID` ในรายละเอียด

```
curl -i -X PATCH http://localhost:3001/api/v1/employees/104 -H 'Content-Type: application/json' -H 'If-Match: "1"' -d '{"salary": 75000}'
```

ทุกคำสั่งข้างบนไม่เปลี่ยนข้อมูล ถ้าลองแก้จริงด้วย version ที่ถูกต้องแล้วอยากคืนข้อมูลเป็น 5 records เดิม

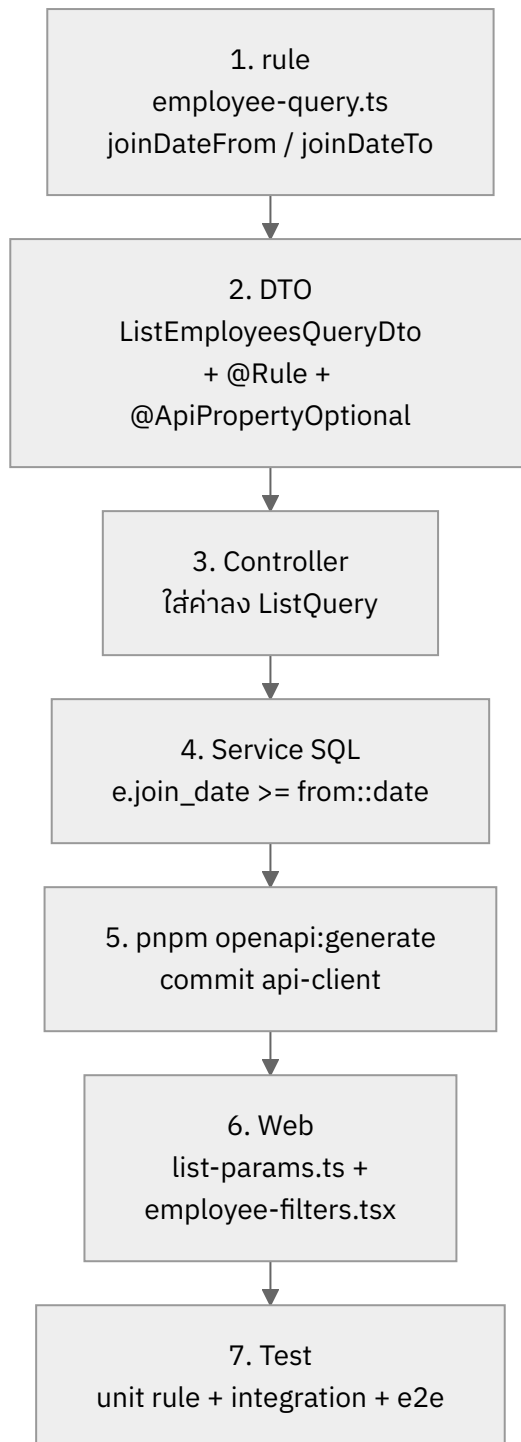
```
pnpm demo:reset --confirm-reset
```

ระหว่างนั้นลองดู log ใน terminal ที่รัน `pnpm dev` จะเห็นบรรทัด JSON `"request"` ของแต่ละคำขอ พร้อม `requestId`, `status` และ `durationMs`

สูตร: เพิ่มของใหม่ทีละชั้น

เพิ่มตัวกรอง Join Date ช่วงเริ่ม-จบ (โจทย์ข้อ Live Coding)

ลำดับนี้มาจาก demo-script.md (docs/demo-script.md) ทำจากชั้นในสุดออกมา



เพิ่ม feature module ใหม่

1. สร้าง folder `apps/api/src/<name>/` มี `<name>.module.ts`, `<name>.controller.ts`, `<name>.service.ts`
2. ใส่ `@Controller('api/v1/<name>')` และ `@Injectable()` ให้ service
3. ลงทะเบียน controller และ provider ใน module แล้วเพิ่ม module เข้า `imports` ของ `AppModule`
4. ตอบด้วย `respond(...)` และโยน error ด้วย `Errors.*`
5. ใส่ป้าย `@ApiEnvelope` / `@ApiErrors` แล้วรัน `pnpm openapi:generate`
6. เขียน integration test ใน `apps/api/test/integration`

กับดักที่เจอบ่อยในโปรเจกต์นี้

กับดัก	ทำไม	ที่มา
import ต้องลงท้าย <code>.js</code> แม้ไฟล์จริงเป็น <code>.ts</code> เช่น <code>'../common/clock.js'</code>	โปรเจกต์เป็น ESM แบบ <code>nodenext</code>	<code>apps/api/AGENTS.md</code>
ห้ามรัน Nest ด้วย <code>tsx/esbuild</code> ใน test หรือ script ที่ต้องใช้ DI	esbuild ไม่สร้าง "decorator metadata" ที่ DI ใช้อ่านชนิดของ constructor ผลคือ DI พังแบบเงิบๆ	D-13, <code>ai-usage.md</code> (<code>docs/ai-usage.md</code>) ข้อ 2
ห้ามอ่าน <code>process.env</code> กระจายในโค้ด	ค่าทุกตัวต้องผ่านการตรวจใน <code>app-config.ts</code> (<code>apps/api/src/config/app-config.ts</code>)	<code>apps/api/AGENTS.md</code>
ห้ามเปลี่ยน query ของ employees เป็น <code>findMany</code>	Prisma คำนวณวันที่เป็น <code>Date</code> ซึ่งอาจเลื่อนวันตาม timezone	D-20, un 3
แก้ controller/DTO แล้วลืม <code>pnpm openapi:generate</code>	Jenkins จะล้มที่ stage Static checks	un 4

ต่อไป: un 3 – ฐานข้อมูลและ Prisma (`docs/learn/03-database-prisma.md`)

บท 3 – ฐานข้อมูลและ Prisma

PostgreSQL อยู่ที่ไหน

PostgreSQL 17 รันอยู่ใน Docker container (ดูบท 5) ตอน dev เปิดด้วย `pnpm dev:up` แล้วรับการเชื่อมต่อที่ `127.0.0.1:5432` เท่านั้น เครื่องอื่นใน LAN ต่อไม่ได้

ใน container เดียวมีหลายฐานข้อมูล แยกตาม environment

ฐานข้อมูล	ใช้กับ	เครื่องหมาย (<code>app_meta.database_purpose</code>)
<code>employee_console_dev</code>	local dev	<code>demo</code>
<code>employee_console_staging</code>	staging (คนละ container และ volume)	<code>demo</code>
<code>employee_console_test_*</code>	test แต่ละรอบ สร้างใหม่ทุกครั้ง	<code>test</code>

"เครื่องหมาย" ในตาราง `app_meta` คือป้ายบอกว่าฐานนี้เอาไว้ทำอะไร คำสั่งที่ลบข้อมูล (`demo:reset` , `test:reset`) จะตรวจป้ายนี้ก่อนเสมอ และปฏิเสธถ้าไม่ตรง ป้องกันการลบฐานผิดตัว (D-22)

ตารางมีอะไรบ้าง

DEPARTMENTS			
varchar	id	PK	engineering, marketing, sales, hr
varchar	name	UK	Engineering ...
int	sort_order		ลำดับใน GET /api/v1/departments

มีพนักงาน

EMPLOYEES			
int	id	PK	identity, seed 101-105, คนใหม่เริ่ม 106
varchar	name		1-100 ตัวอักษร
varchar	department_id	FK	ลบแผนกที่มีคนอยู่ไม่ได้
numeric	salary		numeric 12,2
date	join_date		วันที่ลวัน ไม่มีเวลา
boolean	is_active		true = Active
date	last_updated_date		ระบบตั้งให้
int	version		เพิ่มทีละ 1 ทุกครั้งที่แก้
timestampz	created_at		
timestampz	updated_at		

IDEMPOTENCY_KEYS			
bigint	id	PK	
varchar	scope		employees.create
uuid	key		Idempotency-Key จาก client
varchar	request_hash		hash ของ body
int	response_status		
jsonb	response_body		ผลที่เก็บไว้ replay
timestampz	expires_at		24 ชั่วโมง

APP_META			
varchar	key	PK	database_purpose
varchar	value		demo, test

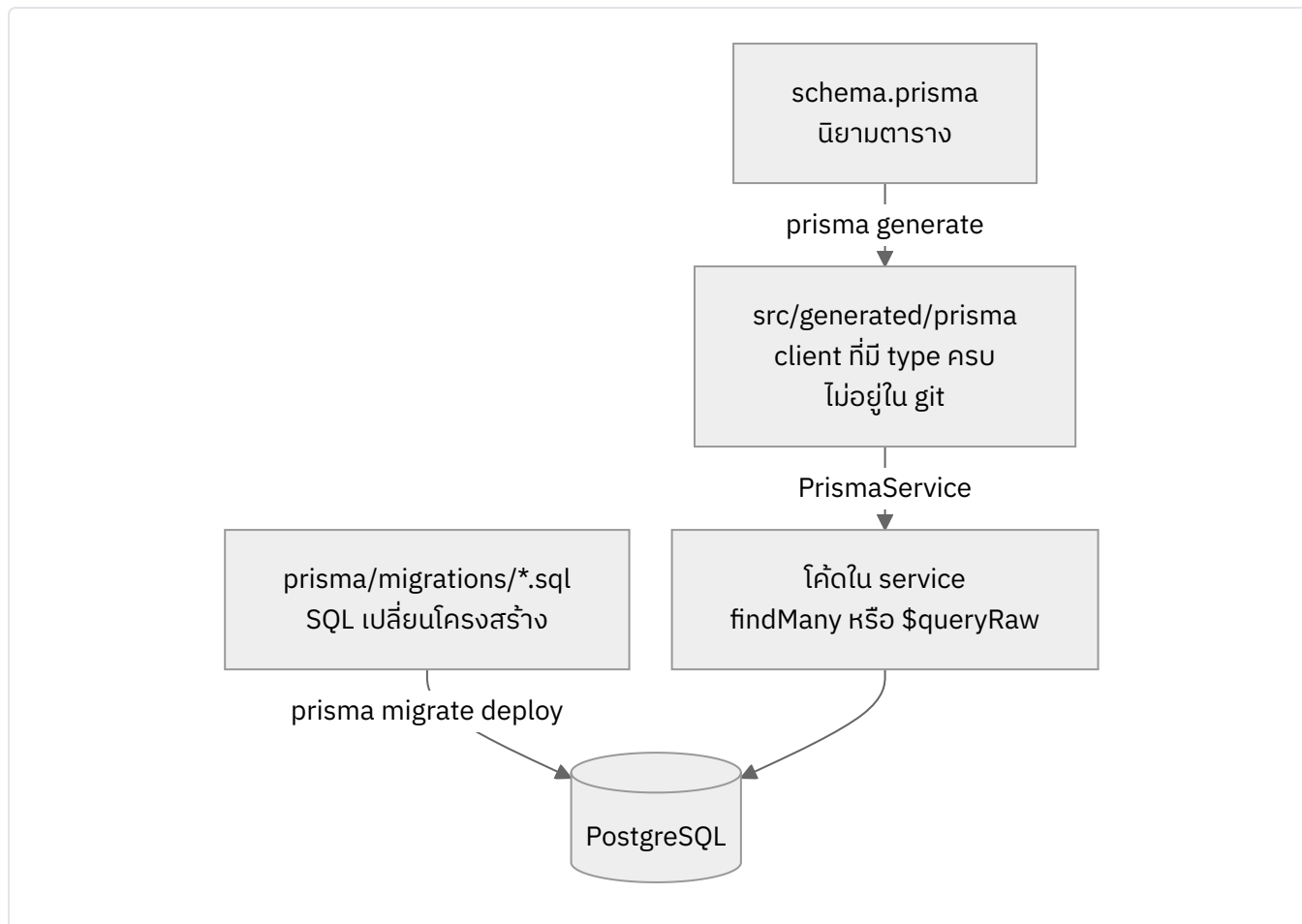
นอกจากนี้ยังมีตาราง `_prisma_migrations` ที่ Prisma ใช้จดว่า migration ไหนถูก apply แล้ว

ตาราง `users`, `sessions`, `reports`, `integration_state` เคยมี แต่ถูกลบโดย migration

`20261003000000_remove_login_and_reports` ตอนตัดระบบ Login และ AI reports ออก (D-46) ถ้าเห็นชื่อพวกนี้ในเอกสารเก่าให้ถือว่าเป็นประวัติ

Prisma คืออะไร

Prisma เป็น **ORM** (Object-Relational Mapping) คือเครื่องมือที่ให้เขียนโค้ด TypeScript แทน SQL ได้ และรู้ชนิดของข้อมูลทุกคอลัมน์ ในโปรเจกต์นี้ Prisma มีสามบทบาท



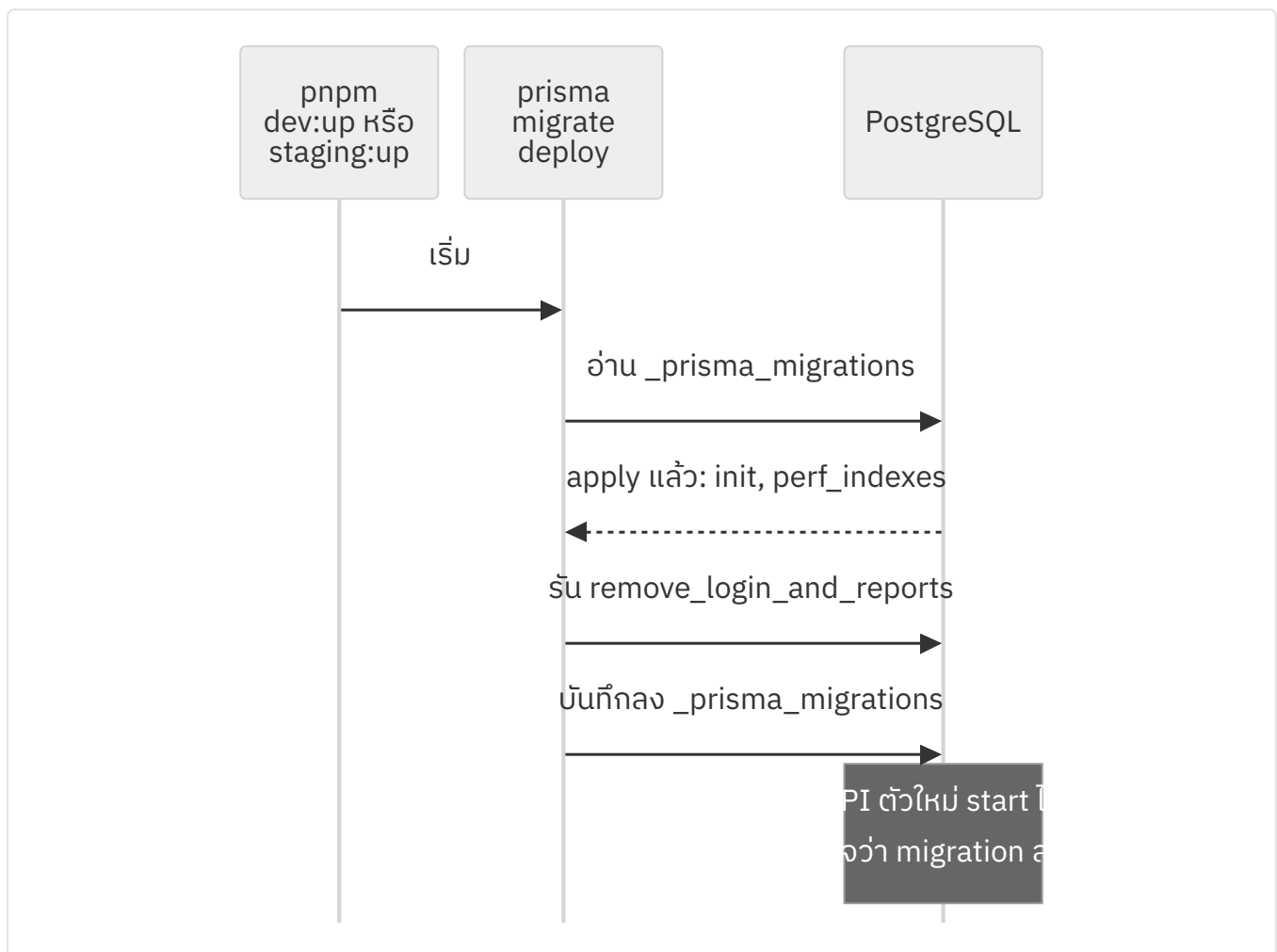
1. **Schema** — `schema.prisma` (`apps/api/prisma/schema.prisma`) บอกว่ามีตารางอะไร คอลัมน์ชนิดไหน ฝั่ง SQL ใช้ชื่อแบบ `snake_case` (เช่น `join_date`) ฝั่งโค้ดใช้ `camelCase` (เช่น `joinDate`) โดยผูกกันด้วย `@map`
2. **Client** — คำสั่ง `prisma generate` อ่าน schema แล้วสร้างโค้ดไว้ใน `apps/api/src/generated/prisma` ซึ่ง `pnpm dev`, `build` และ `typecheck` ทำให้อัตโนมัติ ห้ามแก้ไฟล์ในนั้นด้วยมือ
3. **Migration** — ไฟล์ SQL ที่บอกวิธีเปลี่ยนฐานข้อมูลจากเวอร์ชันหนึ่งไปอีกเวอร์ชัน

`PrismaService` ใน `prisma.service.ts` (`apps/api/src/database/prisma.service.ts`) คือ Prisma client ที่ห่อเป็น Nest provider เพื่อให้ inject ได้ (un 2) จุดที่ควรรู้คือมันบังคับ timezone ของการเชื่อมต่อเป็น UTC (`-c TimeZone=UTC`) และปิดการเชื่อมต่อหลังจาก HTTP server หยุดรับคำขอแล้ว

Migration ทำงานอย่างไร

ลองนึกถึง migration เป็น "บันทึกการต่อเติมบ้าน" แต่ละไฟล์เป็นการต่อเติมหนึ่งครั้ง เรียงตามเวลา ฐานข้อมูลจดไว้ว่าต่อเติมถึงครั้งไหนแล้ว เวลา deploy ระบบจะทำเฉพาะครั้งที่ยังไม่ได้ทำ

Migration	ทำอะไร
20261001000000_init (apps/api/prisma/migrations/20261001000000_init/migration.sql)	สร้างตารางทั้งหมด พร้อม identity column, CHECK constraint (เช่น <code>salary >= 0</code> , ชื่อต้อง trim แล้ว)
20261002000000_perf_indexes (apps/api/prisma/migrations/20261002000000_perf_indexes/migration.sql)	เพิ่ม index สำหรับกรองแผนก+สถานะ และ trigram index สำหรับค้นหาชื่อ (D-32)
20261003000000_remove_login_and_reports (apps/api/prisma/migrations/20261003000000_remove_login_and_reports/migration.sql)	ลบตารางของ Login และ AI reports (D-46)



กติกายของโปรเจกต์ (จาก prisma/AGENTS.md (apps/api/prisma/AGENTS.md))

- **append-only** — ห้ามแก้ไขไฟล์ migration ที่ apply ไปแล้ว ถ้าต้องการเปลี่ยนให้เพิ่มไฟล์ใหม่
- สิ่งที่ `schema.prisma` เขียนไม่ได้ (identity, CHECK, partial unique index, trigram index) ต้องเขียน SQL เองใน migration แล้วใส่คอมเมนต์ใน schema ชี้ไป

- `/api/health/ready` ตอบ 503 จนกว่า migration ล่าสุดที่มากับ build จะถูก apply (D-33) staging จึงไม่ปล่อย API ที่ schema ไม่ตรงขึ้นมารับงาน

ทำไมบางที่ใช้ `findMany` แต่บางที่ใช้ SQL ตรง ๆ

โปรเจกต์นี้ query ฐานข้อมูลสองแบบ

แบบที่ 1: Prisma API — ใช้กับของง่าย ๆ เช่นรายการแผนกใน `departments.controller.ts`

(`apps/api/src/departments/departments.controller.ts`)

```
const rows = await this.prisma.department.findMany({ orderBy: { sortOrder: 'asc' } });
```

แบบที่ 2: `$queryRaw` — ใช้กับ `employees` ทั้งหมดใน `employees.service.ts`

(`apps/api/src/employees/employees.service.ts`)

```
const COLUMNS = Prisma.sql`e.id, e.name, e.department_id AS "departmentId", d.name AS
"departmentName",
  e.salary::text AS salary, e.join_date::text AS "joinDate", e.is_active AS "isActive",
  e.last_updated_date::text AS "lastUpdatedDate", e.version`;
```

เหตุผลที่เลือก SQL ตรง ๆ สำหรับ `employees` (D-20)

ปัญหาถ้าใช้ <code>findMany</code>	วิธีแก้ด้วย SQL
คอลัมน์ <code>date</code> กลายเป็น JavaScript <code>Date</code> (เวลาเที่ยงคืน UTC) พอแสดงในเขตเวลาอื่นวันอาจเลื่อน	cast เป็น <code>::text</code> ได้ <code>"2023-01-15"</code> ตรง ๆ
<code>numeric</code> กลายเป็น object <code>Decimal</code> ที่ต้องแปลงอีกที	cast เป็น <code>::text</code> ได้ <code>"65000.00"</code> ตรง ๆ
ค้นหาชื่อด้วย <code>LIKE</code> ต้อง escape <code>%</code> และ <code>_</code> ที่ผู้ใช้พิมพ์มา	<code>escapeLike()</code> + <code>ESCAPE '\\'</code>
เรียงชื่อแบบ case-insensitive ด้วย collation ที่กำหนดเอง	<code>lower(e.name) COLLATE "C"</code>

แล้วปลอดภัยจาก SQL injection ไหม

ปลอดภัย เพราะ `$queryRaw`...${value}`` เป็น **tagged template** ค่าใน `${}` ถูกส่งเป็น parameter แยกจากตัว SQL ไม่ได้เอาไปต่อ string ส่วนชื่อคอลัมน์ที่ใช้เรียงลำดับ (ซึ่งส่งเป็น parameter ไม่ได้) จะเลือกจาก whitelist `SORT_SQL` เท่านั้น ค่าที่ผู้ส่งมาไม่มีทางกลายเป็นข้อความ SQL

```
const SORT_SQL: Record<SortField, string> = {
  id: 'e.id',
  name: 'lower(e.name) COLLATE "C"',
  // ...
};
```

Transaction และการล็อก

Transaction คือการรวมหลายคำสั่งให้ "สำเร็จทั้งหมด หรือไม่เกิดอะไรเลย" โปรเจกต์ใช้ในสามที่สำคัญ

ที่	ทำไ
<code>list()</code> นับจำนวนแล้วถึงหน้า	ใช้ <code>RepeatableRead</code> ให้ <code>meta.total</code> ตรงกับข้อมูลที่ได้ แม้มีคนเพิ่มหรือลบระหว่างนั้น
<code>update()</code>	<code>SELECT ... FOR UPDATE</code> ล็อกแถว แล้วตรวจ version ก่อน UPDATE
<code>create()</code> ผ่าน <code>IdempotencyService</code>	บันทึก key กับสร้างพนักงานใน transaction เดียว ถ้าสร้างล้ม key ก็ไม่ค้าง

Seed และ reset

คำสั่ง	ทำอะไร	ข้อจำกัด
<code>pnpm db:migrate</code>	apply migration ที่ยังไม่ได้ทำ	—
<code>pnpm db:seed</code>	เติมพนักงานจาก Excel เฉพาะ ID ที่ยังไม่มี ไม่ทับข้อมูลเดิม (D-02)	—
<code>pnpm demo:reset --confirm-reset</code>	ลบพนักงานทั้งหมดแล้วใส่ 5 records ใหม่ ID ถัดไป กลับเป็น 106	เฉพาะ <code>APP_ENV</code> local/staging และฐานที่มีป้าย <code>demo</code>

seed อ่านจาก `test-exam-data.json` (`apps/api/prisma/seed-data/test-exam-data.json`) ซึ่งเป็นสำเนาของไฟล์ Excel และ **คง Last Updated Date เดิมจาก Excel ไว้** ไม่ได้เปลี่ยนเป็นวันนี้ (PRD §4.3) reset ทั้งหมดทำใน transaction เดียวพร้อม `LOCK TABLE` (D-37)

สิทธิ์ของผู้ใช้ฐานข้อมูล

มีผู้ใช้ (role) แยกกันตามหลัก "ให้สิทธิ์เท่าที่จำเป็น" (D-43)

- app role** — แอปใช้ตัวนี้ อ่านเขียนข้อมูลได้แต่ **สร้างฐานข้อมูลไม่ได้** (`NOCREATEDB`)
- test role** — test runner ใช้ตัวนี้ สร้างและลบได้เฉพาะฐาน `employee_console_test_*`

สคริปต์ที่สร้าง role คือ `init-databases.sh` (`infra/postgres/init-databases.sh`) ซึ่งรันซ้ำได้ไม่พัง (idempotent) และรันทุกครั้งที `pnpm dev:up` หรือ deploy staging

ลองเอง

ดูข้อมูลพนักงานใน dev โดยตรง (อ่านอย่างเดียว ไม่ต้องใส่รหัสผ่าน เพราะต่อจากข้างใน container)

```
docker compose exec postgres psql -U postgres -d employee_console_dev -c 'SELECT id, name, salary, join_date, is_active, version FROM employees ORDER BY id'
```

ดูว่า migration ไหน apply แล้วบ้าง

```
docker compose exec postgres psql -U postgres -d employee_console_dev -c 'SELECT migration_name, finished_at FROM _prisma_migrations ORDER BY migration_name'
```

ดูป้ายของฐานข้อมูล

```
docker compose exec postgres psql -U postgres -d employee_console_dev -c 'SELECT * FROM app_meta'
```

ต่อไป: un 4 — OpenAPI (docs/learn/04-openapi.md)

บท 4 – OpenAPI: สัญญาระหว่าง API กับเว็บ

ปัญหาที่ OpenAPI แก้

สมมติฝั่ง API เปลี่ยนชื่อฟิลด์จาก `salary` เป็น `monthlySalary` แต่ฝั่งเว็บไม่รู้ เว็บยังส่ง `salary` ไปเหมือนเดิม ผลคือ API ตอบ 400 `UNKNOWN_FIELD` และกว่าจะรู้ก็ตอนผู้ใช้กด Save ไปแล้ว

ถ้ามี **เอกสารกลาง** ที่บอกว่า API รับอะไรและตอบอะไร แล้วฝั่งเว็บสร้าง TypeScript type จากเอกสารนั้น พอ API เปลี่ยน type ของเว็บก็เปลี่ยนตาม และ `pnpm typecheck` จะฟ้องทันทีว่าโค้ดตรงไหนยังใช้ชื่อเก่า

OpenAPI คืออะไร

OpenAPI คือมาตรฐานสำหรับเขียนเอกสาร API เป็นไฟล์ JSON (หรือ YAML) ที่ทั้งคนและโปรแกรมอ่านได้ในไฟล์บอกคร่าวๆ

- มี path อะไรบ้าง แต่ละ path รับ method อะไร
- ต้องส่ง parameter, header และ body แบบไหน
- ตอบกลับได้ที่แบบ (200, 400, 409 ...) และแต่ละแบบมีหน้าตาอย่างไร
- schema ของข้อมูลแต่ละชนิด เช่น `EmployeeDto` มีฟิลด์อะไร ชนิดอะไร บังคับหรือไม่

Swagger คือชื่อเดิมของมาตรฐานนี้ และเป็นชื่อชุดเครื่องมือรอบ ๆ มัน เช่น Swagger UI (หน้าเว็บที่แสดงเอกสารและทดลองยิง API ได้) ส่วน `@nestjs/swagger` คือ library ของ Nest ที่อ่าน decorator แล้วสร้างไฟล์ OpenAPI ให้

เปรียบเทียบง่าย ๆ ได้ว่า OpenAPI คือ **เมนูของร้าน** ที่เขียนและเอียงว่าส่งอะไรได้ ต้องบอกอะไรบ้าง และจะได้อะไรกลับมา

หน้าตาของ OpenAPI ในโปรเจกต์นี้

ไฟล์อยู่ที่ `packages/api-client/openapi.json` มี 3 path

Path	Method
<code>/api/v1/departments</code>	GET
<code>/api/v1/employees</code>	GET , POST
<code>/api/v1/employees/{id}</code>	GET , PATCH , DELETE

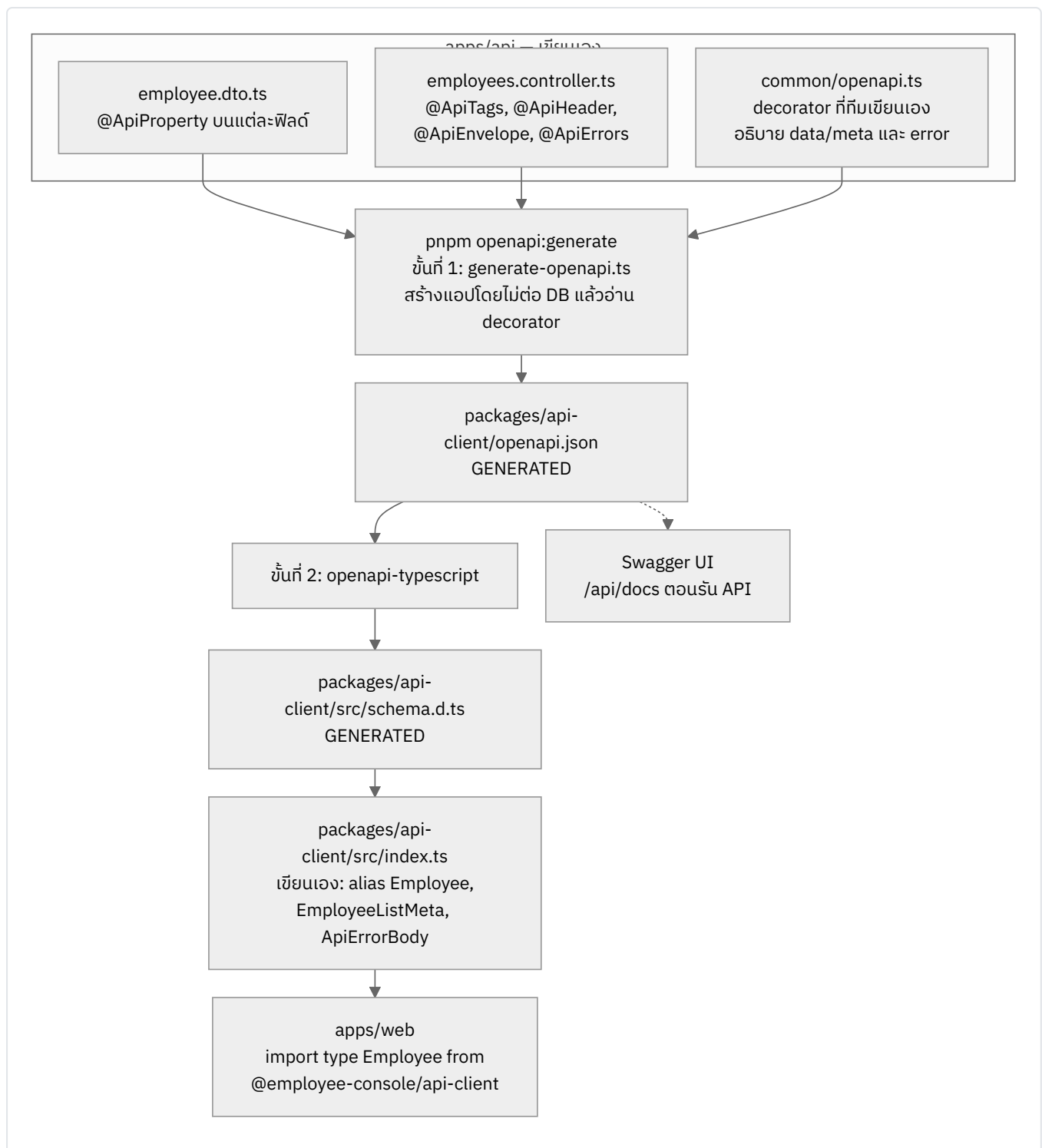
endpoint ของ health ไม่อยู่ในเอกสาร เพราะ `HealthController` ติดป้าย `@ApiExcludeController()` ไว้

ตัวอย่างส่วนหนึ่ง (ย่อ) ของ `PATCH /api/v1/employees/{id}`

```
{
  "operationId": "Employees_update",
  "parameters": [
    { "in": "path", "name": "id", "required": true, "schema": { "type": "string" } },
    { "in": "header", "name": "If-Match", "required": true, "description": "Quoted employee
version, e.g. \"1\" " }
  ],
  "requestBody": { "content": { "application/json": { "schema": { "$ref":
"#/components/schemas/UpdateEmployeeDto" } } } },
  "responses": {
    "200": { "description": "Success", "headers": { "ETag": { ... } } },
    "409": { "description": "VERSION_CONFLICT ..." },
    "428": { "description": "PRECONDITION_REQUIRED ..." }
  }
}
```

OpenAPI ถูกสร้างจากอะไร — ภาพทั้งเส้นทาง

ในโปรเจกต์นี้ **ไม่มีใครเขียน openapi.json** เอง มันถูกสร้างจาก decorator ในโค้ดของ API แล้วไหลต่อไปจนถึงเว็บ



ตามรอยฟิลด์ `salary` หนึ่งฟิลด์

1. ใน DTO ของ API — `employee.dto.ts` (`apps/api/src/employees/employee.dto.ts`)

```
@ApiProperty({ type: String, example: '62000.00' })
salary: string;
```

2. ใน openapi.json — คลังรับ `pnpm openapi:generate`

```
"salary": { "example": "62000.00", "type": "string" }
```

3. ใน schema.d.ts — schema.d.ts (packages/api-client/src/schema.d.ts)

```
EmployeeDto: {  
  /** @example 62000.00 */  
  salary: string;  
  // ...  
}
```

4. alias ให้เรียกง่าย — index.ts (packages/api-client/src/index.ts)

```
export type Employee = Schemas['EmployeeDto'];
```

5. เว็บนำไปใช้ — queries.ts (apps/web/src/lib/queries.ts)

```
import type { Department, Employee, EmployeeListMeta } from '@employee-console/api-client';  
// ...  
api<Employee[], EmployeeListMeta>(`/api/v1/employees?${qs}`, { signal })
```

ถ้าวันหนึ่งเปลี่ยน `salary` ใน DTO แล้วรัน generate โค้ดเว็บที่ใช้ `employee.salary` จะ typecheck ไม่ผ่านทันที

คำสั่ง `pnpm openapi:generate` ทำอะไรบ้าง

ดูใน package.json ที่ root

```
pnpm --filter @employee-console/api run openapi:generate  
→ prisma generate  
→ tsc -p tsconfig.scripts.json (compile สคริปต์ด้วย tsc)  
→ node .tmp/scripts/scripts/generate-openapi.js  
→ เขียน packages/api-client/openapi.json  
pnpm --filter @employee-console/api-client run generate  
→ openapi-typescript openapi.json -o src/schema.d.ts
```

สคริปต์ generate-openapi.ts (apps/api/scripts/generate-openapi.ts) ทำสามอย่าง

1. เรียก `createApp()` ตัวเดียวกับที่ใช้รันจริง แต่ให้ config ปลอมที่ไม่มี secret และไม่ต่อฐานข้อมูลจริง
2. เรียก `buildOpenApiDocument()` ให้ `@nestjs/swagger` อ่าน decorator ทั้งหมด
3. เรียง key ทุกตัว ก่อนเขียนไฟล์ ผลจึงเหมือนเดิมทุกครั้งถ้าโค้ดไม่เปลี่ยน ทำให้ตรวจ drift ได้ (ดูด้านล่าง)

กับดัก: ทำไมต้อง compile ด้วย `tsc` ก่อน แทนที่จะรันด้วย `tsx` ตรง ๆ? เพราะ `tsx` ใช้ esbuild ซึ่งไม่สร้าง decorator metadata ทำให้ Nest DI หายของไม่เจอ ปัญหานี้เกิดขึ้นจริงระหว่างพัฒนา (ai-usage.md (docs/ai-usage.md) ข้อ 2)

Swagger UI – เอกสารที่ทดลองได้

ตอนรัน `pnpm dev` เปิด <http://localhost:3001/api/docs> จะเห็นหน้าเอกสาร API แบบโต้ตอบได้ (ไฟล์ JSON ดับอยู่ที่ `/api/openapi.json`) ตั้งค่าไว้ใน `createApp()` ของ `bootstrap.ts` (`apps/api/src/bootstrap.ts`)

```
SwaggerModule.setup('api/docs', app, document, {
  jsonDocumentUrl: 'api/openapi.json',
  raw: ['json'], // เสริมเฉพาะ JSON ไม่เปิด /api/docs-yaml
});
```

ใช้ประกอบการอธิบายตอน demo ได้ดี เพราะเห็น endpoint, header ที่บังคับ และ error ที่เป็นไปได้ครบในหน้าเดียว

Decorator ที่ทีมเขียนเอง (`common/openapi.ts`)

ทุก response ของโปรเจกต์ห่อเป็น `{ data, meta }` และทุก error เป็น `{ error: { code, message, requestId } }` ซึ่ง `@nestjs/swagger` ไม่รู้จักเอง จึงมี helper ใน `common/openapi.ts` (`apps/api/src/common/openapi.ts`) (D-42)

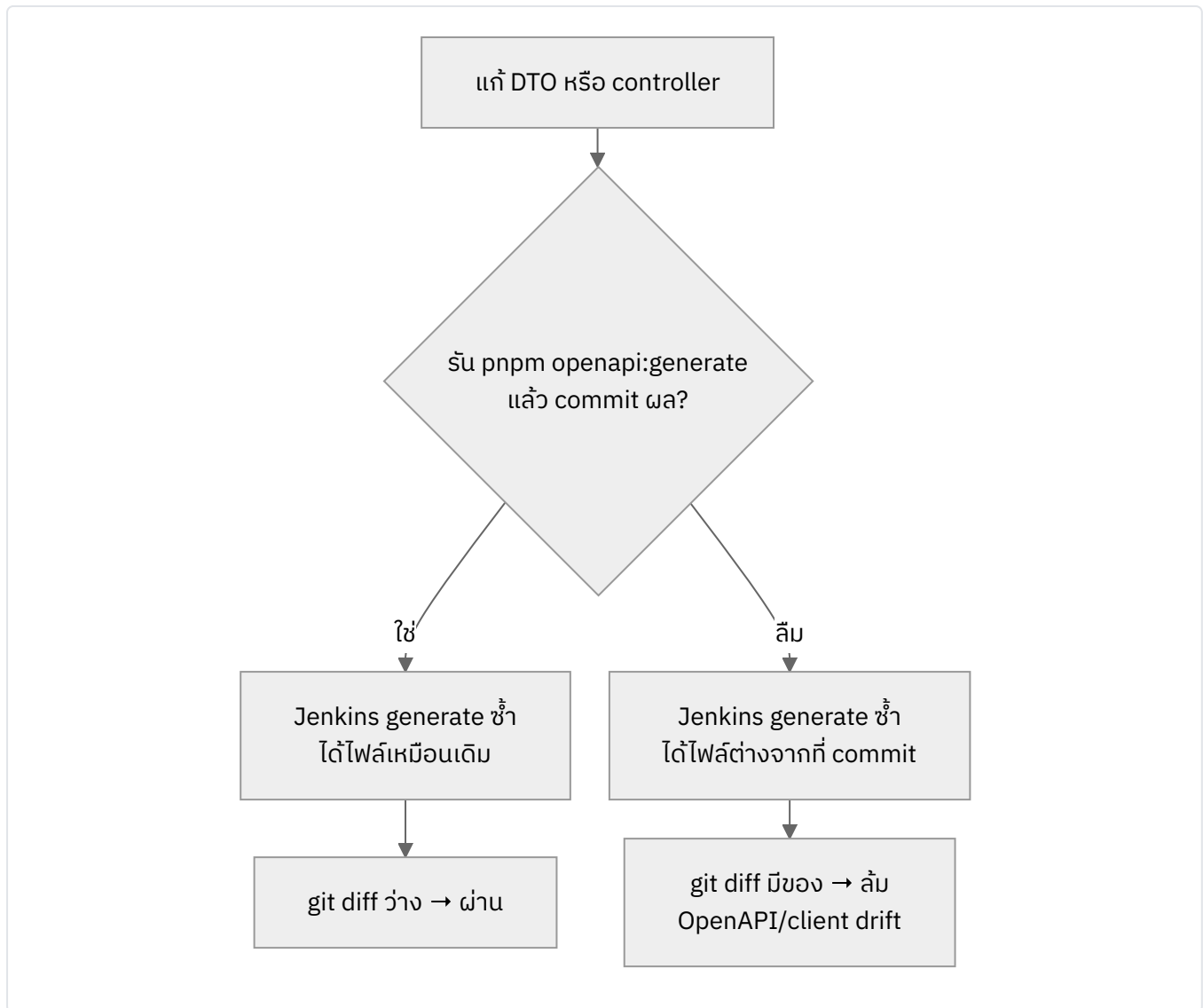
Decorator	ใช้ออกเอกสารว่า
<code>@ApiEnvelope(EmployeeDto, { isArray, meta, status, headers })</code>	response สำเร็จหน้าตาเป็น <code>{ data: EmployeeDto, meta: ... }</code> พร้อม header เช่น <code>ETag</code> , <code>Location</code>
<code>@ApiErrors(400, 404, 409, ...)</code>	status ที่เป็นไปได้ แต่ละตัวใช้ <code>ErrorEnvelopeDto</code> และระบุ error code
<code>@ApiNoContent('Deleted')</code>	ตอบ 204 ไม่มี body

Drift check ใน Jenkins

Drift แปลว่า "หลุดจากกัน" คือ `openapi.json` ที่ commit ไว้ไม่ตรงกับโค้ดปัจจุบัน ใน stage `Static checks` ของ Jenkinsfile มีสองบรรทัดนี้

```
pnpm openapi:generate
git diff --exit-code -- packages/api-client || { echo "OpenAPI/client drift: run pnpm
openapi:generate and commit"; exit 1; }
```

`git diff --exit-code` คั้นค่าผิดพลาดถ้ามีไฟล์เปลี่ยน แปลว่า Jenkins สร้างใหม่แล้วได้ไม่เหมือนที่ commit ไว้ แสดงว่ามีคนแก้ API แต่ลืม generate pipeline จึงหยุดตรงนั้น



เวิร์กโฟลว์เมื่อแก้ API

1. แก้ controller, DTO หรือ decorator ใน `apps/api`
2. รัน

```
pnpm openapi:generate
```

3. ดูว่าจะไรเปลี่ยน

```
git diff -- packages/api-client
```

4. ถ้าเพิ่ม DTO ใหม่ที่เว็บต้องใช้ ให้เพิ่ม alias ใน `index.ts` (`packages/api-client/src/index.ts`)
5. รัน `pnpm typecheck` ให้เว็บผ่าน แล้ว commit ทั้ง `openapi.json` และ `schema.d.ts` พร้อมโค้ด API

ข้อห้าม

- ห้ามแก้ไข `openapi.json` และ `schema.d.ts` ด้วยมือ รอบหน้าที่ generate ก็จะถูกเขียนทับ
- ห้ามนิยาม type ของ response ซ้ำในเว็บ ให้ import จาก `@employee-console/api-client`
- type จาก OpenAPI ใช้ตอน **compile** เท่านั้น ตอนรันจริงไม่ได้ตรวจสอบข้อมูล การตรวจจริงยังเป็นหน้าที่ของ ValidationPipe ฝั่ง API และ zod schema ของฟอร์มฝั่งเว็บ

ต่อไป: บท 5 – Docker ([docs/learn/05-docker.md](#))

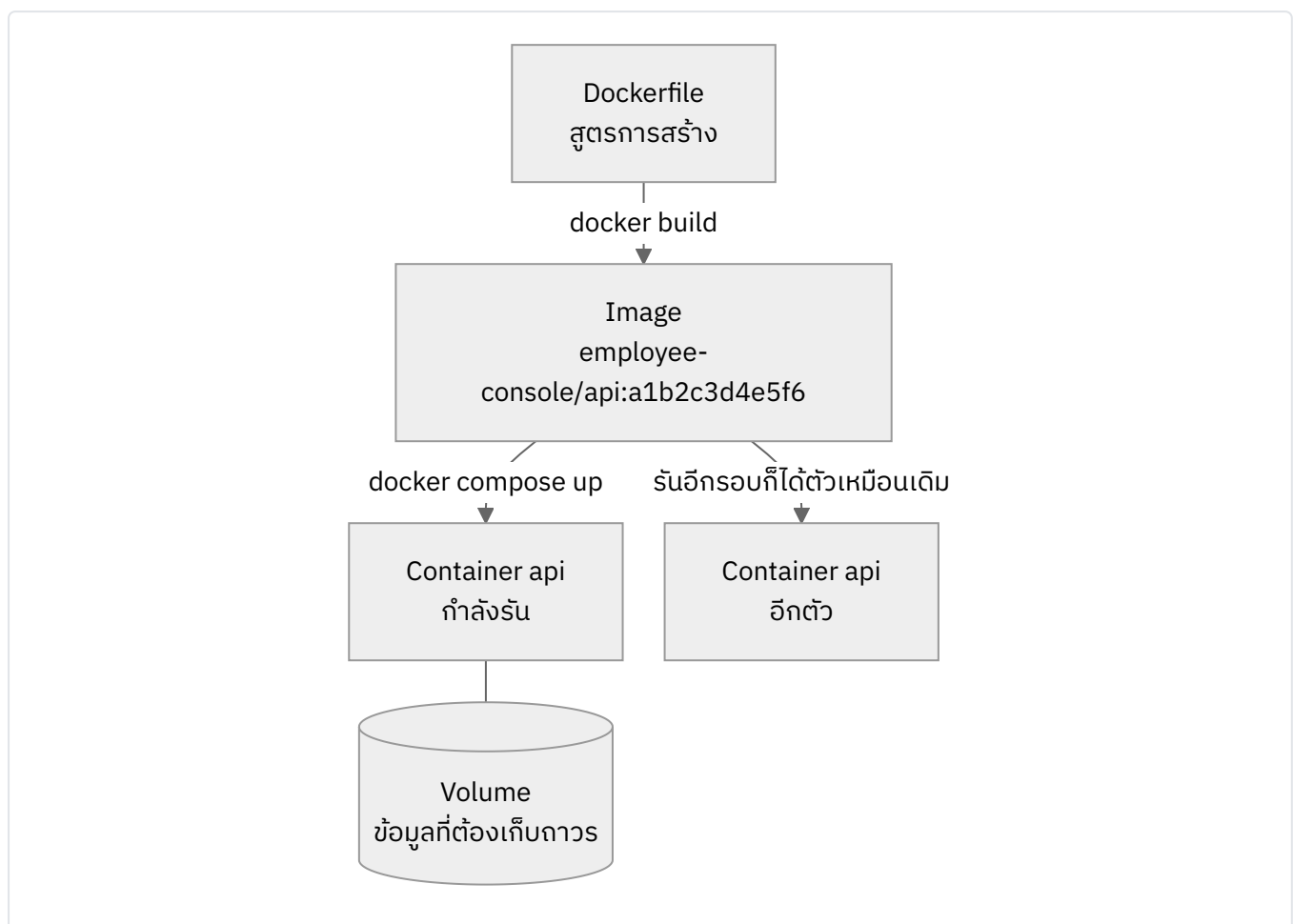
Un 5 – Docker

ก่อนจะเข้าใจ Jenkins ต้องรู้จัก Docker ก่อน เพราะทั้งฐานข้อมูล, staging และตัว Jenkins เองรันอยู่ใน Docker

คำศัพท์ 4 คำ

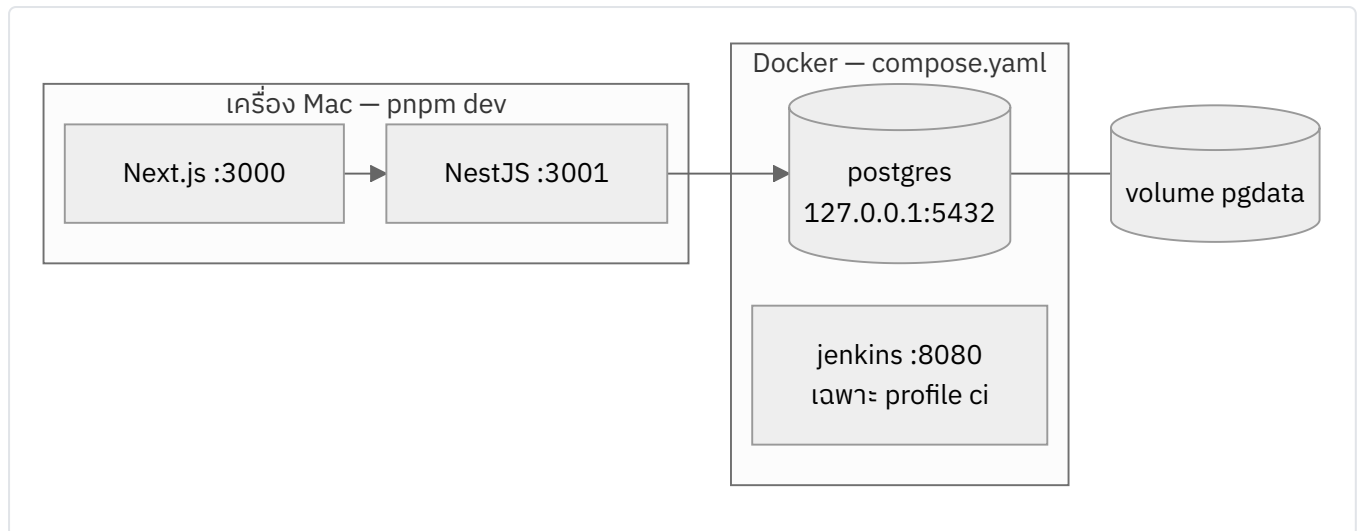
คำ	เปรียบเทียบ	ในโปรเจกต์นี้
Image	แม่พิมพ์ หรือกล่องข้าวแช่แข็งที่ปิดฝาแล้ว เปลี่ยนข้างในไม่ได้	<code>employee-console/api:<sha></code> , <code>employee-console/web:<sha></code> , <code>postgres:17.11-bookworm</code>
Container	ขนมที่อบออกมาจากแม่พิมพ์ หรือข้าวที่อุ่นแล้ว กำลังกิน = image ที่กำลังรันอยู่	container <code>postgres</code> , <code>api</code> , <code>web</code>
Volume	ตู้เก็บของที่อยู่นอก container ทั้ง container แล้วของยังอยู่	<code>pgdata</code> (dev), <code>pgdata_staging</code> (staging)
Compose	ใบสั่งงานที่บอกว่าจะเปิด container อะไรบ้าง ต่อกันอย่างไร	<code>compose.yaml</code> , <code>compose.staging.yaml</code>

ข้อดีที่สำคัญที่สุดคือ **image** ที่ผ่าน **test** แล้วคือตัวเดียวกับที่เอาไป **deploy** ไม่มีปัญหา "เครื่องรันได้แต่เครื่องอื่นรันไม่ได้"



Dev: Docker แคลฐานข้อมูล

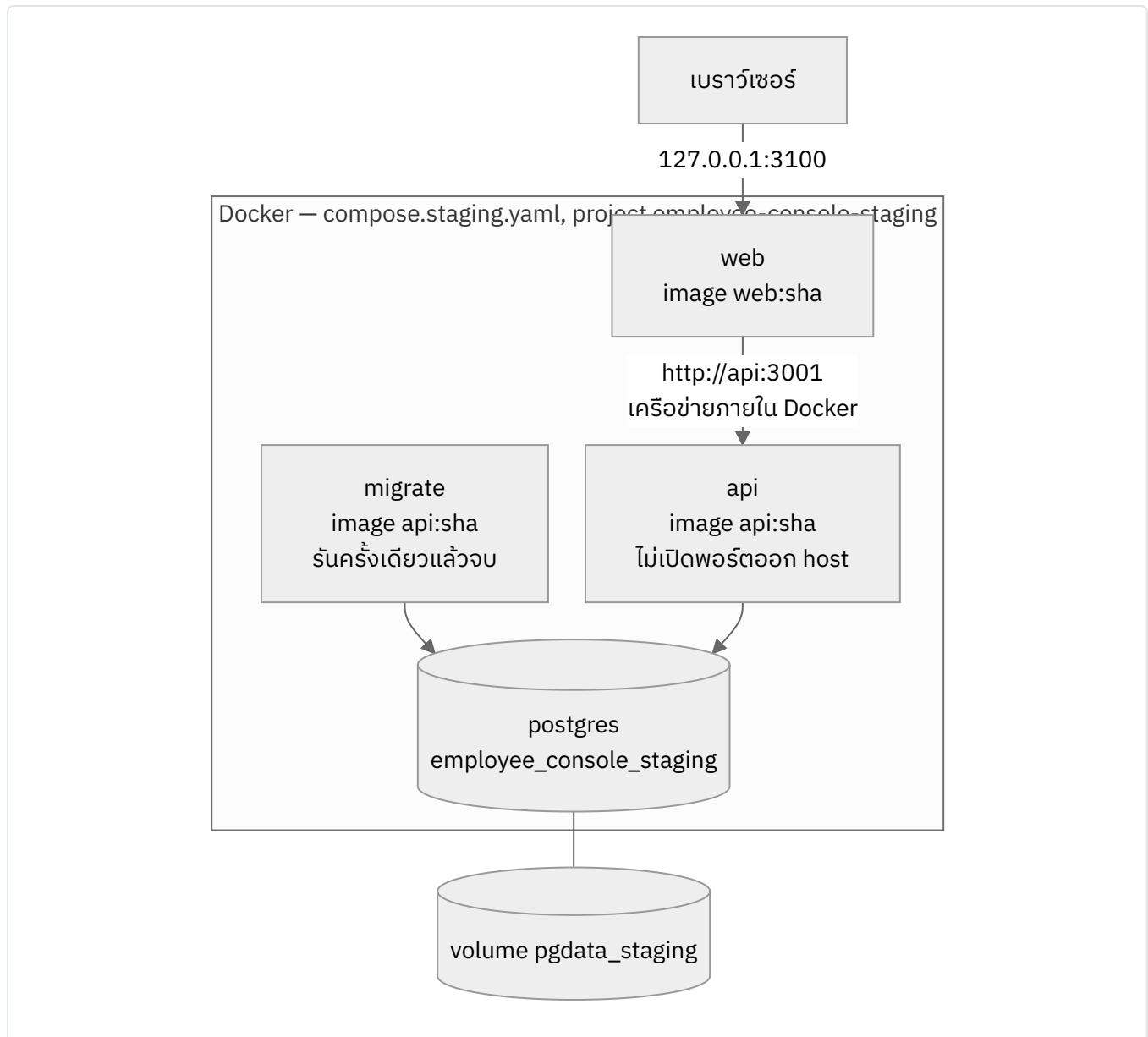
ตอนพัฒนา เว็บและ API รันบนเครื่องโดยตรง (hot reload เร็วกว่า) ส่วน Docker มีแค่ PostgreSQL



- `pnpm dev:up` → เปิด postgres, รอ healthy, ปรับ role ของฐานข้อมูล, apply migration แล้ว seed ครั้งแรก (scripts/dev-up.mjs)
- `pnpm down` → หยุด container ทั้งหมด **ไม่ลบ volume** ข้อมูลจึงยังอยู่
- Jenkins อยู่ใน `compose.yaml` เหมือนกัน แต่ติด `profiles: [ci]` จึงเปิดเฉพาะตอน `pnpm ci:up`
- ทุพพอร์ต bind ที่ `127.0.0.1` เครื่องอื่นในเครือข่ายเข้าไม่ได้

Staging: ทุกอย่างอยู่ใน Docker

staging คือการจำลอง production บนเครื่องเรา ใช้ image จริงที่ build จาก commit



- เปิดพอร์ตออกมาแค่ **web :3100** เบราวเซอร์เข้าถึง API ได้ทาง `/api/*` ของ web เท่านั้น
- `migrate` เป็น service ที่ใช้ image ของ API แต่สั่งให้รัน `prisma migrate deploy` แล้วจบ (ติด `profiles: [tools]` จึงไม่เปิดเองตอน `up`)
- `depends_on` + `healthcheck` ลำดับ: postgres ต้อง healthy ก่อน api และ api ต้อง healthy ก่อน web
- ชื่อ project คนละชื่อกับ dev (`employee-console-staging`) container และ volume จึงไม่ชนกัน

Image ของเรา

ไฟล์	สร้าง	จุดที่ควรรู้
api.Dockerfile (infra/docker/api.Dockerfile)	employee- console/api: <sha>	multi-stage (base → build → prod-deps → runtime) image สุดท้ายมีแค่ของที่ต้องใช้รัน, ต้องมี OpenSSL ให้ Prisma, มี dist- scripts/ ไว้ให้ staging รัน seed
web.Dockerfile (infra/docker/web.Dockerfile)	employee- console/web: <sha>	ใช้ Next.js standalone output, ฝั่ง API_INTERNAL_URL=http://api:3001 ตอน build

multi-stage build คือการแบ่ง Dockerfile เป็นหลายช่วง ช่วงแรกติดตั้งเครื่องมือครบเพื่อ build ช่วงสุดท้ายคัดลอกเฉพาะผลลัพธ์ไปใส่ image เล็ก ๆ image ที่ได้จึงเล็กและไม่มีเครื่องมือที่ไม่จำเป็น

ก๊าดัก: API_INTERNAL_URL ของเว็บถูก ฝั่งตอน build ไม่ได้อ่านตอนรัน image ที่ build ด้วย http://api:3001
จึงใช้ได้เฉพาะใน Compose ที่มี service ชื่อ api เท่านั้น

ก๊าดัก: Dockerfile ทั้งสองคัดลอก package.json ของทุก workspace package ก่อน npm install --frozen-lockfile ถ้าเพิ่มหรือลบ package ใน monorepo ต้องแก้ Dockerfile ทั้งสองไฟล์ (infra/AGENTS.md)

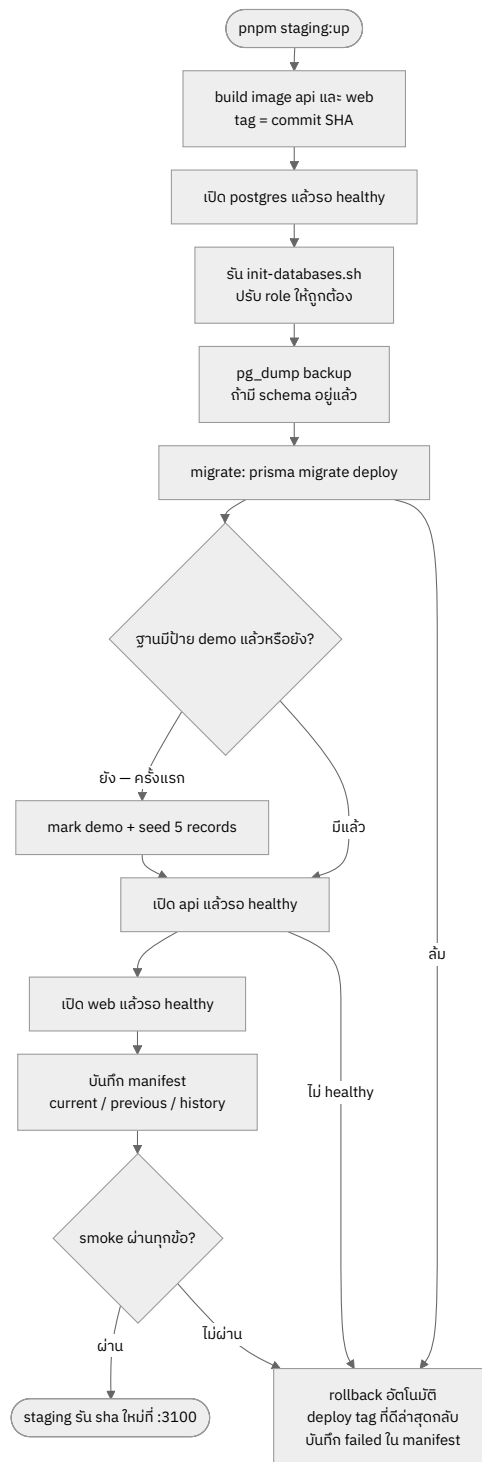
Image tag = commit SHA

ทุก image ติด tag เป็น commit SHA (12 ตัวแรก) เช่น employee-console/api:a1b2c3d4e5f6 ไม่ใช่ latest

- รู้กันที่ว่า staging รันโค้ดจาก commit ไหน
- rollback ได้ง่าย เพราะ image ของ commit ก่อนหน้ายังอยู่ในเครื่อง
- ถ้า build ตอนแก้ไฟล์ที่ git track อยู่แต่ยังไม่ commit tag จะเป็น <sha>-dirty เพื่อไม่ให้อ้างว่าเป็น commit ที่ไม่ตรง (D-30) ไฟล์ใหม่ที่ยังไม่ git add ไม่นับ เพราะ commitTag() ใน staging.mjs (scripts/staging.mjs) ใช้ git status --porcelain --untracked-files=no

Deploy staging ทำอะไรบ้าง

คำสั่ง `pnpm staging:up` (หรือ Jenkins stage `Deploy staging`) เรียก `scripts/staging.mjs` ทำตามลำดับนี้



smoke test คือการตรวจเร็ว ๆ ว่าระบบ "ยังมีลมหายใจ" หลัง deploy ตรวจ 5 ข้อ

1. `/api/health/live` ตอบ 200
2. `/api/health/ready` ตอบ `ready` (ฐานข้อมูลต่อได้ + migration ครบ)
3. หน้า `/employees` โหลดได้
4. ไฟล์ static (CSS/JS) โหลดได้

5. `/api/v1/employees` ผ่าน web origin ตอบ 200

สถานะการ deploy เกิดที่ `~/.employee-console/staging/` (`manifest.json` และ `backups/`) ใช้ร่วมกันระหว่างคำสั่งในเครื่องกับ Jenkins (D-41)

Rollback ไม่ย้อน schema

`pnpm staging:rollback` เปลี่ยน image กลับไปเป็น tag ก่อนหน้า แต่ **ไม่ย้อน migration** ซึ่งทำได้เพราะ migration ส่วนใหญ่เป็นแบบ "expand" (เพิ่มอย่างเดียว image เก่ายังใช้ได้) ข้อยกเว้นคือ migration ที่ลบตาราง (`remove_login_and_reports`) ถ้าจะย้อนไป image ที่เก่ากว่านั้นต้อง restore backup ตาม `runbook.md` (`docs/runbook.md`) หัวข้อ 4

ลองเอง

ดู container ที่รันอยู่ของ dev

```
docker compose ps
```

ดู image ของโปรเจกต์ที่ build ไว้

```
docker images employee-console/api
```

ดู container ของ staging (ชื่อขึ้นต้นด้วย `employee-console-staging-`)

```
docker ps --filter name=employee-console-staging
```

ตรวจ staging หลัง deploy

```
pnpm staging:smoke
```

ต่อไป: บท 6 – Jenkins และ CI/CD (`docs/learn/06-jenkins.md`)

บท 6 – Jenkins และ CI/CD

CI/CD คืออะไร

คำ	ย่อมาจาก	ความหมายง่าย ๆ	ในโปรเจกต์นี้
CI	Continuous Integration	ทุกครั้งที่มีการโค้ดใหม่ ให้เครื่องตรวจอัตโนมัติว่ายังดีอยู่ (lint, typecheck, test, build)	stage Checkout ถึง Images
CD	Continuous Delivery / Deployment	โค้ดที่ผ่านการตรวจแล้ว ส่งขึ้น environment ให้อัตโนมัติ	stage Deploy staging (รวม smoke test และ rollback)

เปรียบเทียบกับโรงงาน: CI คือ **สายพานตรวจคุณภาพ** ที่ทุกชิ้นต้องผ่านทุกด่าน ชี้นไหนตกด่านใดด่านหนึ่งก็ถูกคัดออก ส่วน CD คือ **รถขนส่ง** ที่เอาเฉพาะชิ้นที่ผ่านไปส่งถึงหน้าร้าน

ข้อดีคือความผิดพลาดถูกจับได้เร็วและเหมือนกันทุกครั้ง ไม่ต้องพึ่งความจำว่า "ก่อน deploy ต้องรันอะไรบ้าง"

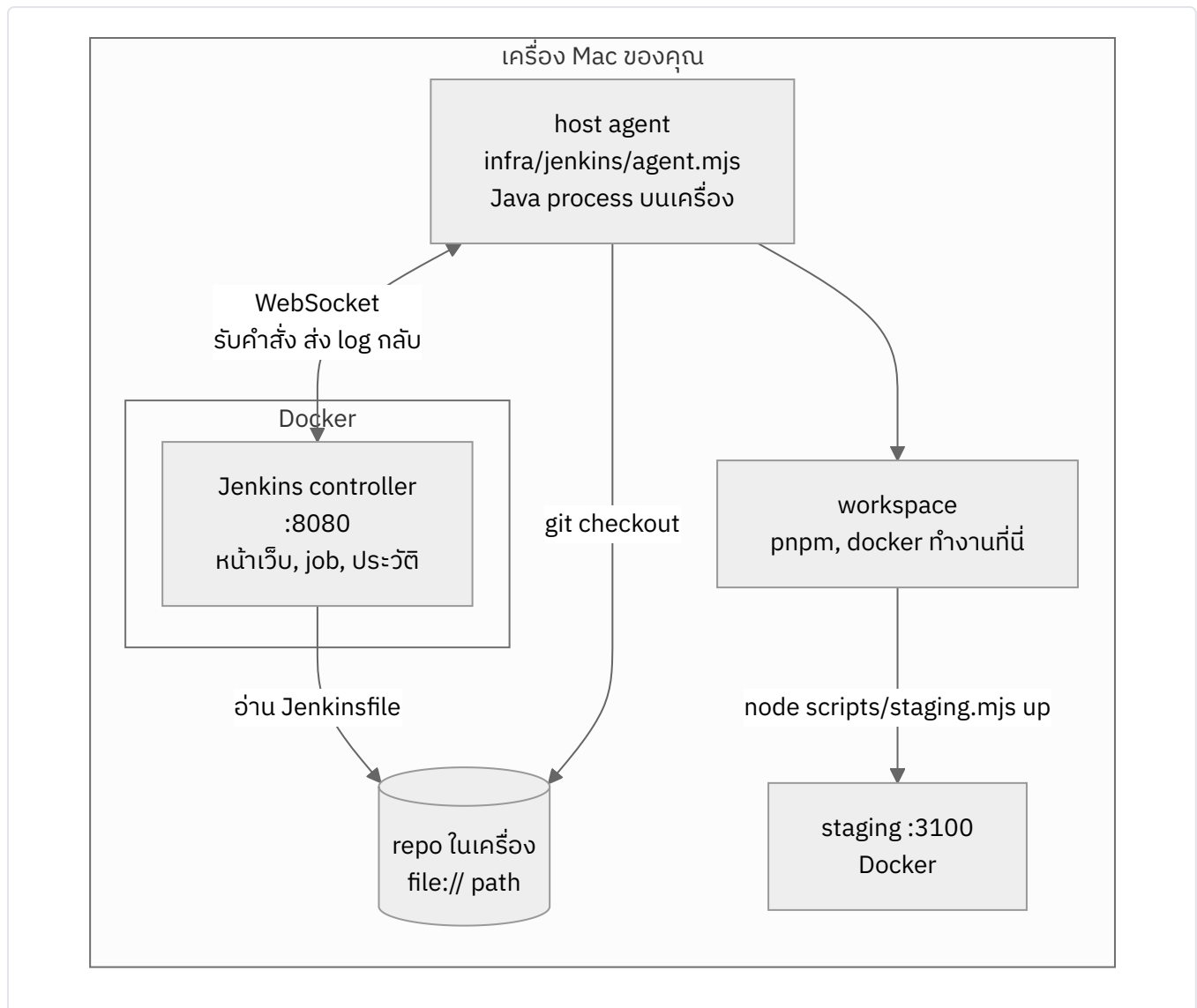
Jenkins คืออะไร

Jenkins เป็นโปรแกรม automation server แบบ open source มีหน้าเว็บให้ดูผล ทำหน้าที่อ่าน "สูตร" จากไฟล์ Jenkinsfile แล้วสั่งเครื่องที่ทำงานจริงให้รันทีละขั้น

คำศัพท์ของ Jenkins

คำ	ความหมาย	ในโปรเจกต์นี้
Controller	สมองและหน้าเว็บ เก็บ job, ประวัติ build, ฝาก test ไม่รัน build เอง	container Jenkins ใน Docker ที่ http://localhost:8080 (numExecutors: 0)
Agent	เครื่องที่ลงมือรันคำสั่งจริง	host-agent = เครื่อง Mac ของเราเอง (มี Node 24, pnpm, Docker) label employee-console
Job	งานหนึ่งงานที่ตั้งค่าไว้	employee-console (pipeline job)
Pipeline	ชุดขั้นตอนทั้งหมด เขียนเป็นโค้ด	ไฟล์ Jenkinsfile
Stage	ด่านหนึ่งด่านใน pipeline เห็นเป็นช่องในหน้าเว็บ	Static checks , Unit , Build + E2E ฯลฯ
Step	คำสั่งย่อยใน stage	sh 'pnpm lint'
Build	การรัน pipeline หนึ่งครั้ง มีเลขกำกับ	#12 a1b2c3d4e5f6
Workspace	folder ที่ agent checkout โค้ดมารัน	ภายใต้ JENKINS_AGENT_WORKDIR
Artifact	ไฟล์ผลลัพธ์ที่เก็บไว้กับ build	ฝาก test, Playwright report, manifest ของ staging
Credentials	ความลับที่ Jenkins เก็บให้ ไม่ต้องเขียนในโค้ด	employee-console-staging-env = ไฟล์ .env.staging
Plugin	ส่วนเสริมของ Jenkins	78 ตัว pin เวอร์ชันใน plugins.txt (infra/jenkins/plugins.txt) (D-44)
JCasC	Jenkins Configuration as Code = ตั้งค่า Jenkins ด้วยไฟล์ YAML แทนการคลิก	casc.yaml (infra/jenkins/casc.yaml)

Jenkins ของโปรเจกต์นี้ต่อกันอย่างไร



จุดที่ต่างจาก Jenkins ทั่วไป (D-29)

- **ไม่มี Git remote** เช่น GitHub ทั้ง controller และ agent อ่าน repo จาก path ในเครื่อง (`file://...`) ถ้าตั้ง `JENKINS_GIT_URL` ใน `.env` ก็ใช้ remote แทนได้
- **ต้อง commit ก่อนกด build** เพราะ Jenkins checkout จาก commit ไม่ใช่จากไฟล์ที่ยังแก้ค้างอยู่
- agent รับบนเครื่องจริงไม่ใช่ใน container เพราะต้องสั่ง Docker เพื่อ build image และเปิด staging ได้

อ่าน Jenkinsfile ทีละส่วน

```
pipeline {
  agent { label 'employee-console' }           // ① รับบน agent ที่มี label นี้ = host-agent

  options {
    disableConcurrentBuilds()                  // ② ห้ามรันสอง build พร้อมกัน
    timestamps()                               //   ใส่เวลาหน้า log ทุกบรรทัด
    timeout(time: 75, unit: 'MINUTES')        //   เกิน 75 นาทีให้ตัด
    buildDiscarder(logRotator(numToKeepStr: '30')) // เก็บประวัติ 30 build ล่าสุด
  }

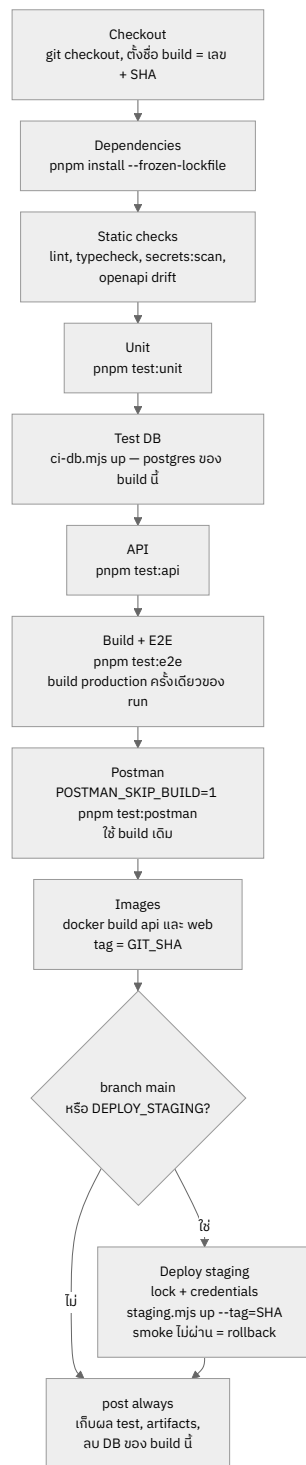
  parameters {                                // ③ ตัวเลือกตอนกด "Build with Parameters"
    booleanParam(name: 'DEPLOY_STAGING', defaultValue: false, ...)
  }

  environment {                                // ④ env ที่ทุก stage เห็น
    CI = 'true'
    CI_PROJECT = "employee-console-ci-${env.BUILD_NUMBER}" // ชื่อ Compose project เฉพาะ build นี้
  }

  stages {                                    // ⑤ ด้านต่าง ๆ เรียงตามลำดับ
    stage('Checkout') { ... }
    // ...
  }

  post {                                       // ⑥ ทำเสมอหลังจบ ไม่ว่าผ่านหรือล้ม
    always { ... }
  }
}
```

ภาษาที่ใช้คือ **Groovy** (Declarative Pipeline) ไม่ต้องเรียนลึก เพราะแต่ละ stage แค่เรียกคำสั่ง `pnpm` ที่เรารันในเครื่องได้อยู่แล้ว



ด้านใดล้ม ด้านถัดไปจะไม่รัน (แต่ `post` ยังรันเสมอ)

Stage	รันอะไร	จับปัญหาแบบไหน	รันเองในเครื่องได้ด้วย
Checkout	<code>checkout scm</code> , พิมพ์เวอร์ชัน <code>node/pnpm/docker</code>	เครื่องมือบน agent ไม่ครบ	—
Dependencies	<code>pnpm install --frozen-lockfile</code>	lockfile ไม่ตรงกับ <code>package.json</code>	<code>pnpm install --frozen-lockfile</code>
Static checks	<code>pnpm lint</code> , <code>typecheck</code> , <code>secrets:scan</code> , <code>openapi:generate</code> + <code>git diff --exit-code</code>	โค้ดผิดรูปแบบ, type ผิด, secret หลุด เข้า repo, ลืม generate OpenAPI (un 4)	คำสั่งเดียวกัน
Unit	<code>pnpm test:unit</code>	กฎของไฟล์, config, การ format ผิด เว็บผิด	<code>pnpm test:unit</code>
Test DB	<code>node scripts/ci-db.mjs up</code>	— (เตรียมฐานข้อมูลแยกของ build นี้ บนพอร์ตว่าง)	—
API	<code>pnpm test:api</code>	API ทำงานผิดกับ PostgreSQL จริง	ต้อง <code>pnpm dev:up</code> ก่อน
Build + E2E	<code>pnpm test:e2e</code> (build production ของ API + web ครั้งเดียวของ run)	build production ไม่ผ่าน, flow ใน เบราว์เซอร์พัง เช่น ฟอรั่ม, filter, layout มือถือ	ต้อง <code>pnpm dev:up</code> ก่อน
Postman	<code>POSTMAN_SKIP_BUILD=1 pnpm test:postman</code> (ใช้ build จาก stage ก่อน หน้า)	สัญญา status/header/error code เปลี่ยน	<code>pnpm test:postman</code> (ต้อง <code>pnpm dev:up</code> ก่อน)
Images	<code>docker build</code> สอง image ติด tag <code>\$GIT_SHA</code>	Dockerfile พัง	—
Deploy staging	<code>staging.mjs up --tag=\$GIT_SHA</code> ภายใต้ <code>lock</code> (รัน smoke test ในตัว)	migration ล้ม, container ไม่ healthy, smoke ไม่ผ่าน (rollback อัตโนมัติ)	<code>pnpm staging:up</code> , ตรวจสอบ ซ้ำด้วย <code>pnpm staging:smoke</code>

ส่วนพิเศษใน Jenkinsfile

- `when { anyOf { ... } }` — ใส่เงื่อนไขให้ stage รันเฉพาะบางกรณี `Deploy staging` รันเมื่อ branch ลงท้ายด้วย `main` หรือเลือก `DEPLOY_STAGING` ไว้
- `lock('employee-console-staging')` — จอกรับพยากรณ์ข้อผิดพลาด (ประกาศใน `casc.yaml`) ให้มี deploy staging ได้ทีละอันเท่านั้น
- `withCredentials([file(...)])` — ดึงไฟล์ `.env.staging` จากที่เก็บความลับของ Jenkins มาเป็นตัวแปร `STAGING_ENV_FILE` ช่วยคร่าว ค่าไม่ถูกพิมพ์ลง log
- `post { always { ... } }`
 - `junit` อ่านไฟล์ XML ใน `test-results/` แล้วแสดงเป็นตารางผล test ในหน้า build
 - `archiveArtifacts` เก็บ `test-results/**`, `playwright-report/**` และ `.deploy/staging-manifest.json` ให้ดาวน์โหลดจากหน้า build
 - `ci-db.mjs down` ล้าง Compose project และ volume ของ build นี้เท่านั้น ไม่แตะ dev หรือ staging

วิธีใช้งาน

1. เปิด Jenkins (controller ใน Docker + agent บนเครื่อง) ต้องมี Java 17 ขึ้นไป

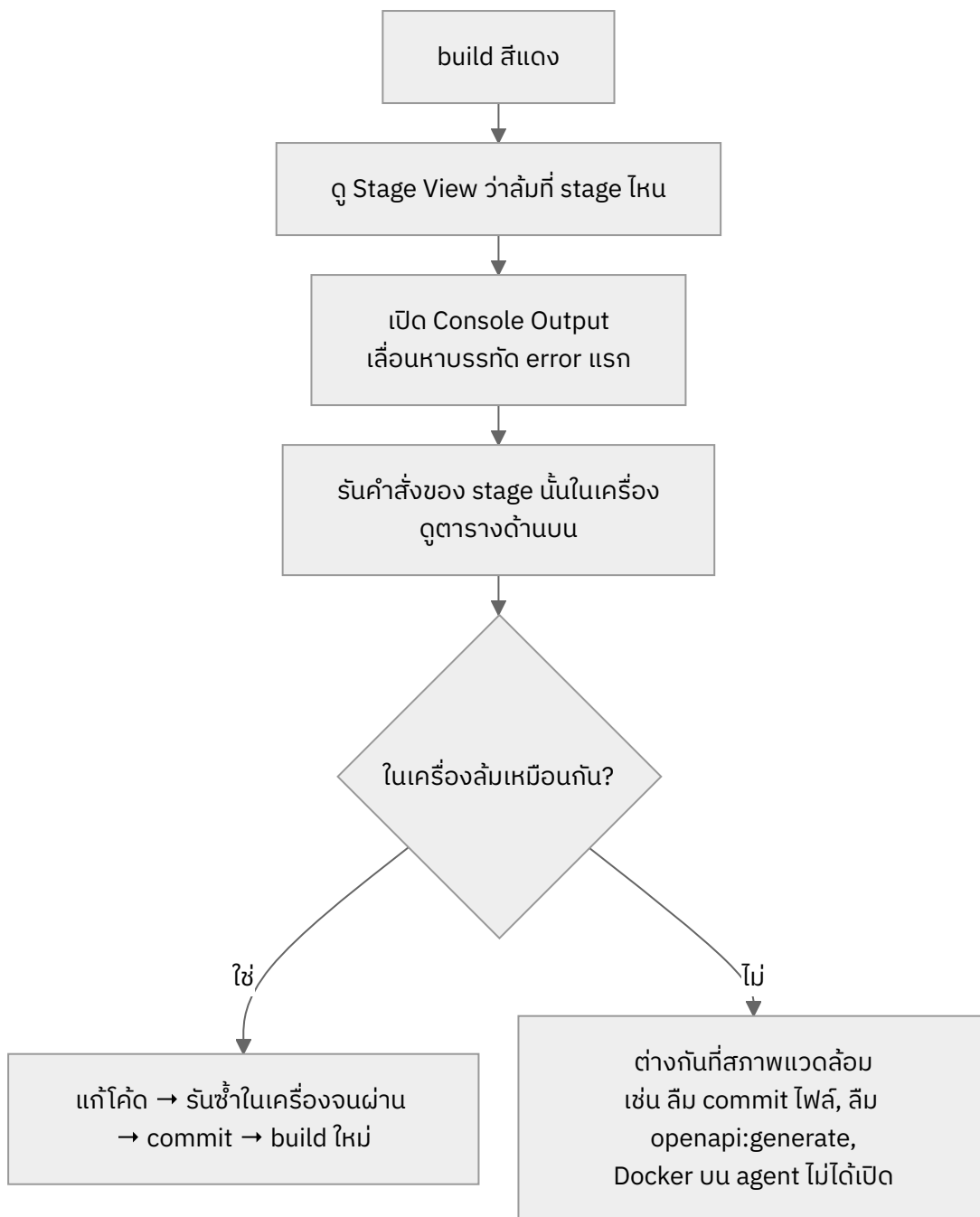
```
pnpm ci:up
```

2. เปิด <http://localhost:8080> แล้ว login ด้วยผู้ใช้ `admin` รหัสผ่านคือค่า `JENKINS_ADMIN_PASSWORD` ในไฟล์ `.env` (เปิดดูในไฟล์เอง อย่าคัดลอกไปวางในแชทหรือเอกสาร)
3. commit โค้ดที่ต้องการให้ Jenkins ทดสอบก่อน
4. เข้า job `employee-console` → **Build with Parameters** → เลือก `DEPLOY_STAGING` ถ้าต้องการ deploy staging จาก branch อื่นที่ไม่ใช่ main → **Build**
5. กดเข้า build ที่กำลังรันเพื่อดู **Stage View** (ช่องเขียว = ผ่าน, แดง = ล้ม) และ **Console Output** (log ทั้งหมด)
6. หลังจบดู **Test Result** และ **Build Artifacts** ในหน้า build
7. ปิด Jenkins (ข้อมูล job ยังอยู่ใน volume `jenkins_home`)

```
pnpm ci:up --stop
```

`pnpm ci:up` ยังตรวจสอบหลังเปิดว่า JCasC ถูก apply ครบ (job, parameters, node `host-agent`) และ plugin ทั้ง 78 ตัวตรงเวอร์ชันที่ pin ไว้ ถ้าไม่ตรงจะ exit 1

เมื่อ build ล้ม ทำอย่างไร



ตัวอย่างข้อความ error ที่เจอบ่อย

ข้อความ	สาเหตุ	แก้
OpenAPI/client drift: run pnpm openapi:generate and commit	แก้ API แต่ไม่ได้ generate	รัน <code>pnpm openapi:generate</code> แล้ว commit <code>packages/api-client</code>
ERR_PNPM_OUTDATED_LOCKFILE	เพิ่ม dependency แต่ไม่ได้ commit <code>pnpm-lock.yaml</code>	<code>pnpm install</code> แล้ว commit lockfile
E2E ผ่านในเครื่องแต่ล้มใน Jenkins	timing หรือข้อมูลค้าง	เปิด <code>playwright-report</code> จาก Build Artifacts ดู screenshot/trace

ประวัติปัญหาจริงที่เคยเจอกับ Jenkins ในโปรเจกต์นี้ (agent หลุด, parameters หาย, migrate ใน image ล้ม) อยู่ใน ai-usage.md (docs/ai-usage.md) ข้อ 16-18

ทำไมเลือกแบบนี้

- **Jenkinsfile** เรียกคำสั่ง `pnpm` เดียวกับที่ใช้ในเครื่อง — ผลใน Jenkins กับในเครื่องจึงตรงกัน และถ้า Jenkins ใช้ไม่ได้ ตอน demo ก็ยัง deploy ด้วย `pnpm staging:up` ได้ (สคริปต์เดียวกัน)
- ทุกอย่าง **pin** เวอร์ชัน — Jenkins image, plugin ทุกตัว, Node, pnpm ทำให้ build วันนีกับเดือนหน้าได้ผลเหมือนกัน
- **ฐานข้อมูลของ CI แยกต่อ build** — test ไม่มีทางไปลบข้อมูล dev หรือ staging

ต่อไป: บท 7 — แนวคิดสำคัญของโปรเจกต์ (docs/learn/07-key-concepts.md)

บท 7 – แนวคิดสำคัญของโปรเจกต์

บทนี้รวมเรื่องที่ทำให้โปรเจกต์นี้ต่างจาก CRUD ทั่วไป และเป็นเรื่องที่มีมักถูกถามตอนนำเสนอ

1. รูปแบบ response: `{ data, meta }` และ `{ error }`

ทุก response ที่สำเร็จมีหน้าตาเดียวกัน

```
{
  "data": { "id": 104, "name": "Bob Brown", "salary": "72000.00", "version": 1, "...": "..." },
  "meta": { "requestId": "3f1c2a9e-..." }
}
```

หน้า list มีข้อมูลการแบ่งหน้าเพิ่มใน `meta`

```
{
  "data": [ { "id": 101, "...": "..." } ],
  "meta": { "requestId": "...", "page": 1, "pageSize": 20, "total": 5, "totalPages": 1,
    "sortBy": "id", "sortOrder": "asc" }
}
```

ทุก error ก็มีหน้าตาเดียวกัน

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Please correct the highlighted fields.",
    "requestId": "...",
    "details": [ { "field": "salary", "code": "DECIMAL_SCALE_EXCEEDED", "message": "Salary must
      have at most 2 decimal places." } ]
  }
}
```

- `code` เป็นค่าคงที่ให้โปรแกรมอ่าน ส่วน `message` ให้คนอ่าน
- `details` บอก error ระดับฟิลด์ หน้าเว็บนำไปแสดงใต้ช่องที่ผิด
- `requestId` ตรงกับ header `X-Request-Id` และบสสทักใน log ผู้ใช้แจ้ง ID นี้มา ก็หา log ของคำขอนั้นเจอทันที

Error code ที่ควรจำ

Status	Code	เกิดเมื่อ
400	VALIDATION_ERROR	body ผิด (ดู details)
400	INVALID_QUERY	query string ผิด เช่น pageSize=1000
400	INVALID_IDEMPOTENCY_KEY	Idempotency-Key ไม่ใช่ UUID
400	MALFORMED_JSON	JSON ผัง
404	EMPLOYEE_NOT_FOUND	ไม่มี ID นี้
409	VERSION_CONFLICT	version ใน If-Match ไม่ตรงกับในฐานข้อมูล
409	IDEMPOTENCY_CONFLICT	ใช้ key เดิมกับ body คนละแบบ
413	PAYLOAD_TOO_LARGE	body เกิน 32 KB
415	UNSUPPORTED_MEDIA_TYPE	ไม่ได้ส่งเป็น application/json
428	PRECONDITION_REQUIRED	แก้หรือลบโดยไม่ส่ง If-Match
429	RATE_LIMITED	ขอถี่เกินไป (มี header Retry-After)
503	DEPENDENCY_UNAVAILABLE	ฐานข้อมูลใช้ไม่ได้

ทั้งหมดอยู่ใน `api-exception.ts` (`apps/api/src/common/api-exception.ts`) และ `http-exception.filter.ts` (`apps/api/src/common/http-exception.filter.ts`)

2.甘การเขียนทับกัน: version + ETag + If-Match

ปัญหา: lost update

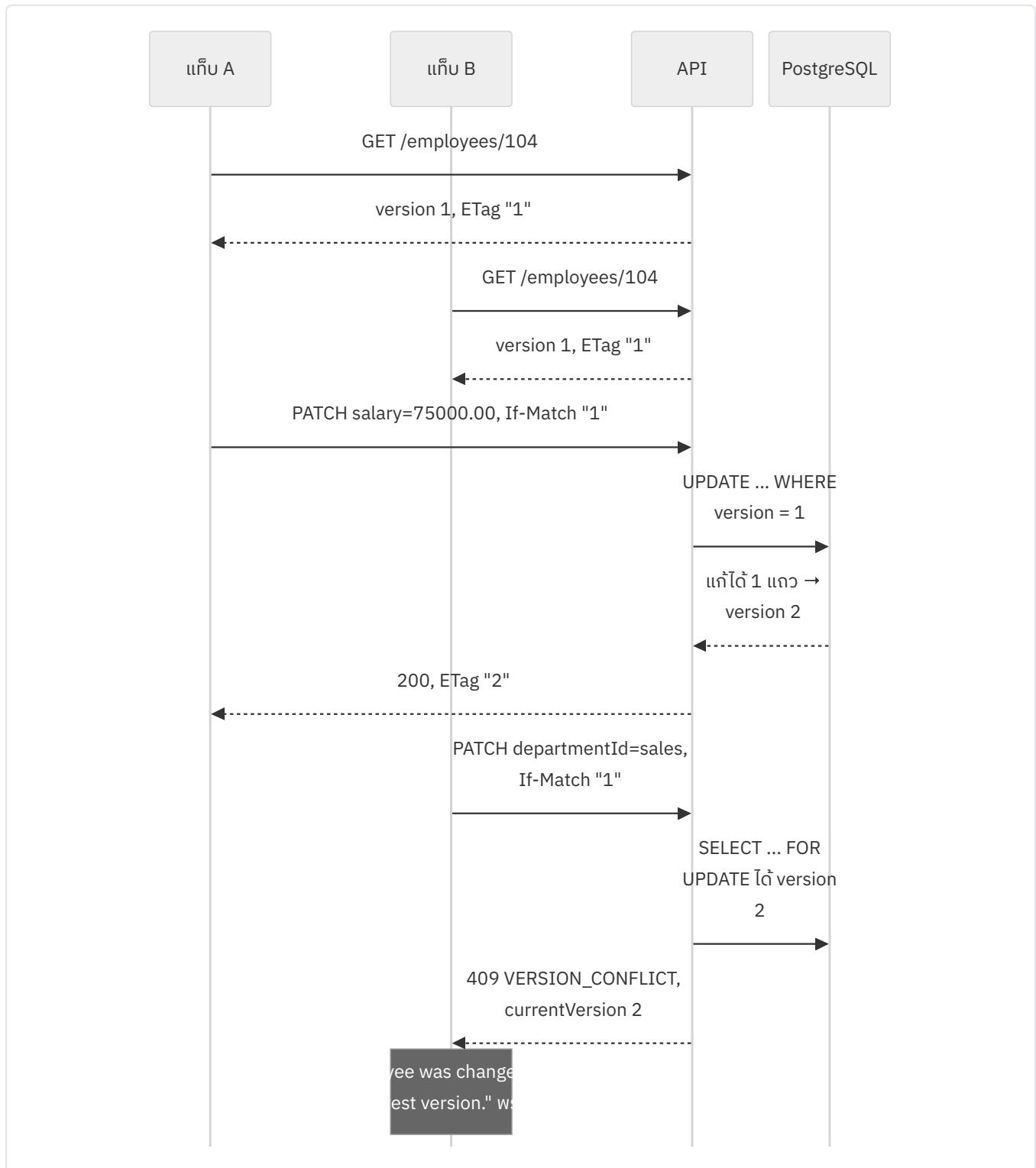
สองคนเปิดหน้า Edit ของ Bob พร้อมกัน คนแรกแก้ไขเงินเดือน คนที่สองแก้ไขแผนก ถ้าระบบไม่ตรวจสอบอะไรเลย คนที่กด Save ทีหลังจะเขียนทับสิ่งที่คนแรกแก้ไขไว้โดยไม่มีใครรู้ตัว

วิธีแก้: optimistic concurrency

ทุกแถวมีคอลัมน์ `version` เริ่มที่ 1 และเพิ่มขึ้นทีละ 1 ทุกครั้งที่แก้ไขสำเร็จ

- ตอนอ่าน API ส่ง version ปัจจุบันกลับใน header `ETag: "1"`
- ตอนแก้ไขหรือลบ client ต้องส่ง version ที่ตัวเองเห็นกลับมาใน `If-Match: "1"`
- ถ้าในฐานข้อมูลยังเป็น 1 → แก้ได้ แล้ว version กลายเป็น 2
- ถ้าในฐานข้อมูลกลายเป็น 2 ไปแล้ว → ตอบ **409** ให้ผู้ใช้โหลดข้อมูลล่าสุดก่อน

เรียกว่า "optimistic" เพราะมองโลกในแง่ดีว่าส่วนใหญ่ไม่ชนกัน จึงไม่ล็อกไว้ตลอดเวลาที่เปิดฟอร์ม แต่ตรวจตอนบันทึก



ถ้าไม่ส่ง `If-Match` เลยจะได้ **428** ไม่ใช่ทำงานต่อแบบไม่ตรวจ ระบบบังคับให้ทุก client ตรวจ version เสมอ (`If-Match` รับทั้ง `"3"` และ `3` — D-23)

ดูโค้ด: `parseIfMatch` ใน `employees.controller.ts` (`apps/api/src/employees/employees.controller.ts`), `update` และ `remove` ใน `employees.service.ts` (`apps/api/src/employees/employees.service.ts`), test "two tabs editing the same record" ใน `employee-crud.spec.ts` (`tests/e2e/specs/employee-crud.spec.ts`) (AC-16)

3. กันการซ้ำซ้ำ: Idempotency-Key

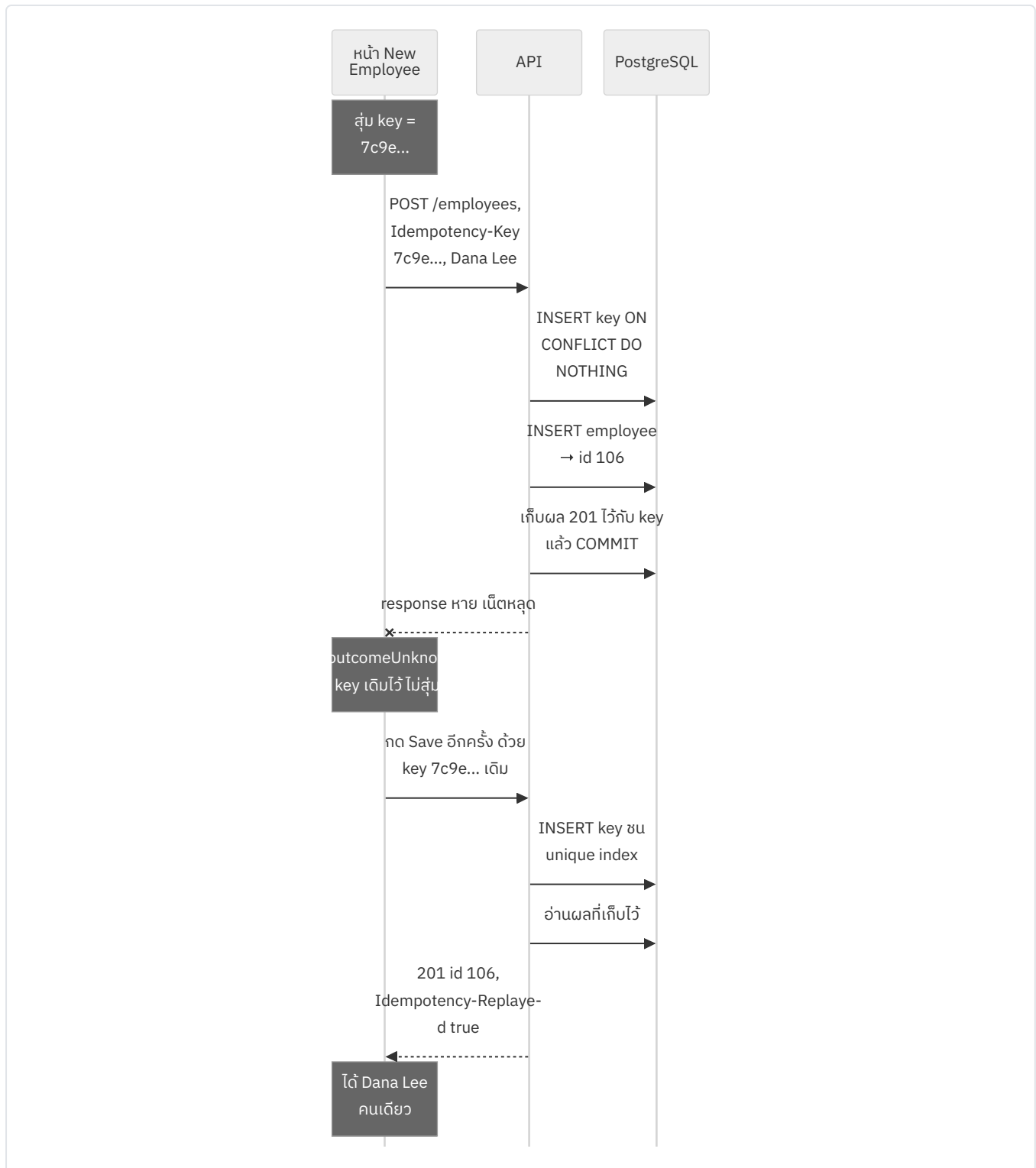
ปัญหา: ไม่รู้ว่าสร้างไปแล้วหรือยัง

ผู้ใช้กด Save เพื่อสร้าง Dana Lee แล้วเน็ตหลุดพอดี หน้าเว็บไม่ได้คำตอบ ไม่รู้ว่า server สร้างไปแล้วหรือยัง ถ้ากดใหม่แล้ว server สร้างอีกรอบ จะได้ Dana Lee สองคน

วิธีแก้: ติดเลขกำกับให้ "ความตั้งใจ" หนึ่งครั้ง

Idempotent แปลว่า ทำซ้ำกี่ครั้งก็ได้ผลเหมือนทำครั้งเดียว

1. ตอนเปิดฟอร์มสร้าง หน้าเว็บสุ่ม UUID หนึ่งตัวเป็น Idempotency-Key
2. ส่ง POST พร้อม key นั้น server บันทึก key กับผลลัพธ์ไว้ 24 ชั่วโมง
3. ถ้าส่งซ้ำด้วย key เดิมและ body เดิม server ไม่สร้างใหม่ แต่ **ส่งผลเดิมกลับ** พร้อม header Idempotency-Replayed: true
4. ถ้าใช้ key เดิมกับ body คนละแบบ → 409 IDEMPOTENCY_CONFLICT



จุดละเอียดที่ควรรู้

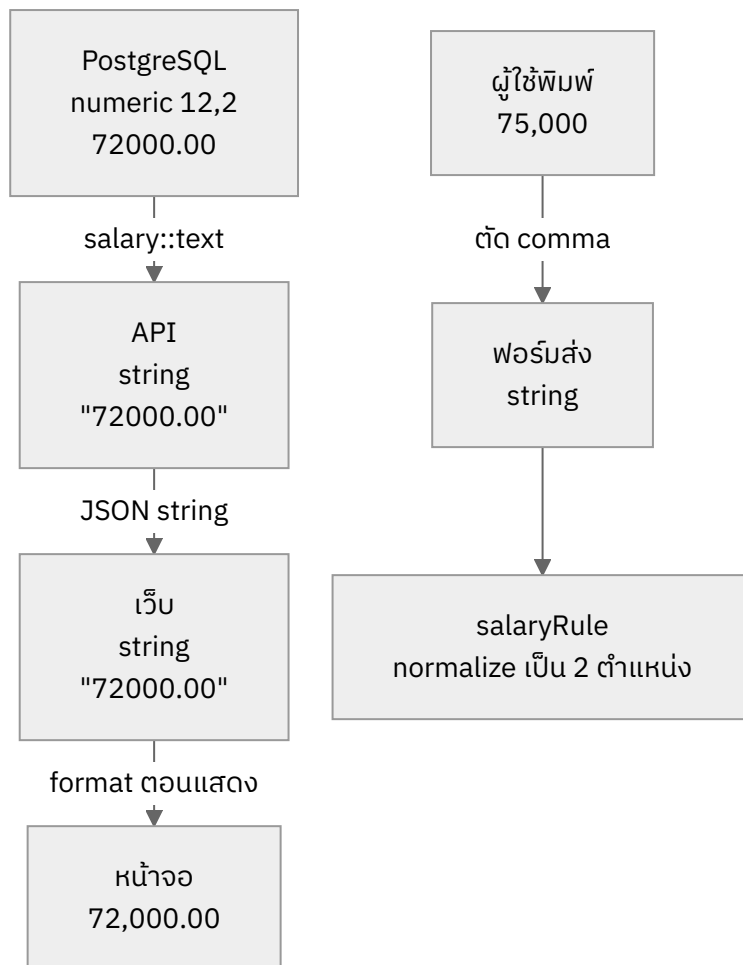
- key กับการสร้างพนักงานอยู่ใน **transaction เดียวกัน** ถ้าสร้างล้ม key ก็ไม่ถูกบันทึก (D-21)
- คำขอพร้อมกัน 5 ครั้งด้วย key เดียวได้พนักงาน 1 คน มี integration test พิสูจน์
- ฝั่งเว็บ **เก็บ key เดิมไว้** เมื่อไม่รู้ผล (network error หรือ 5xx) ดู `outcomeUnknown` ใน `api.ts` (`apps/web/src/lib/api.ts`) — AI เคยทำพลาดตรงนี้โดยสุ่ม key ใหม่เมื่อได้ 5xx (`ai-usage.md` (`docs/ai-usage.md`) ข้อ 13)

ดูโค้ด: `idempotency.service.ts` (`apps/api/src/idempotency/idempotency.service.ts`), test "lost create response" ใน `employee-crud.spec.ts` (`tests/e2e/specs/employee-crud.spec.ts`) (AC-20)

4. เงินเดือนเป็น string ทุกชั้น

```
0.1 + 0.2 // 0.30000000000000004
```

`number` ของ JavaScript เป็น floating point เกือบทุกชนิดบางค่าได้ไม่ตรง เมื่อเป็นเรื่องเงินจึงไม่ใช่ `number` เลย



- ฐานข้อมูลใช้ `numeric(12,2)` เก็บได้ถึง 9,999,999,999.99 แบบแม่นยำ
- API ส่งและรับเป็น string ถ้าส่ง `75000` (ตัวเลข) จะได้ 400 `SALARY_TYPE_INVALID`
- การแสดงผล `#,##0.00` ทำในเว็บเท่านั้น (`format.ts` (`apps/web/src/lib/format.ts`), `salary.ts` (`apps/web/src/lib/salary.ts`))
- ช่อง Salary format ตอน blur ไม่ใช่ตอน focus เพราะเคยเกิดบั๊ก `62000.0063000.00` ที่ Playwright จับได้ (D-31)

5. วันที่แบบไม่มีเวลา (YYYY-MM-DD)

Join Date และ Last Updated Date เป็น "วันที่ล้วน" ไม่มีเวลาและไม่มี timezone

```
new Date('2023-01-15') // = 2023-01-15T00:00:00Z (เที่ยงคืน UTC)
new Date('2023-01-15').toLocaleDateString('en-US', { timeZone: 'America/New_York' })
// "1/14/2023" ← วันเลื่อนถอยไป 1 วัน
```

โปรเจกต์จึงส่งเป็น string "2023-01-15" ตลอดทาง ตั้งแต่ฐานข้อมูล (::text) ถึงฟอร์ม ไม่แปลงเป็น Date เลย (D-05) E2E test ตรวจสอบว่าวันที่ไม่เลื่อนเมื่อเปิดเบราว์เซอร์ในหลาย timezone (AC-11 ใน dates-and-layout.spec.ts (tests/e2e/specs/dates-and-layout.spec.ts))

ส่วน **Last Updated Date** ระบบตั้งให้เป็น "วันนี้" ตามเขตเวลา Asia/Bangkok (APP_TIMEZONE) ด้วย businessDate() ใน dates.ts (apps/api/src/common/dates.ts) ส่วน timestamp อย่าง created_at เก็บเป็น UTC

6. แก้แต่ไม่เปลี่ยนอะไร = ไม่เขียน (no-op)

ถ้าผู้ใช้กด Save โดยค่าเหมือนเดิมทุกช่อง (หลัง normalize เช่น ตัดช่องว่างแล้ว) API ตอบ 200 พร้อม meta.changed: false ไม่เขียนฐานข้อมูล version และ Last Updated Date คงเดิม (PRD §9.4, AC-14)

7. ข้อมูลตั้งต้นจาก Excel ต้องไม่ผิด

จุด	ทำอะไร
Bob Brown มีสถานะ In Active (มีช่องว่าง)	seed map เป็น is_active = false ห้ามใช้ Boolean("In Active") เพราะได้ true
Last Updated Date เดิมของ 5 records	seed เก็บค่าจาก Excel ไม่เปลี่ยนเป็นวันนี้
ID 101-105	seed ใส่ ID เดิม แล้วตั้ง sequence ให้คนใหม่เริ่มที่ 106
รูปแบบ 65,000.00	เป็นการแสดงผลเท่านั้น ค่าจริงคือ "65000.00"

ดู seed-original.ts (apps/api/src/seed/seed-original.ts) และ unit test seed-mapping.test.ts (apps/api/test/unit/seed-mapping.test.ts)

8. สถานะของหน้า list อยู่ใน URL

ค้นหา กรอง เรียง และเลขหน้าเก็บใน query string เช่น /employees?q=john&departmentId=engineering&status=inactive&page=2 จึง reload, กด back/forward หรือส่งลิงก์ให้คนอื่น แล้วเห็นผลเดียวกัน (PRD §8.3) ดู list-params.ts (apps/web/src/lib/list-params.ts)

9. Rate limit ต่อ IP

guard นับคำขอต่อ IP ในหน่วยความจำ แบบ fixed window 1 นาที อ่านได้ 300 ครั้ง เขียนได้ 60 ครั้ง เกินแล้วได้ 429 พร้อม Retry-After เขียนเองแทน @nestjs/throttler เพราะ policy เฉพาะและมี API instance เดียว (D-17) ปิดได้เฉพาะ APP_ENV=test (D-36)

10. Health check สองแบบ

Endpoint	ตอบ 200 เมื่อ	ใช้ทำอะไร
/api/health/live	process ยังตอบได้	บอกว่าแอปไม่ค้าง
/api/health/ready	ฐานข้อมูลต่อได้ และ migration ล่าสุดที่มากับ build ถูก apply แล้ว	Docker healthcheck และ smoke test ใช้ตัดสินว่ารับงานได้หรือยัง

ดู health.controller.ts (apps/api/src/health/health.controller.ts)

ต่อไป: บท 8 – การทดสอบ (docs/learn/08-testing.md)

บท 8 – การทดสอบ

ภาพรวม: พีระมิดของ test



ยิ่งอยู่ล่าง ยิ่งเร็วและมีจำนวนเยอะ ยิ่งอยู่บน ยิ่งช้าแต่ใกล้เคียงผู้ใช้จริง แต่ละชั้นจับปัญหาคนละแบบ ไม่ได้แทนกัน

ชั้น	เครื่องมือ	อยู่ที่	ต้องมี Docker	ตัวอย่างสิ่งที่ตรวจ
Unit	Vitest	apps/api/test/unit, apps/web/src/lib/*.test.ts	ไม่	salaryRule("65000") ได้ "65000.00", seed map In Active เป็น false, config ผิดแล้วหยุด
Integration	Vitest + Supertest	apps/api/test/integration	ใช่	สร้างได้ ID 106 และวันนี้ตามเวลากรุงเทพ, ส่ง id มาใน body โดนปฏิเสธ, 5 คำขอพร้อมกัน ด้วย key เดียวได้ 1 แถว
Contract	Newman (Postman CLI)	tests/postman	ใช่	q=% คั่นแบบตัวอักษรจริง ไม่ใช่ wildcard, ส่ง salary เป็นตัวเลขได้ SALARY_TYPE_INVALID
E2E	Playwright	tests/e2e/specs	ใช่	สร้าง → แก้ → ลบ, สองแท็บชนกันได้ 409, วันที่ ไม่เลื่อนในหลาย timezone, จอ 375 px ไม่ล้น

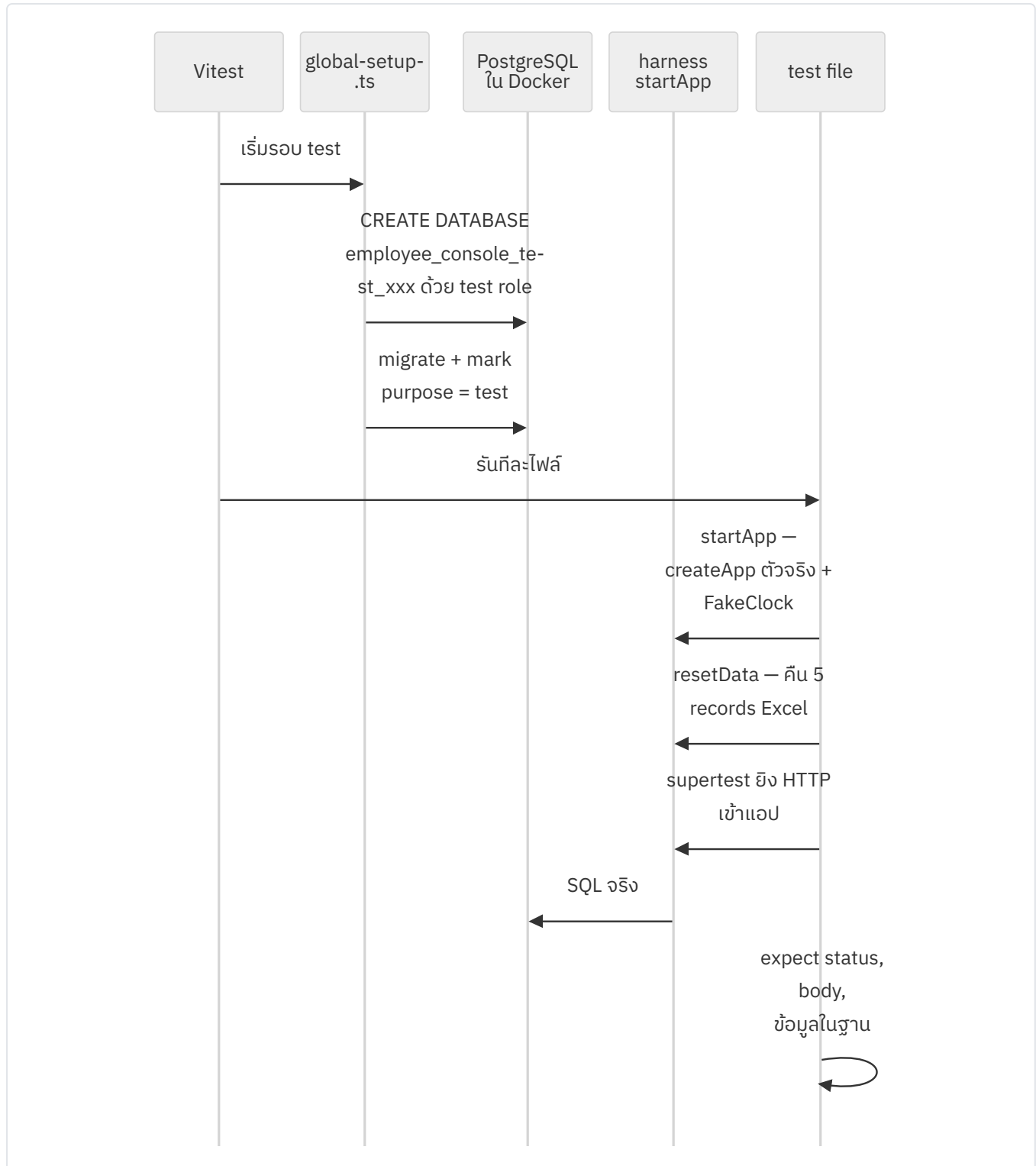
Unit test – เร็วที่สุด

ทดสอบฟังก์ชันบริสุทธิ์ (pure function) ที่รับค่าแล้วคืนค่า ไม่มีฐานข้อมูล ไม่มี HTTP เพราะกฎของฟิวด์ทุกตัวเขียนเป็น pure function (D-19) จึงทดสอบละเอียดได้ทุกกรณีในเวลาไม่กี่วินาที

```
pnpm test:unit
```

Integration test – API ท้าก์่อนกับ PostgreSQL จรง

ไม่ใช้ฐานข้อมูลจำลอง เพราะพฤติกรรมสำคัญหลายอย่าง (transaction, row lock, unique index, `numeric`, `date`) จำลองไม่ได้



- `global-setup.ts` (`apps/api/test/support/global-setup.ts`) สร้างฐานชั่วคราวของรอบนั้น และ `assertTestDatabase` กันไม่ให้รันกับฐาน demo หรือ staging
- `harness.ts` (`apps/api/test/support/harness.ts`) เปิดแอปด้วย `createApp()` ตัวเดียวกับของจริง แต่ใส่ `FakeClock` ตรึงเวลาไว้ที่ 2026-10-01 10:00 กรุงเทพ ผลจึงเหมือนเดิมทุกวัน
- ใช้ Vitest + SWC (`unplugin-swc`) ไม่ใช้ esbuild เพราะ Nest DI ต้องใช้ decorator metadata (D-13)

```
pnpm dev:up
```

```
pnpm test:api
```

Contract test — Postman / Newman

Postman เป็นโปรแกรมยิง API ที่บันทึกชุดคำขอเป็น "collection" ได้ ส่วน **Newman** คือตัวรัน collection นั้นจาก command line ใช้ใน CI ได้

collection ของโปรเจกต์ (employee-console.postman_collection.json (tests/postman/employee-console.postman_collection.json)) แบ่งเป็นกลุ่ม เช่น

- **Health** — live, ready
- **Employees (happy path)** — list, ค้นหา, กรอง, สร้าง Dana Lee, replay ด้วย key เดิม, PATCH ด้วย `If-Match`, no-op
- **Employees (validation & concurrency negatives)** — key ซ้ำแต่ body ต่าง, salary ทศนิยม 3 ตำแหน่ง, ส่ง `id` มาใน body

เปิดไฟล์นี้ในโปรแกรม Postman แล้วเลือก environment `local` ก็กดยิงทีละคำขอเพื่อเรียนรู้ API ได้ (local.postman_environment.json (tests/postman/local.postman_environment.json))

```
pnpm test:postman
```

E2E test — Playwright

Playwright เปิดเบราว์เซอร์จริง (Chromium) แล้วคลิก พิมพ์ และตรวจหน้าจอเหมือนผู้ใช้ runner (scripts/test-e2e.mjs) build แอปแบบ production สร้างฐานทดสอบชั่วคราว แล้วเปิด web/API บนพอร์ต 3020/3021

ตัวอย่าง test ใน employee-crud.spec.ts (tests/e2e/specs/employee-crud.spec.ts)

- seed data: 5 records with all 7 fields; Bob Brown is In Active (AC-01, AC-03, AC-26)
- create → edit → delete journey (AC-04, AC-12, AC-13, AC-17)
- two tabs editing the same record: the stale tab gets a conflict (AC-16)
- lost create response: retry with the same key lands on the same record, no duplicate (AC-20)

และใน dates-and-layout.spec.ts (tests/e2e/specs/dates-and-layout.spec.ts)

- dates do not shift in <timezone> (AC-11)
- mobile 375px: no page overflow, stacked filters, usable form (AC-35)
- keyboard only: focus order, visible focus, checkbox, dialog Escape, submit with Enter (AC-35)

```
pnpm test:e2e
```

เก็บภาพหน้าจอที่ 375 / 1024 / 1440 px ไว้ด้วย

```
E2E_SCREENSHOT_DIR=./screenshots pnpm test:e2e
```

สหัส **AC-xx** คืออะไร

AC ย่อมาจาก **Acceptance Criteria** คือข้อกำหนดที่ต้องผ่านใน `prd.md` ([docs/prd.md](#)) §16 ชื่อ test ทุกชั้นอ้างรหัสนี้ไว้ ทำให้ตอบได้ว่า "ข้อกำหนดข้อนี้ถูกทดสอบที่ไหน" ด้วยการค้นหา เช่น

```
grep -rn "AC-16" apps/api/test tests/
```

ก่อนบอกร่างงานเสร็จ

จาก `AGENTS.md`

```
pnpm lint && pnpm typecheck && pnpm test:unit && pnpm build
```

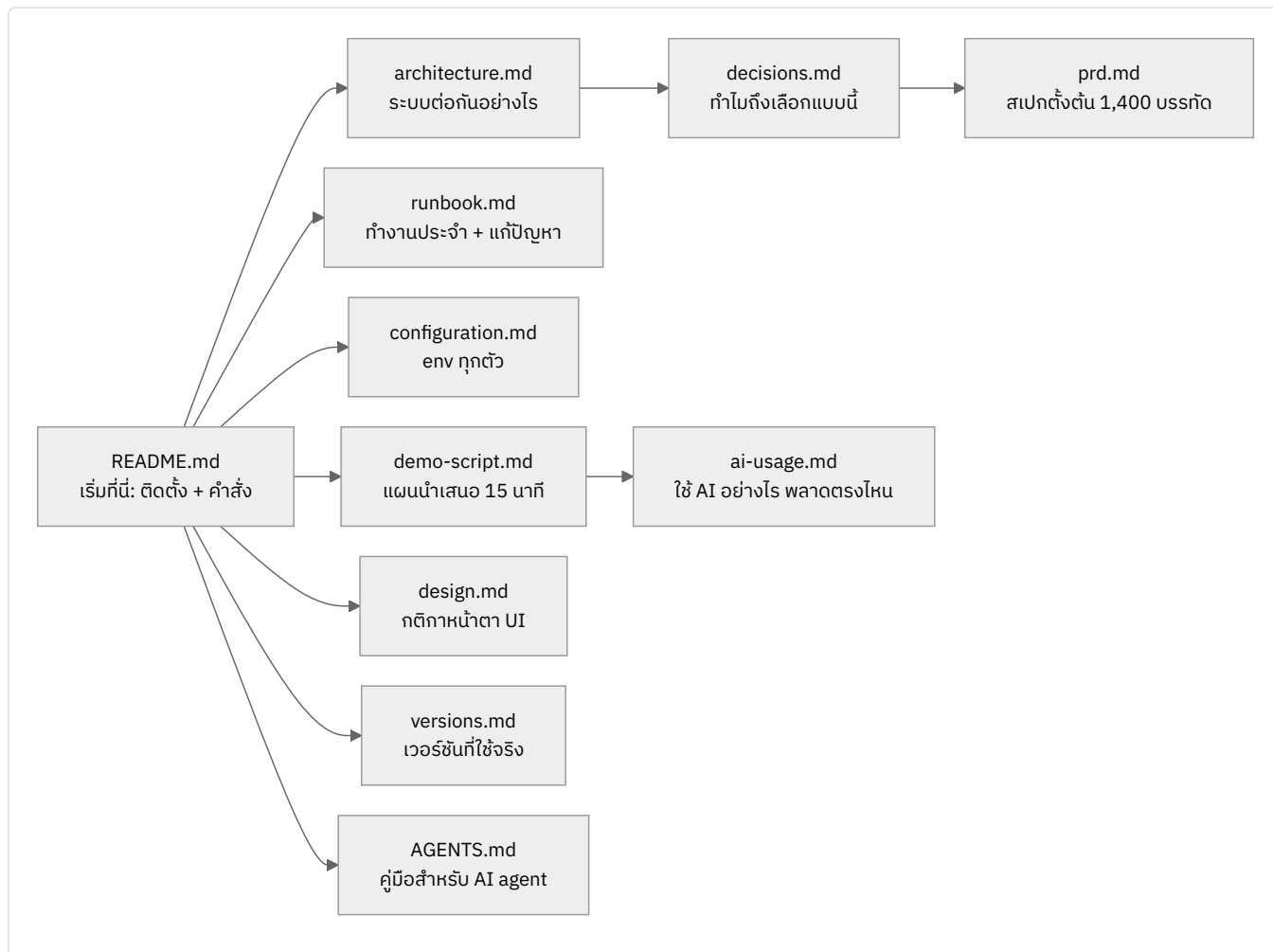
- แก้ API → รัน `pnpm test:api` และ `pnpm openapi:generate` แล้วดูว่า `packages/api-client` ไม่มีอะไรค้าง
- แก้ UI หรือ flow → รัน `pnpm test:e2e`

กับดัก: อย่ารัน `playwright test` หรือ `newman` ตรง ๆ ให้ใช้คำสั่ง `pnpm` เสมอ เพราะ runner เป็นคนสร้างฐานข้อมูลชั่วคราว เปิดแอปบนพอร์ตเฉพาะ และเขียน env ให้ ถ้ารันตรง ๆ อาจไปลบข้อมูลในฐาน dev (`tests/AGENTS.md`)

ต่อไป: บท 9 – แผนที่เอกสาร ([docs/learn/09-docs-guide.md](#))

บท 9 – แผนที่เอกสาร ฉบับอ่านง่าย

เอกสารในโปรเจกต์เขียนให้ "แม่นยำ" มากกว่า "อ่านง่าย" บทนี้สรุปแต่ละไฟล์เป็นภาษาธรรมดา บอกว่าไฟล์นี้มีไว้ทำอะไร ควรเปิดตอนไหน และใจความสำคัญคืออะไร



README.md – ประตุนำบ้าน

README.md บอกว่าโปรเจกต์ทำอะไร ต้องติดตั้งอะไร และมีคำสั่งอะไรบ้าง

- **อ่านเมื่อ:** วันแรกที่เปิดโปรเจกต์ หรือลืมนำคำสั่งไหนทำอะไร
- **ใจความ:** ต้องมี Node 24, pnpm 12, Docker Desktop (และ Java 17+ ถ้าจะใช้ Jenkins) ขั้นตอนเริ่มคือ `pnpm install` → `pnpm run setup` → `pnpm dev:up` → `pnpm dev` ตาราง "คำสั่งทั้งหมด" คือสารบัญของ `package.json`

AGENTS.md และ CLAUDE.md — คู่มือสำหรับ AI

AGENTS.md ที่ root และในแต่ละ folder หลัก (`apps/api` , `apps/web` , `packages/api-client` , `tests` , `infra` , `scripts`) เขียนให้ AI coding assistant อ่านก่อนแก้ไขโค้ด ส่วน `CLAUDE.md` แค่ import ไฟล์ `AGENTS.md`

- **อ่านเมื่อ:** อยากรู้ "กฎที่อ่านจากโค้ดไม่ออก" เช่น salary ต้องเป็น string, import ต้องลงท้าย `.js` , ห้ามแก้ไข migrationเก่า
- **ใจความ:** คนก็อ่านได้ดีเหมือนกัน เพราะเป็นรายการกับดักที่สั้นที่สุดของแต่ละส่วน

architecture.md — ระบบต่อกันอย่างไร

`architecture.md` (`docs/architecture.md`) มีแผนภาพ service, ตาราง module ของ API, ลำดับชั้นของ request และ data model

- **อ่านเมื่อ:** ต้องอธิบายระบบให้คนอื่นฟัง หรือจะเพิ่ม module ใหม่
- **ใจความ**
 - เบราร์เซอร์คุยกับ Next.js ที่เดียว Next.js ส่ง `/api/*` ต่อให้ NestJS และไม่แตะฐานข้อมูล
 - API แบ่งเป็น module ตามงาน (`employees` , `departments` , `health` , `rate-limit` , `config` , `common`) แต่ละ module มี Controller → Service → Prisma
 - request ผ่าน 6 ชั้น: request ID → security header/ตรวจ JSON → rate limit → validation → service + interceptor → exception filter (ตรงกับภาพในบท 2)
 - ด้านความปลอดภัย: ไม่เปิด CORS, SQL ใช้ parameter ทั้งหมด, log ไม่บันทึก salary หรือ body

configuration.md — ตัวแปร env ทุกตัว

`configuration.md` (`docs/configuration.md`) อธิบาย environment variable ทั้งหมดที่ API อ่าน

- **อ่านเมื่อ:** API ไม่ยอม start (`Invalid configuration`) หรือจะเพิ่ม env ใหม่
- **ใจความ**
 - `pnpm run setup` สร้าง `.env` และ `.env.staging` พร้อม secret แบบสุ่มให้เอง ไม่ต้องกรอกอะไร
 - `APP_ENV` (`local` , `staging` , `test`) เป็นตัวคุมว่าคำสั่งอันตรายทำได้หรือไม่
 - `API_INTERNAL_URL` ถูกฝังตอน build เว็บ ถ้าเปลี่ยน `PORT` ต้องเปลี่ยนตัวนี้ด้วย
 - ค่าทุกตัวถูกตรวจด้วย zod ตอน start ผิดแล้วหยุดทันที
 - ห้ามใส่ secret ใน `NEXT_PUBLIC_*` และห้าม commit `.env*` (ยกเว้น `.example`)
- เพิ่ม env ใหม่ต้องแก้สามที่: `app-config.ts` (`apps/api/src/config/app-config.ts`), `.env.example` และ ไฟล์นี้

runbook.md — คู่มือปฏิบัติงาน

runbook.md (docs/runbook.md) คือ "ทำอะไรเมื่อ..." มี 6 หัวข้อ

หัวข้อ	ใช้เมื่อ
1-2 ตั้งเครื่อง, Dev	วันแรก, พอร์ตชน
3 Reset ข้อมูล demo	อยากให้ข้อมูลกลับเป็น 5 records ก่อน demo
4 Local staging	deploy, smoke, restart เมื่อแก้ <code>.env.staging</code> , rollback, restore backup
5 Jenkins	เปิด Jenkins, login, กด build, อัปเดต plugin
6 ปัญหาที่พบบ่อย	หน้าเว็บ error, API start ไม่ได้, test ล้มเรื่องสิทธิ์ฐานข้อมูล

ประโยคที่ควรจำ: **rollback ไม่ยื้อ schema** เพราะ migration ออกแบบให้ image เก่ายังใช้ได้ ยกเว้น migration ที่ลบตาราง Login/Reports ซึ่งต้อง restore backup

decisions.md — ทำไมถึงเลือกแบบนี้

decisions.md (docs/decisions.md) เป็นบันทึกการตัดสินใจ (decision log) แต่ละข้อมีรหัส `D-xx` พร้อมเหตุผล D-01 ถึง D-12 มาจาก PRD ส่วน D-13 ขึ้นไปตัดสินใจระหว่างลงมือทำ กติกาคือ **เพิ่มแถวใหม่เท่านั้น ห้ามแก้แถวเก่า** ถ้าเปลี่ยนใจให้เขียนแถวใหม่ว่าแทนที่ข้อไหน

- **อ่านเมื่อ:** มีคนถามว่า "ทำไมไม่ใช้ X" หรือเห็นโค้ดแปลก ๆ แล้วสงสัย

จัดกลุ่มให้อ่านง่าย

กลุ่ม	ข้อ	สรุปสั้น ๆ
Stack และเวอร์ชัน	D-01, D-13-D-16	Next 16 + Nest 12 + Postgres 17 + Prisma 7; ไม่ใช้ <code>latest</code> ที่เป็น RC หรือยังไม่เข้ากัน; test ใช้ Vitest + SWC
ความถูกต้องของข้อมูล	D-02, D-04, D-05, D-19, D-20, D-22, D-31, D-37	seed ไม่กับ, เงินเป็น decimal string, วันที่เป็น string, กฎเป็น pure function, SQL ตรง ๆ เพื่อคุมชนิดข้อมูล, ป้ายบอกชนิดฐาน, reset ใน transaction เดียว
การเขียนพร้อมกัน	D-06, D-07, D-21, D-23	ลบจริง + ยืนยันก่อน, version + If-Match, idempotency ใน transaction เดียว
ความปลอดภัยและการเดินระบบ	D-17, D-35 (เหตุผลปัจจุบันอยู่ที่ D-50), D-36, D-43, D-45	rate limit เขียนเอง, trust proxy แค่ hop เดียว, ปิด limiter ได้เฉพาะ test, role ฐานข้อมูลแยก, secret ของ test สุ่มใหม่ทุกครั้ง
Delivery และ CI/CD	D-10, D-29, D-30, D-33, D-34, D-41, D-44, D-54	staging :3100, Jenkins + agent บนเครื่อง, tag = SHA (<code>-dirty</code>), readiness ตรวจ migration, <code>pnpm run setup</code> , state ของ staging อยู่ใน home, pin plugin ทุกตัว, CI build ครั้งเดียวต่อ run
Performance	D-32, D-52 (ยกเลิก D-11)	index แผนก+สถานะ และ trigram index ค้นหาชื่อ อยู่ใน migration แยก; ชุด performance test ถูกตัดออก (D-52)
เอกสาร API	D-42, D-53	OpenAPI อธิบาย envelope และ header ครบ; รายการแผนกมาจาก <code>DEPARTMENT_IDS</code> ใน API → enum ใน OpenAPI → <code>lib/departments.ts</code> ของเว็บ
ขอบเขต	D-08, D-12, D-46 , D-51, D-55	UI อังกฤษ/เอกสารไทย; ไม่ทำ Redis, K8s; D-46 ตัด Login และ AI reports ออก
ยกเลิกแล้วโดย D-46 (อ่านเป็นประวัติ)	D-03, D-09, D-18, D-24-D-28, D-38-D-40	Google OAuth, session, สิทธิ์ Admin/Viewer, n8n + Gemini

design.md – กติกาหน้าตา UI

design.md (docs/design.md) อธิบายว่าทำไม UI หน้าตาเหมือนโปรแกรม ERP/HR ขององค์กร (ตารางแน่น มีเส้นกริดครบ) แทนที่จะเป็น dashboard สำเร็จรูป

- **อ่านเมื่อ:** จะแก้หน้าตาเว็บ
- **ใจความ**
 - ส้อยู่ใน `apps/web/src/app/globals.css` เป็นโทนเขียว Chememan สีแดงใช้กับการลบและ error เท่านั้น
 - ฟอนต์ IBM Plex Sans Thai ตระกูลเดียว ขนาด 14 px ตาราง 13 px ตัวเลขใช้ `tabular-nums`
 - ไม่ใช้ของที่ทำให้ดูเป็น template: sidebar ไอคอน, การ์ดลอยมุมโค้ง, badge มีกรอบ, หัวข้อตัวใหญ่, gradient
 - Accessibility: ทุก input มี label, สถานะไม่พ้องสื่ออย่างเดียว (Active = จุดทึบ, In Active = วงกลวง), ใช้คีย์บอร์ดได้ครบ

prd.md – สเปกเริ่มต้น

prd.md (docs/prd.md) ยาวราว 1,400 บรรทัด เขียนก่อนเริ่มโค้ดเพื่อสั่งงาน AI โค้ดอ้างอิงเป็น `PRD §...` และรหัส `REQ-xx`, `USR-xx`, `AC-xx`

- **อ่านเมื่อ:** อยากรู้ว่าข้อกำหนดเดิมคืออะไร หรือเห็น `PRD §9.4` ในคอมเมนต์แล้วอยากตามไปอ่าน
- **ไม่ต้องอ่านทั้งเล่ม** ใช้ตารางนี้เลือก

ส่วน	สถานะ	อ่านเพื่อ
§3–§4 โจทย์และข้อมูล Excel	ใช้อยู่	รู้ว่าโจทย์คืออะไร และกับดักของข้อมูล (Bob Brown <code>In Active</code> , วันที่เดิมต้องคงไว้)
§7 Architecture	ใช้อยู่	ทำไมแยก Next/Nest
§8 หน้าจอและ user journey	ใช้อยู่	หน้า list, create/edit/delete ต้องทำงานอย่างไร
§9 Data model และกฎธุรกิจ	ใช้อยู่	validation, วันเวลา, concurrency, seed
§10 API contract	ใช้อยู่ (§10.6 เหลือแค่ <code>DepartmentDto</code>)	endpoint, error code, idempotency
§11 Authentication	เหลือแค่หัวข้อ (D-46) ยกเว้น §11.4 ที่ยังใช้	§11.4: HTTPS, CORS, rate limit, redact log
§12 AI report, n8n, Gemini	เหลือแค่หัวข้อ (D-46)	—
§13 Environment และคำสั่ง	ใช้อยู่ (ตัด env ของ auth/AI)	env, health, logging
§14 CI/CD	ใช้อยู่	stage ที่ต้องมีและ rollback policy
§15 Performance	เหลือแค่หัวข้อ (D-52)	—
§16 Acceptance criteria	ใช้อยู่ ยกเว้น §16.4 (AI, เหลือแค่หัวข้อ) และส่วนสิทธิ์ใน §16.3	รหัส <code>AC-xx</code> ที่ test อ้างถึง
§17 แผนสำหรับ AI	ใช้อยู่	ลำดับงานและข้อปฏิบัติของ AI

versions.md — เวอร์ชันที่ใช้จริง

versions.md (docs/versions.md) บันทึกเวอร์ชันที่ติดตั้งและรันจริง ไม่ใช่ที่คาดหวัง

- **อ่านเมื่อ:** มีคนถามว่าใช้เวอร์ชันอะไร หรือทำไมไม่อัปเดต
- **ใจความ:** Node 24.21.0, pnpm 12.8.1, TypeScript 5.9.3 (ไม่ใช่ 7 เพราะ @nestjs/swagger ยังไม่รองรับ), Prisma 7.10.0 (latest เป็น 8.0 RC), ESLint 9 (plugin ของ Next ยังไม่รองรับ 10)

ai-usage.md — ใช้ AI อย่างไร และ AI พลาดตรงไหน

ai-usage.md (docs/ai-usage.md) บันทึกวิธีใช้งาน AI และปัญหาจริง 21 ข้อที่เจอระหว่างพัฒนา

- **อ่านเมื่อ:** เตรียมตอบคำถามเรื่องการใช้ AI ตอนสัมภาษณ์
- **ใจความ**
 - วิธีการ: ส่ง PRD ที่ฉบับให้ AI → ตรวจสอบเข้ากันของเวอร์ชันก่อน → ทำทีละชิ้นพร้อม test → ให้ test เป็นตัวตัดสิน ไม่ใช่การอ่านโค้ด
 - ตัวอย่างที่ AI พลาดแล้ว test จับได้: ช่อง Salary พิมพ์แล้วข้อความต่อกัน (Playwright จับ), สุ่ม Idempotency-Key ใหม่เมื่อได้ 5xx, เปิด /api/docs-yaml โดยไม่ตั้งใจ (reviewer จับ), app role มีสิทธิ์สร้างฐานข้อมูล (reviewer จับ)
 - ข้อ 4, 6, 14, 15 และตัวอย่างใน "สิ่งที่ AI ไม่ได้ทำแทน" เป็นเรื่องของ Login/session/reports ซึ่งถูกตัดออกแล้ว (D-46) อ่านเป็นประวัติ

demo-script.md — แผนนำเสนอ 15 นาที

demo-script.md (docs/demo-script.md) คือนำเสนอ

- **อ่านเมื่อ:** ก่อนวันนำเสนอ
- **ใจความ**
 - เตรียมล่วงหน้า: build staging ไว้ก่อน, pnpm staging:smoke ต้องผ่าน, reset ให้เหลือ 5 records
 - ลำดับ: เปิดหน้า Employees → ทำ CRUD ตามสคริปต์ → เปิดโค้ด → เล่าเรื่องการใช้ AI → Jenkins (stages, build ครั้งเดียวต่อ run, smoke/rollback) → Q&A
 - มีโจทย์ข้อ Live Coding สองข้อ (filter Join Date และเพิ่ม validation) ซึ่งบุน 2 อธิบายลำดับการแก้ไข
 - คำถามที่ควรตอบได้ อยู่ในบท 10 พร้อมคำตอบ

ข้อสังเกต: ร่องรอยของระบบที่ตัดออก

ตอนเขียนคู่มือเคยพบจุดที่ยังพูดถึงระบบที่ตัดออกไปแล้วหรือไม่ตรงกับโค้ด (แถบเมนูใน design.md, ค่า default ของ TRUST_PROXY, เหตุผลของ D-35, smoke ข้อ /internal/..., ชื่อไฟล์ e2e ที่มีคำว่า admin, mapping 401/403) ตอนนี้แก้ครบแล้วตาม D-50, D-51 และ D-55 ส่วนที่เหลือไว้โดยตั้งใจมีดังนี้

ที่	ทำไมยังอยู่
prd.md (docs/prd.md) หัวข้อ Login/AI reports	เหลือแค่หัวข้อเปล่า เพราะโค้ดและ migration อ้างเลขหัวข้อและรหัส AC อยู่ (D-48)
decisions.md (docs/decisions.md) แถวที่มีป้าย ยกเลิก (D-46)	decision log ห้ามแก้ไขของเดิม ให้เพิ่มแถวใหม่แทน
ai-usage.md (docs/ai-usage.md) แถวที่มีป้าย ประวัติก่อน D-46	เป็นบันทึกการใช้ AI ระหว่างพัฒนา ไม่ใช่โค้ดที่ยังมีอยู่
migration 20261002000000_perf_indexes	ชื่อยังมีคำว่า perf แม้ชุด performance test ถูกตัดแล้ว (D-52) แต่ index ยังใช้อยู่ และ migration ที่ apply แล้วเปลี่ยนชื่อไม่ได้
migration 20261001000000_init	สร้างตาราง users, sessions, reports, integration_state และ enum Role ไว้ และ migration ที่ apply แล้วห้ามแก้ไข จึงลบใน 20261003000000_remove_login_and_reports แทน

ต่อไป: บท 10 – เตรียมตอบคำถาม (docs/learn/10-qa-prep.md)

บท 10 – เตรียมตอบคำถาม

คำถามชุดแรกมาจากรายการ "คำถามที่ควรตอบได้" ใน `demo-script.md` (`docs/demo-script.md`) ชุดที่สองเป็นคำถามที่คนไม่คุ้นกับ NestJS, OpenAPI หรือ Jenkins มักถูกถาม ทุกคำตอบมีไฟล์ให้เปิดประกอบ

วิธีใช้: อ่านคำตอบแล้วลองพูดด้วยคำของตัวเอง ไม่ต้องท่องจำ ถ้าโดนถามลืกละให้เปิดไฟล์ที่ระบุไว้

คำถามจาก demo-script

ทำไมแยก Next.js กับ NestJS

แยกเพื่อให้กฎธุรกิจมีที่อยู่ที่เดียว Next.js ทำแค่ UI กับส่งต่อ `/api/*` ไม่ต่อฐานข้อมูลและไม่มี Server Actions ทำ CRUD ซ้ำ ทุกการตรวจข้อมูล, concurrency และ error code อยู่ใน NestJS ผลคือทดสอบ API ได้ตรง ๆ ด้วย integration test กับ Postman โดยไม่ต้องผ่านหน้าเว็บ มี OpenAPI เป็นสัญญาาระหว่างสองฝั่ง และบน staging พอร์ตของ API ไม่ต้องเปิดออกนอก Docker เลย เบราวเซอร์เห็นแค่ origin เดียว

เปิดประกอบ: `next.config.ts` (`apps/web/next.config.ts`), `architecture.md` (`docs/architecture.md`), `compose.staging.yaml`

ทำไมเงินเป็น decimal string

`number` ของ JavaScript เป็น floating point บวกทศนิยมแล้วเพี้ยนได้ เช่น $0.1 + 0.2$ ไม่เท่ากับ 0.3 พอดี ฐานข้อมูลจึงเก็บเป็น `numeric(12,2)` แล้วส่งออกเป็น string ด้วย `salary::text` และรับเข้าเป็น string เท่านั้น ถ้าส่งเป็นตัวเลขมาจะได้ 400 `SALARY_TYPE_INVALID` การใส่ comma `#,##0.00` ทำตอนแสดงผลในเว็บอย่างเดียว ค่าจริงไม่เคยผ่าน float เลยตั้งแต่ฐานข้อมูลจนถึงฟอร์ม

เปิดประกอบ: `salaryRule` ใน `employee-rules.ts` (`apps/api/src/employees/employee-rules.ts`), `salary.ts` (`apps/web/src/lib/salary.ts`), D-04

ทำไมตัด Login และ AI report ออก

โจทย์หลักคือ List, Search/Filter และ CRUD ของข้อมูลจาก Excel ระบบ Login แบบ Google OAuth และรายงาน AI ผ่าน n8n + Gemini อยู่ใน PRD ตั้งต้น และเคยทำไว้แล้ว แต่ตัดสินใจตัดออกเพื่อให้โปรเจกต์เรียบง่ายขึ้นและตรงโจทย์ บันทึกลงเป็น D-46 พร้อม migration ที่ลบตารางที่เกี่ยวข้อง และเปลี่ยน rate limit จากต่อ session เป็นต่อ IP

เสริมได้ถ้าตรงกับเหตุผลจริงของคุณ (ข้อนี้ไม่ได้บันทึกไว้ใน D-46): สองระบบนี้ต้องใช้บัญชี Google และ Gemini API key ภายนอก เมื่อตัดออกแล้วเดโมได้ในเครื่องทั้งหมด

เปิดประกอบ: D-46 ใน `decisions.md` (`docs/decisions.md`), `migration remove_login_and_reports` (`apps/api/prisma/migrations/20261003000000_remove_login_and_reports/migration.sql`)

ทำไม seed ไม่ stamp วันที่ใหม่

Last Updated Date ใน Excel เป็นข้อมูลตั้งต้นของโจทย ถ้า seed เปลี่ยนเป็นวันที่รัน ข้อมูลจะไม่ตรงต้นฉบับ PRD §4.3 ระบุไว้ว่าต้องคง ID และวันที่แก้ไขเดิมของทั้ง 5 รายการ seed จึงใส่ค่าจาก Excel ตรง ๆ และเติมเฉพาะ ID ที่ยังไม่มี ไม่กับข้อมูลที่ถูกแก้ไขแล้ว วันที่จะเปลี่ยนเป็น "วันนี้ตามเวลากรุงเทพ" ก็ต่อเมื่อผู้ใช้สร้างหรือแก้ไข record ผ่านระบบเท่านั้น

เปิดประกอบ: `seed-original.ts` (`apps/api/src/seed/seed-original.ts`), `seed-mapping.test.ts` (`apps/api/test/unit/seed-mapping.test.ts`), D-02

ป้องกัน lost update อย่างไร

ใช้ optimistic concurrency ทุกแถวมี `version` ตอนอ่าน API ส่ง `ETag` เป็นเลข version ตอนแก้ไขหรือลบ client ต้องส่ง `If-Match` กลับมา ไม่ส่งได้ 428 ส่ง version เก่าได้ 409 `VERSION_CONFLICT` ฝั่ง service ทำใน transaction ล็อกแถวด้วย `SELECT ... FOR UPDATE` แล้ว UPDATE แบบมีเงื่อนไข `WHERE version = expected` อีกชั้น มี E2E test เปิดสองแท็บแก้ไข record เดียวกัน แท็บที่ช้ากว่าต้องได้ข้อความให้ reload (AC-16)

เปิดประกอบ: `update()` ใน `employees.service.ts` (`apps/api/src/employees/employees.service.ts`), un 7 หัวข้อ 2 (`docs/learn/07-key-concepts.md`)

ใช้หลักฐานอะไรตัดสินใจเพิ่ม index

ดูจาก EXPLAIN ANALYZE (คำสั่งของ PostgreSQL ที่บอกว่า query อ่านทุกแถวหรือใช้ index) ไม่ได้เดาเอา index มีสองตัวอยู่ใน migration แยก ตัวแรกเป็น B-tree บน `(department_id, is_active)` ใช้กับ filter แผนก+สถานะและ query นับจำนวน EXPLAIN แสดงว่าเปลี่ยนจากอ่านทุกแถวเป็นใช้ index ตัวที่สองเป็น trigram GIN index บน `lower(name)` เพราะการค้นหาชื่อแบบ `LIKE '%...%'` ใช้ B-tree ไม่ได้ (ช่วยเมื่อคำค้นยาว 3 ตัวอักษรขึ้นไป) ตอนข้อมูลยังน้อย PostgreSQL อาจยังไม่เลือกใช้ จะเห็นผลเมื่อข้อมูลโตขึ้น migration นี้เพิ่ม index อย่างเดียว image เก่ายังใช้ได้ จึง rollback ได้ปลอดภัย ส่วนชุด performance test ถูกตัดออกภายหลังเพื่อความเรียบง่าย (D-52) แต่ index ยังอยู่

เปิดประกอบ: `perf_indexes migration` (`apps/api/prisma/migrations/20261002000000_perf_indexes/migration.sql`), D-32, D-52

AI ทำอะไรผิดจริงบ้าง

มีบันทึกไว้ 21 ข้อ ตัวอย่างที่เล่าง่าย:

1. จะติดตั้ง Prisma 8 RC เพราะ `latest` ชี้ไป RC ต้อง pin 7.10.0
2. รับสคริปต์ OpenAPI ด้วย `tsx` แล้ว Nest DI พัง เพราะ esbuild ไม่สร้าง decorator metadata
3. ช่อง Salary format ตอน focus ทำให้พิมพ์ใหม่แล้วข้อความต่อท้าย `62000.0063000.00` Playwright จับได้
4. พอร์มสร้างพนักงานสุ่ม Idempotency-Key ใหม่เมื่อได้ 5xx ซึ่งอาจทำให้สร้างซ้ำ แก้ให้เก็บ key เดิมเมื่อไม่รู้ผล
5. reviewer subagent จับได้ว่า app role มีสิทธิ์สร้างฐานข้อมูล จึงแยก test role

บทเรียนคือให้ test เป็นตัวตัดสิน ไม่เชื่อแค่การอ่านโค้ดหรือคำอธิบายของ AI

เปิดประกอบ: `ai-usage.md` (`docs/ai-usage.md`)

คำถามพื้นฐานที่อาจโดนถาม

NestJS ต่างจาก Express อย่างไร

Nest ใช้ Express อยู่ข้างใน แต่เพิ่มโครงสร้าง: Module รวมของที่เกี่ยวข้อง, Controller รับ HTTP, Service ทำงานจริง และมี Dependency Injection สร้างของให้ นอกจากนี้มีจุดเสียบมาตรฐานสำหรับงานที่ต้องทำกับทุก request คือ Guard (rate limit), Pipe (validation), Interceptor (ห่อ response) และ Exception Filter (แปลง error) โค้ดจึงไม่กองรวมในที่เดียว และทดสอบแต่ละชั้นได้

Dependency Injection มีประโยชน์อะไรในโปรเจกต์นี้

ตัวอย่างชัดที่สุดคือ `Clock` service ไม่ได้เรียก `new Date()` เอง แต่ขอ `Clock` ผ่าน constructor ตอนรันจริงได้ `SystemClock` ตอน test ได้ `FakeClock` ที่ตรึงเวลาไว้ ทำให้ทดสอบ "Last Updated Date = วันที่ตามเวลากรุงเทพ" และกรณีใกล้เคียงกันได้แน่นอนทุกครั้ง

เปิดประกอบ: `clock.ts` (`apps/api/src/common/clock.ts`), `harness.ts` (`apps/api/test/support/harness.ts`)

OpenAPI ใช้ทำอะไร

เป็นเอกสารสัญญาของ API ที่สร้างจาก decorator ในโค้ด ไม่ได้เขียนมือ คำสั่ง `pnpm openapi:generate` สร้าง `openapi.json` แล้วแปลงเป็น TypeScript type ให้เว็บ import ใช้ ถ้า API เปลี่ยน type ของเว็บก็เปลี่ยนตามและ typecheck จับได้ทันที Jenkins ยัง generate ซ้ำแล้วเทียบกับที่ commit ไว้ ถ้าไม่ตรง pipeline ล้ม จึงไม่มีทางที่เอกสารกับโค้ดหลุดกัน และเปิดดูแบบโต้ตอบได้ที่ `/api/docs`

Jenkins pipeline มีกี่ขั้น ทำอะไรบ้าง

Checkout → ติดตั้ง dependency → static checks (lint, typecheck, scan secret, ตรวจ OpenAPI drift) → unit test → เปิดฐานข้อมูลเฉพาะ build → integration test ของ API → build production ครั้งเดียวพร้อม E2E → Postman ใช้ build เดิม → build Docker image ติด tag เป็น commit SHA → ถ้าเป็น main ก็ deploy staging ซึ่งรัน smoke test ในตัว ถ้า smoke ไม่ผ่านจะ rollback ไป image ก่อนหน้าอัตโนมัติ ทุกขั้นเรียกคำสั่ง `pnpm` เดียวกับที่รันในเครื่อง

เปิดประกอบ: Jenkinsfile, un 6 (docs/learn/06-jenkins.md)

ทำไม image tag เป็น commit SHA ไม่ใช่ `latest`

ดู tag แล้วรู้กันที่ว่า staging รันโค้ดจาก commit ไหน และ rollback ได้ด้วยการสลับกลับไป tag ก่อนหน้า ถ้าใช้ `latest` ทุก build จะทับกัน ไม่รู้ว่าตัวที่รันอยู่คือรุ่นไหน ถ้า build ตอนแก้ไฟล์ที่ git track อยู่แต่ยังไม่ commit tag จะเป็น `<sha>-dirty` เพื่อไม่ให้อ้างว่าเป็น commit นั้นตรง ๆ (D-30)

rollback แล้วฐานข้อมูลย้อนด้วยไหม

ไม่ย้อน migration ออกแบบให้เป็นแบบ expand คือเพิ่มอย่างเดียว image เรายังทำงานกับ schema ใหม่ได้ ข้อ ยกเว้นคือ migration ที่ลบตาราง Login/Reports ถ้าจะย้อนไป image ที่เก่ากว่านั้นต้อง restore backup ที่ทำไว้ ก่อน migrate ซึ่ง `staging:up` ทำ `pg_dump` ให้ทุกครั้ง

เปิดประกอบ: runbook.md (docs/runbook.md) หัวข้อ 4

กดสร้างแล้วเน็ตหลุด จะได้ข้อมูลซ้ำไหม

ไม่ซ้ำ ทุกการสร้างส่ง `Idempotency-Key` เป็น UUID ที่สุ่มครั้งเดียวต่อความตั้งใจ server เก็บ key กับผลลัพธ์ไว้ใน transaction เดียวกับการสร้าง ถ้าส่งซ้ำด้วย key เดิม server ส่งผลเดิมกลับพร้อม `Idempotency-Replayed: true` ฝั่งเว็บเก็บ key เดิมไว้เมื่อไม่รู้จักผล (network error หรือ 5xx) มีทั้ง integration test ยิงพร้อมกัน 5 ครั้ง และ E2E test จำลอง response หาย (AC-20)

ป้องกัน SQL injection อย่างไร ในเมื่อใช้ `$queryRaw`

`$queryRaw` ของ Prisma เป็น tagged template ค่าใน `{ }` ถูกส่งเป็น parameter ไม่ได้ต่อเป็น string ส่วน คอลัมน์ที่ใช้เรียง (ซึ่งเป็น parameter ไม่ได้) เลือกจาก whitelist `SORT_SQL` เท่านั้น และคำค้นหาชื่อ escape `%` กับ `_` ก่อนใส่ `LIKE` มี Postman test ยืนยันว่าค้น `%` แล้วได้ 0 แถว ไม่ใช่ทุกแถว (AC-23)

test ไปลบข้อมูล dev ได้ไหม

ไม่ได้ ทุก runner สร้างฐานข้อมูลชั่วคราวของตัวเองด้วย test role ที่สร้างได้เฉพาะฐานชื่อ

`employee_console_test_*` ก่อนลบข้อมูลยังตรวจป้าย `app_meta.database_purpose` ต้องเป็น `test`

ส่วน app role ที่แอปใช้สร้างหรือลบฐานข้อมูลไม่ได้เลย ใน Jenkins แต่ละ build ยังมี Compose project ของตัวเองที่ลบทิ้งหลังจบ

ถ้าโดนถามสิ่งที่ไม่รู้

- บอกตรง ๆ ว่าส่วนนั้นยังไม่ได้ลงลึก แล้วชี้ว่าจะหาคำตอบจากที่ไหน เช่น "การตัดสินใจนี้บันทึกไว้ใน decisions.md ขอเปิดดูครับ"
- เปิดโค้ดจริงแทนการเดา ทุกเรื่องในคู่มือนี้มีลิงก์ไปไฟล์
- ใช้ test เป็นหลักฐาน: ค้นหาคำ `AC-xx` แล้วเปิด test ที่เกี่ยวข้องให้ดู

กลับไป: สารบัญ ([docs/learn/README.md](#))

อภิธานศัพท์

เรียงตามหมวด แต่ละคำบอกความหมายสั้น ๆ และบทที่อธิบายละเอียด

เว็บและ HTTP

คำ	ความหมาย	un
Request / Response	คำขอที่ส่งไป และคำตอบที่ได้กลับ ประกอบด้วย method, path, header, body / status code, header, body	1 (docs/learn/01-big-picture.md)
Method	คำกริยาของ HTTP: GET อ่าน, POST สร้าง, PATCH แก้บางส่วน, DELETE ลบ	1 (docs/learn/01-big-picture.md)
Status code	ตัวเลขสามหลักบอกผล เช่น 200 สำเร็จ, 404 ไม่พบ, 409 ชนกัน	1 (docs/learn/01-big-picture.md)
Header	ข้อมูลประกอบ request/response เช่น Content-Type , If-Match , ETag	1 (docs/learn/01-big-picture.md)
JSON	รูปแบบข้อมูลที่ส่งไปมา หน้าตาเหมือน object ของ JavaScript	1 (docs/learn/01-big-picture.md)
REST	แนวคิดที่มองข้อมูลเป็น resource มี URL ของตัวเอง แล้วใช้ method บอกการกระทำ	1 (docs/learn/01-big-picture.md)
Origin	scheme + host + port เช่น http://localhost:3000 เบราวเซอร์แยกสิทธิ์ตาม origin	1 (docs/learn/01-big-picture.md)
CORS	กลไกที่อนุญาตให้หน้าเว็บเรียก origin อื่น โปรเจกต์นี้ไม่เปิดเพราะใช้ origin เดียว	1 (docs/learn/01-big-picture.md)
Rewrite / Proxy	Next.js รับคำขอ /api/* แล้วส่งต่อให้ NestJS โดยเบราวเซอร์ไม่รู้	1 (docs/learn/01-big-picture.md)
ETag	header ที่บอก "รุ่น" ของข้อมูล ในโปรเจกต์นี้คือเลข version เช่น "2"	7 (docs/learn/07-key-concepts.md)
If-Match	header ที่ client ส่ง version ที่ตัวเองเห็นกลับมา เพื่อให้ server ตรวจสอบแก้ไข	7 (docs/learn/07-key-concepts.md)
Idempotency-Key	UUID ที่ติดกับคำขอสร้าง ส่งซ้ำด้วย key เดิมจะไม่สร้างซ้ำ	7 (docs/learn/07-key-concepts.md)
UUID	รหัสสุ่มยาว 128 bit ที่แทบไม่มีทางซ้ำ เช่น 7c9e6679-7425-40de-944b-e07fc1f90ae7	7 (docs/learn/07-key-concepts.md)
Rate limit	จำกัดจำนวนคำขอต่อช่วงเวลา เกินแล้วได้ 429	7 (docs/learn/07-key-concepts.md)

NestJS

คำ	ความหมาย	บท
Decorator	ป้าย <code>@...</code> ที่แปะบน class/method/property บอก framework ว่าโค้ดนั้นคืออะไร	2 (docs/learn/02-nestjs.md)
Module	กล่องที่รวม controller และ provider ของเรื่องเดียวกัน	2 (docs/learn/02-nestjs.md)
Controller	class ที่รับ HTTP request แทะข้อมูล แล้วเรียก service	2 (docs/learn/02-nestjs.md)
Provider / Service	class ที่ Nest สร้างแล้วแจกจ่ายผ่าน DI; service คือที่อยู่ของกฎธุรกิจ	2 (docs/learn/02-nestjs.md)
Dependency Injection (DI)	ขอมของที่ต้องใช้ผ่าน constructor แล้ว Nest สร้างและส่งให้	2 (docs/learn/02-nestjs.md)
Token	"ชื่อ" ที่ใช้ขอของจาก DI เช่น <code>APP_CONFIG</code>	2 (docs/learn/02-nestjs.md)
<code>@Global()</code>	module ที่ทุก module ใช้ของได้โดยไม่ต้อง import	2 (docs/learn/02-nestjs.md)
Dynamic module	module ที่รับค่าตอนสร้าง เช่น <code>AppModule.register(config, clock, logger)</code>	2 (docs/learn/02-nestjs.md)
Middleware	ฟังก์ชันที่กำกับทุก request ก่อนเข้า Nest เช่น ใส่ request ID	2 (docs/learn/02-nestjs.md)
Guard	ตัดสินใจว่า request ผ่านได้ไหม เช่น <code>RateLimitGuard</code>	2 (docs/learn/02-nestjs.md)
Pipe	ตรวจสอบและแปลงข้อมูลก่อนถึง controller เช่น <code>ValidationPipe</code>	2 (docs/learn/02-nestjs.md)
Interceptor	ทำงานรอบ ๆ handler ห่อผลลัพธ์ เช่น <code>EnvelopeInterceptor</code>	2 (docs/learn/02-nestjs.md)
Exception Filter	แปลง error เป็น response มาตรฐาน เช่น <code>HttpExceptionFilter</code>	2 (docs/learn/02-nestjs.md)
DTO	class ที่นิยามหน้าตาของ body/query เช่น <code>CreateEmployeeDto</code>	2 (docs/learn/02-nestjs.md)
<code>@Rule()</code>	decorator ของโปรเจกต์ ผูก pure function ที่ตรวจสอบฟิลด์เข้ากับ DTO	2 (docs/learn/02-nestjs.md)
Decorator metadata	ข้อมูลชนิดของ constructor ที่ TypeScript ฝังไว้ให้ DI อ่าน esbuild/tsx ไม่สร้างให้	2 (docs/learn/02-nestjs.md)
Envelope	รูปแบบห่อ response: <code>{ data, meta }</code> หรือ <code>{ error }</code>	7 (docs/learn/07-key-concepts.md)

ฐานข้อมูล

คำ	ความหมาย	un
ORM	เครื่องมือที่ให้เขียนโค้ดแทน SQL พร้อม type (โปรเจกต์นี้ใช้ Prisma)	3 (docs/learn/03-database-prisma.md)
Schema (schema.prisma)	ไฟล์นิยามตารางและคอลัมน์	3 (docs/learn/03-database-prisma.md)
Migration	ไฟล์ SQL ที่เปลี่ยนโครงสร้างฐานข้อมูลทีละขั้น เพิ่มได้อย่างเดียว	3 (docs/learn/03-database-prisma.md)
Seed	การใส่ข้อมูลตั้งต้น (5 records จาก Excel)	3 (docs/learn/03-database-prisma.md)
Transaction	ชุดคำสั่งที่สำเร็จทั้งหมดหรือไม่เกิดอะไรเลย	3 (docs/learn/03-database-prisma.md)
Row lock (FOR UPDATE)	ล็อกแถวไว้จนจบ transaction คนอื่นที่จะแก้แถวเดียวกันต้องรอ	3 (docs/learn/03-database-prisma.md)
Index	โครงสร้างช่วยค้นหาเร็ว เหมือนสารบัญท้ายหนังสือ	3 (docs/learn/03-database-prisma.md)
Trigram index	index ที่ช่วยค้นหาข้อความแบบ "มีคำนี้อยู่ข้างใน" (LIKE '%...%')	3 (docs/learn/03-database-prisma.md)
numeric(12,2)	ชนิดตัวเลขทศนิยมแม่นยำ 12 หลัก ทศนิยม 2 ตำแหน่ง	7 (docs/learn/07-key-concepts.md)
Date-only	วันที่ที่ไม่มีเวลาและ timezone เช่น 2023-01-15	7 (docs/learn/07-key-concepts.md)
Role	ผู้ใช้ของฐานข้อมูล โปรเจกต์แยก app role กับ test role	3 (docs/learn/03-database-prisma.md)
SQL injection	การโจมตีด้วยการแทรก SQL ผ่านคำที่ผู้ใช้ส่งมา ป้องกันด้วย parameter	3 (docs/learn/03-database-prisma.md)

OpenAPI

คำ	ความหมาย	un
OpenAPI	มาตรฐานเอกสาร API เป็น JSON/YAML บอกรoute, path, parameter, body, response	4 (docs/learn/04-openapi.md)
Swagger	ชื่อเดิมของ OpenAPI และชุดเครื่องมือ เช่น Swagger UI	4 (docs/learn/04-openapi.md)
Swagger UI	หน้าเว็บที่แสดงเอกสาร API และกดลองยิงได้ (/api/docs)	4 (docs/learn/04-openapi.md)
Schema (OpenAPI)	นิยามหน้าตาของข้อมูลชนิดหนึ่ง เช่น EmployeeDto	4 (docs/learn/04-openapi.md)
openapi-typescript	เครื่องมือแปลง openapi.json เป็น TypeScript type (schema.d.ts)	4 (docs/learn/04-openapi.md)
Drift	เอกสารหรือ type ที่ commit ไว้ไม่ตรงกับโค้ดปัจจุบัน	4 (docs/learn/04-openapi.md)

Docker

คำ	ความหมาย	บท
Image	แม่พิมพ์ของแอปที่แพ็คเกจพร้อมรัน แก่ข้างในไม่ได้	5 (docs/learn/05-docker.md)
Container	image ที่กำลังรันอยู่	5 (docs/learn/05-docker.md)
Volume	ที่เก็บข้อมูลถาวรนอก container	5 (docs/learn/05-docker.md)
Compose	ไฟล์ YAML ที่บอกว่าจะเปิด container อะไร ต่อกันอย่างไร	5 (docs/learn/05-docker.md)
Dockerfile	สูตรสร้าง image	5 (docs/learn/05-docker.md)
Multi-stage build	Dockerfile ที่แบ่งช่วง build กับช่วง runtime ให้ image สุดท้ายเล็ก	5 (docs/learn/05-docker.md)
Tag	ชื่อรุ่นของ image ในโปรเจกต์นี้คือ commit SHA	5 (docs/learn/05-docker.md)
Healthcheck	คำสั่งที่ Docker ใช้ตรวจว่า container พร้อมหรือยัง	5 (docs/learn/05-docker.md)
Profile (Compose)	กลุ่ม service ที่เปิดเฉพาะเมื่อระบุ เช่น <code>ci</code> สำหรับ Jenkins	5 (docs/learn/05-docker.md)

CI/CD และ Jenkins

คำ	ความหมาย	บท
CI	ตรวจโค้ดอัตโนมัติทุกครั้งที่มีโค้ดใหม่	6 (docs/learn/06-jenkins.md)
CD	ส่งโค้ดที่ผ่านแล้วขึ้น environment อัตโนมัติ	6 (docs/learn/06-jenkins.md)
Pipeline	ชุดขั้นตอน CI/CD ทั้งหมด เขียนใน <code>Jenkinsfile</code>	6 (docs/learn/06-jenkins.md)
Stage / Step	ด้านหนึ่งใน pipeline / คำสั่งย่อยในด้าน	6 (docs/learn/06-jenkins.md)
Controller	ตัวกลางของ Jenkins: หน้าเว็บ, job, ประวัติ	6 (docs/learn/06-jenkins.md)
Agent	เครื่องที่รันคำสั่งจริง (<code>host-agent</code>)	6 (docs/learn/06-jenkins.md)
Build	การรัน pipeline หนึ่งครั้ง	6 (docs/learn/06-jenkins.md)
Artifact	ไฟล์ผลลัพธ์ที่เก็บกับ build	6 (docs/learn/06-jenkins.md)
Credentials	ความลับที่ Jenkins เก็บแทนการเขียนในโค้ด	6 (docs/learn/06-jenkins.md)
JCasC	ตั้งค่า Jenkins ด้วยไฟล์ YAML (<code>casc.yaml</code>)	6 (docs/learn/06-jenkins.md)
Staging	environment ที่จำลอง production ไว้ทดสอบก่อนใช้จริง (:3100)	5 (docs/learn/05-docker.md)
Smoke test	การตรวจเร็ว ๆ หลัง deploy ว่าระบบยังใช้งานได้	5 (docs/learn/05-docker.md)
Rollback	ย้อนกลับไปรุ่นก่อนหน้าเมื่อรุ่นใหม่มีปัญหา	5 (docs/learn/05-docker.md)
Manifest	ไฟล์บันทึกว่า staging รัน tag ไหน ก่อนหน้าคืออะไร	5 (docs/learn/05-docker.md)

การทดสอบ

คำ	ความหมาย	un
Unit test	ทดสอบฟังก์ชันเล็ก ๆ ไม่ใช่ฐานข้อมูล	8 (docs/learn/08-testing.md)
Integration test	ทดสอบ API ทั้งก่อนกับฐานข้อมูลจริง	8 (docs/learn/08-testing.md)
Contract test	ทดสอบว่า API ตอบตามสัญญา (status, header, error code)	8 (docs/learn/08-testing.md)
E2E test	ทดสอบผ่านเบราว์เซอร์จริงเหมือนผู้ใช้	8 (docs/learn/08-testing.md)
Vitest	ตัวรัน test ของ JavaScript/TypeScript	8 (docs/learn/08-testing.md)
Supertest	library ยิง HTTP เข้าแอป Nest ใน test	8 (docs/learn/08-testing.md)
Postman / Newman	โปรแกรมยิง API / ตัวรัน collection ของ Postman จาก command line	8 (docs/learn/08-testing.md)
Playwright	เครื่องมือควบคุมเบราว์เซอร์สำหรับ E2E	8 (docs/learn/08-testing.md)
FakeClock	นาฬิกาปลอมที่ตรงเวลาไว้ใช้ใน test	2 (docs/learn/02-nestjs.md)
AC-xx	สหส acceptance criteria ใน PRD ที่ชื่อ test อ้างถึง	8 (docs/learn/08-testing.md)

เฉพาะโปรเจกต์นี้

คำ	ความหมาย	un
D-xx	รหัสการตัดสินใจใน decisions.md (docs/decisions.md)	9 (docs/learn/09-docs-guide.md)
PRD Sx.y	อ้างถึงหัวข้อใน prd.md (docs/prd.md)	9 (docs/learn/09-docs-guide.md)
APP_ENV	ตัวแปรบอกว่ารันใน environment ไหน (local, staging, test)	1 (docs/learn/01-big-picture.md)
database_purpose	ป้ายในตาราง app_meta บอกว่าฐานนี้ใช้ทำอะไร คำสั่งลบข้อมูลตรวจสอบก่อนทุกครั้ง	3 (docs/learn/03-database-prisma.md)
Business date	"วันนี้" ตามเขตเวลา Asia/Bangkok ใช้กับ Last Updated Date	7 (docs/learn/07-key-concepts.md)
No-op update	กด Save โดยไม่ได้เปลี่ยนค่า ระบบไม่เขียนฐานข้อมูล	7 (docs/learn/07-key-concepts.md)
-dirty tag	tag ของ image ที่ build ตอนแก้ไฟล์ที่ git track อยู่แต่ยังไม่ commit (ไฟล์ใหม่ที่ยังไม่ add ไม่นับ)	5 (docs/learn/05-docker.md)

กลับไป: สารบัญ (docs/learn/README.md)